

TRABAJO DE FIN DE GRADO

Grado en Ingeniería Informática

ESCUELA POLITÉCNICA SUPERIOR

Búsqueda en tiempo mínimo en espacios con
incertidumbre mediante algoritmos bioinspirados

Romeo Bernal Alcelay

Tutor: Juan Pedro Llerena

2026

Tribunal:

Presidente: Dr. Javier Arroyo

Vocal: Dr. Adrián Domínguez

Suplente: Dr. Luis Usero

Fecha de depósito: 9 de Febrero de 2026

Agradecimientos

Este trabajo de fin de grado ha sido posible gracias al apoyo y la compañía de muchas personas que han formado parte de mi vida durante estos años.

En primer lugar, quiero agradecer a mis compañeros de clase, con quienes además de compartir frustraciones, dudas y horas de estudio, también he compartido risas, aprendizajes y momentos que recordaré siempre.

A mis amigos, gracias por compartir parte de vuestras vidas conmigo y por hacer que este camino haya sido más ameno, más llevadero y mucho más divertido.

A mis padres, que han sido el pilar más importante en todo este recorrido. Gracias por enseñarme la importancia del esfuerzo y la constancia, por creer en mí desde el primer momento y por apoyarme en cada paso, incluso en aquellos que no han sido fáciles.

A mis profesores, gracias no solo por transmitirme conocimientos, sino también por vuestra dedicación, paciencia y la pasión por aprender que conseguís contagiar. Gracias por despertar en mí curiosidad por el mundo que nos rodea y por guiarme durante esta etapa.

Finalmente, quiero agradecer a todas aquellas personas que, de una manera u otra, han formado parte de este camino: quienes me ofrecieron un consejo, una motivación extra o simplemente un momento de desconexión cuando más lo necesitaba.

Resumen

Este trabajo aborda el problema de búsqueda en tiempo mínimo en espacios con incertidumbre, aplicado a misiones de búsqueda y rescate mediante vehículos aéreos no tripulados (*Unmanned Aerial Vehicles*, UAVs). El objetivo principal consiste en maximizar la probabilidad de detectar objetivos en el menor tiempo posible, partiendo de información inicial limitada y considerando las restricciones operativas de los UAVs.

La naturaleza del problema lo clasifica como *NP-Hard*, lo que implica una elevada complejidad computacional. Para abordar este desafío, se propone el uso de técnicas de optimización bioinspiradas, concretamente tres algoritmos de gran relevancia actual: optimización por colonias de hormigas (*Ant Colony Optimization*, ACO), colonia artificial de abejas (*Artificial Bee Colony*, ABC) y algoritmo de agujero negro (*Black Hole Algorithm*, BHA). Estos algoritmos se adaptan al problema específico de búsqueda y se implementan mediante un framework desarrollado en Python, que permite evaluar su comportamiento en diversos escenarios, incluyendo aquellos que ponen a prueba las limitaciones de las técnicas clásicas.

Adicionalmente, se analiza el desempeño de las trayectorias generadas empleando diferentes funciones objetivo presentes en la literatura, como el tiempo esperado (*Expected Time*, ET), la recompensa de tiempo con descuento (*Discounted Time Reward*, DTR), la pendiente máxima (*Maximum Slope*, MS) y la entropía mínima (*Minimum Entropy*, ME).

La evaluación de los algoritmos bioinspirados se realiza en comparación con técnicas tradicionales, incluyendo estrategias de búsqueda exhaustiva (algoritmo de cortacésped y expansión cuadrada) y búsqueda local (algoritmo voraz miope y voraz con heurística). El análisis se basa en métricas como la distancia recorrida, el número de pasos ejecutados y la tasa de éxito en la detección del objetivo. Las pruebas se llevan a cabo en distintos escenarios configurables, variando el número de indicios, el número de agentes y las posiciones iniciales de cada elemento, lo que permite obtener una evaluación exhaustiva del rendimiento de las diferentes estrategias propuestas.

Palabras clave: Búsqueda probabilística, Búsqueda en tiempo mínimo, Algoritmos bioinspirados, Metaheurísticas, Optimización.

Abstract

This work addresses the minimum time search problem in uncertain environments, applied to search and rescue missions using Unmanned Aerial Vehicles (UAVs). The main objective is to maximize the probability of detecting targets in the shortest possible time, starting from limited initial information and considering the operational constraints of UAVs.

The nature of the problem classifies it as NP-Hard, which implies high computational complexity. To tackle this challenge, the use of bio-inspired optimization techniques is proposed, specifically three algorithms of current relevance: Ant Colony Optimization (ACO), Artificial Bee Colony (ABC), and Black Hole Algorithm (BHA). These algorithms are adapted to the specific search problem and implemented through a framework developed in Python, which allows evaluating their behavior in various scenarios, including those that test the limitations of classical techniques.

Additionally, the performance of the generated trajectories is analyzed using different objective functions from the literature, such as Expected Time (ET), Discounted Time Reward (DTR), Maximum Slope (MS), and Minimum Entropy (ME).

The evaluation of bio-inspired algorithms is conducted in comparison with traditional techniques, including exhaustive search strategies (lawnmower algorithm and square expansion) and local search (myopic greedy algorithm and greedy with heuristic). The analysis is based on metrics such as distance traveled, number of steps executed, and success rate in target detection. Tests are carried out in different configurable scenarios, varying the number of clues, number of agents, and initial positions of each element, which allows obtaining a comprehensive evaluation of the performance of the different proposed strategies.

Keywords: Probabilistic search, Minimum time search, Bio-inspired algorithms, Metaheuristics, Optimization.

Índice general

Agradecimientos	v
Resumen	vii
Abstract	ix
Índice general	xi
Índice de figuras	xv
Índice de tablas	xix
Lista de símbolos	xix
1 Introducción	1
1.1 Motivación	1
1.2 Objetivos	2
1.3 Planteamiento	2
1.4 Organización del manuscrito	3
2 Estado del arte	5
2.1 Búsqueda probabilística	5
2.1.1 Antecedentes históricos	6
2.1.2 Heurística en búsqueda probabilística	7
2.2 Problemas NP-Hard	7
2.2.1 Complejidad del problema de búsqueda probabilística	8
2.2.2 Justificación del uso de algoritmos aproximados	8
2.3 Búsqueda en tiempo mínimo y problemas relacionados	9
2.3.1 Relación con problemas clásicos de optimización	9
2.3.2 Contribuciones recientes en MTS	9
2.4 Técnicas clásicas de búsqueda	10
2.4.1 Búsquedas exhaustivas	10
2.4.1.1 Algoritmo de cortacésped (Lawnmower Search)	10
2.4.1.1.1 Ventajas y limitaciones	10
2.4.1.2 Algoritmo de expansión cuadrada (Expanding Square Search)	11
2.4.2 Búsquedas locales	11
2.4.2.1 Algoritmo voraz	11
2.4.2.1.1 Trabajos relevantes	12
3 Formulación del problema de búsqueda	13

3.1	Espacio de búsqueda	13
3.1.1	Generación del mapa de probabilidad inicial	14
3.2	Modelo de objetivo	14
3.3	Dinámica de los UAVs	15
3.4	Modelo de sensor	16
3.5	Fusión de información: Filtro recursivo bayesiano	16
3.6	Notación	17
4	Funciones objetivo en MTS	19
4.1	Expected Time (ET)	19
4.1.1	Ejemplo ET	20
4.2	Discounted Time Reward (DTR)	22
4.2.1	Ejemplo DTR	23
4.3	Maximum Slope (MS)	25
4.3.1	Ejemplo MS	25
4.4	Minimum Entropy (ME)	27
4.4.1	Ejemplo ME	29
5	Algoritmos bioinspirados	31
5.1	Ant Colony Optimization (ACO)	31
5.1.1	Principio de funcionamiento	31
5.1.2	Fases del algoritmo	32
5.1.3	Parámetros del algoritmo	33
5.2	Artificial Bee Colony (ABC)	33
5.2.1	Principio de funcionamiento	33
5.2.2	Fases del algoritmo	34
5.2.3	Parámetros del algoritmo	34
5.3	Black Hole Algorithm (BHA)	35
5.3.1	Principio de funcionamiento	35
5.3.2	Horizonte de eventos y exploración	36
5.3.3	Fases del algoritmo	36
5.3.4	Parámetros del algoritmo	37
5.4	Comparación entre algoritmos	37
6	Framework	39
6.1	Entorno de simulación	39
6.2	Parámetros de entrada de los algoritmos y métodos	41
6.3	Cuaderno de pruebas	43
6.4	Visualización de la salida del framework	44
6.5	Repositorio de GitHub	44
7	Experimentación	45
7.1	Pruebas básicas	45
7.1.1	Mapa 30×30	48
7.1.2	Mapa 50x50	54
7.1.3	Mapa 100x100	65
7.2	Pruebas masivas	70
7.2.1	Análisis de resultados	71

8 Conclusiones y líneas futuras	77
8.1 Conclusiones	77
8.2 Líneas futuras	78
Bibliografía	81

Índice de figuras

4.1	Matriz 3×3 utilizada para demostrar el funcionamiento del ET.	20
4.2	Evolución del Expected Time (ET) para la trayectoria A.	21
4.3	Evolución del Expected Time (ET) para la trayectoria B.	22
4.4	Evolución del DTR para la trayectoria A.	23
4.5	Evolución del DTR para la trayectoria B.	24
4.6	Evolución del MS para la trayectoria A.	26
4.7	Evolución del MS para la trayectoria B.	27
4.8	Evolución del ME para la trayectoria A.	29
4.9	Evolución del ME para la trayectoria B.	30
6.1	Diagrama de componentes del entorno de experimentación.	41
6.2	Visualización de la animación del entorno.	44
7.1	Mapa del entorno (30×30).	46
7.2	Mapas del entorno (50×50).	46
7.3	Mapa del entorno (100×100).	47
7.4	Trayectoria obtenida con el algoritmo voraz miope en el mapa 30×30	48
7.5	Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 30×30	48
7.6	Trayectoria obtenida con el algoritmo de cortacésped en el mapa 30×30	49
7.7	Trayectoria obtenida con el algoritmo de expansión cuadrada en el mapa 30×30	49
7.8	Resultados del algoritmo ACO para las cuatro funciones objetivo en el mapa 30×30 : trayectorias (izquierda) y sus gráficas asociadas (derecha).	50
7.9	Resultados del algoritmo ABC para las cuatro funciones objetivo en el mapa 30×30 : trayectorias (izquierda) y sus gráficas asociadas (derecha).	51
7.10	Resultados del algoritmo BHA para las cuatro funciones objetivo en el mapa 30×30 : trayectorias (izquierda) y sus gráficas asociadas (derecha).	52
7.11	Comparación de la evolución de las cuatro funciones objetivo con ACO, ABC y BHA para el mapa 30×30	53
7.12	Trayectoria obtenida con el algoritmo voraz miope en el mapa 50×50 con 1 indicio.	54

7.13	Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 50×50 con 1 indicio.	54
7.14	Trayectoria obtenida con el algoritmo de cortacésped en el mapa 50×50 con 1 indicio. . .	55
7.15	Trayectoria obtenida con el algoritmo de expansión cuadrada en el mapa 50×50 con 1 indicio.	55
7.16	Resultados del algoritmo ACO para las cuatro funciones objetivo en el mapa 50×50 con 1 indicio: trayectorias (izquierda) y sus gráficas asociadas (derecha).	56
7.17	Resultados del algoritmo ABC para las cuatro funciones objetivo en el mapa 50×50 con 1 indicio: trayectorias (izquierda) y sus gráficas asociadas (derecha).	57
7.18	Resultados del algoritmo BHA para las cuatro funciones objetivo en el mapa 50×50 con 1 indicio: trayectorias (izquierda) y sus gráficas asociadas (derecha).	58
7.19	Comparación de la evolución de las cuatro funciones objetivo con ACO, ABC y BHA para el mapa 50x50 con 1 indicio.	59
7.20	Trayectoria obtenida con el algoritmo voraz miope en el mapa 50×50 con 2 indicios. . . .	60
7.21	Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 50×50 con 2 indicios.	60
7.22	Trayectoria obtenida con el algoritmo de cortacesped en el mapa 50×50 con 2 indicios. . .	61
7.23	Trayectoria obtenida con el algoritmo de expansión cuadrada en el mapa 50×50 con 2 indicios.	61
7.24	Resultados del algoritmo ACO para las cuatro funciones objetivo en el mapa 50×50 con 2 indicios: trayectorias (izquierda) y sus gráficas asociadas (derecha).	62
7.25	Resultados del algoritmo ABC para las cuatro funciones objetivo en el mapa 50×50 con 2 indicios: trayectorias (izquierda) y sus gráficas asociadas (derecha).	63
7.26	Resultados del algoritmo BHA para las cuatro funciones objetivo en el mapa 50×50 con 2 indicios: trayectorias (izquierda) y sus gráficas asociadas (derecha).	64
7.27	Comparación de la evolución de las cuatro funciones objetivo con ACO, ABC y BHA para el mapa 50x50 con 2 indicio.	65
7.28	Trayectoria obtenida con el algoritmo voraz miope en el mapa 50×50 con 1 indicio. . . .	66
7.29	Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 100×100 con 10 indicios y 3 agentes.	66
7.30	Trayectoria obtenida con el algoritmo de cortacesped en el mapa 100×100 con 10 indicios y 3 agentes.	67
7.31	Trayectoria obtenida con el algoritmo de expansion cuadrada en el mapa 100×100 con 10 indicios y 3 agentes.	67
7.32	Trayectoria obtenida con el algoritmo ACO junto con la función ET en el mapa 100×100 con 10 indicios y 3 agentes.	68
7.33	Trayectoria obtenida con el algoritmo ABC junto con la función ET en el mapa 100×100 con 10 indicios y 3 agentes.	68
7.34	Trayectoria obtenida con el algoritmo BHA junto con la función ET en el mapa 100×100 con 10 indicios y 3 agentes.	69
7.35	Comparación de la evolución de ET con ACO, ABC y BHA para el mapa 100x100. . . .	69

7.36 Representación visual de los cuatro escenarios de pruebas masivas.	70
7.37 Diagrama de barras ordenado del promedio de pasos por algoritmo.	71
7.38 Diagrama de barras del promedio de pasos por algoritmo y número de agentes.	72
7.39 Diagrama de violín de los pasos promedio por algoritmo.	72
7.40 Diagrama de barras del promedio de pasos por algoritmo bioinspirado y función objetivo.	73
7.41 Diagrama de violín de los pasos promedio por algoritmo bioinspirado y función objetivo.	73
7.42 Conjunto de diagramas de cajas y bigotes de las distancias recorridas por algoritmo.	74
7.43 Conjunto de diagramas de cajas y bigotes de las distancias recorridas por algoritmo bioinspirado y función objetivo.	74
7.44 Diagrama de barras de tasa de éxito general por algoritmo.	75
7.45 Diagrama de barras de tasa de éxito general por algoritmo bioinspirado y función objetivo.	75

Índice de tablas

4.1	Cálculo manual de la evolución del ET para la trayectoria A.	21
4.2	Cálculo manual de la evolución del ET para la trayectoria B.	22
4.3	Cálculo manual del Discounted Time Reward (DTR) para la trayectoria A.	24
4.4	Cálculo manual del Discounted Time Reward (DTR) para la trayectoria B.	24
4.5	Cálculo manual del Maximum Slope (MS) para la trayectoria A.	26
4.6	Cálculo manual del Maximum Slope (MS) para la trayectoria B.	27
4.7	Cálculo manual del Minimum Entropy (ME) para la trayectoria A.	29
4.8	Cálculo manual del Minimum Entropy (ME) para la trayectoria B.	30
5.1	Parámetros de configuración de ACO.	33
5.2	Parámetros del algoritmo ABC.	35
5.3	Parámetros del algoritmo BHA.	37
6.1	Parámetros generales de configuración del entorno.	42
6.2	Parámetros del modelo probabilístico.	42
6.3	Parámetros de entrada comunes a los algoritmos de búsqueda.	43

Capítulo 1

Introducción

Este capítulo explica la motivación del trabajo, que es la de comparar el desempeño de diferentes algoritmos bioinspirados en problemas de búsqueda en tiempo mínimo en espacios con incertidumbre frente a los algoritmos tradicionales, y ver su viabilidad en este tipo de problemas. También, se explican los objetivos a alcanzar de este trabajo junto con los procedimientos que se aplican para alcanzarlos. Finalmente se expone la organización del manuscrito en los capítulos que lo componen.

1.1 Motivación

Este trabajo parte de la idea de que las metaheurísticas pueden ser potentes aliados para abordar los problemas de búsqueda en tiempo mínimo en espacios con incertidumbre. En particular, se pretende evaluar su rendimiento en tareas de localización de objetivos realizadas por vehículos aéreos no tripulados (UAVs).

En escenarios donde el tiempo es un factor crítico, encontrar el objetivo en el menor tiempo posible se convierte en la prioridad principal. Sin embargo, cuando la posición del objetivo es incierta y los equipos de búsqueda y rescate solo disponen de información parcial o de una serie de indicios, las técnicas tradicionales pueden presentar limitaciones significativas. Dentro de esta tipología de problemas, y en su aplicación a escenarios reales, se incluyen situaciones como naufragios, catástrofes naturales en las que es necesario localizar supervivientes en áreas extensas, o la búsqueda y recuperación de personas desaparecidas, por ejemplo, en entornos montañosos.

Las técnicas tradicionales presentan diversas limitaciones cuando se aplican a problemas de búsqueda en entornos reales. Por un lado, los métodos exhaustivos intentan explorar la totalidad del espacio de búsqueda; sin embargo, esta aproximación resulta inviable en escenarios de gran extensión, especialmente cuando el tiempo es un factor crítico, como ocurre en situaciones de supervivencia. Por otro lado, las técnicas voraces permiten reducir los tiempos de cómputo al no recorrer todo el espacio, ya que centran la exploración en regiones inmediatamente contiguas. No obstante, este enfoque puede ser problemático cuando las áreas vecinas presentan probabilidades similares y cercanas a cero, lo que dificulta la toma de decisiones y puede conducir a exploraciones poco eficientes.

Esta problemática está estrechamente relacionada con el denominado problema de la miopía, que describe la tendencia de estos algoritmos a basar sus decisiones únicamente en información local o inmediata, sin considerar las consecuencias globales o a largo plazo. Como resultado, los agentes encargados de la exploración pueden quedar atrapados en determinadas zonas del mapa, siendo incapaces de progresar hacia una exploración efectiva y completa del entorno.

Los algoritmos bioinspirados constituyen una alternativa sólida para abordar este tipo de problemas, ya que permiten un equilibrio eficaz entre la exploración y la explotación del espacio de soluciones. Esta característica los hace especialmente adecuados para escenarios complejos en los que los métodos tradicionales tienden a estancarse o no logran progresar de forma satisfactoria. Además, estas técnicas se fundamentan en principios observados en la naturaleza, el mayor banco de experimentos existente, respaldado por millones de años de evolución y adaptación.

No obstante, esta flexibilidad y capacidad de adaptación conllevan ciertas limitaciones, dado que los algoritmos bioinspirados suelen requerir un elevado número de iteraciones para converger hacia soluciones de calidad, lo que incrementa el coste computacional.

En este trabajo de fin de grado se estudian y comparan estos algoritmos frente a los tradicionales y se evalúa su aplicación en posibles entornos reales.

1.2 Objetivos

Los objetivos de este trabajo se enumeran a continuación y se formulan en infinitivo.

1. Analizar el desempeño de algoritmos bioinspirados en la resolución del problema de *búsqueda en tiempo mínimo (MTS, Minimum Time Search)* en entornos probabilísticos con incertidumbre.
2. Integrar los algoritmos de cortacésped, el de expansión cuadrada, el voraz miope, el voraz con heurística, el algoritmo de optimización de hormigas, el algoritmo de colonia de abejas y el algoritmo de agujero negro en un entorno de simulación desarrollado en *Python* que permita su ejecución, configuración y visualización gráfica sobre mapas de probabilidad.
3. Estudiar e implementar funciones objetivo destacadas de la literatura en problemas de MTS.
4. Diseñar un entorno de pruebas que permita parametrizar, ejecutar y comparar el rendimiento de los algoritmos bajo diferentes configuraciones de escenarios.
5. Evaluar y comparar los algoritmos bioinspirados frente a métodos tradicionales.

Todos los objetivos se desarrollan de forma coherente con la arquitectura general del entorno de simulación, garantizando la consistencia en la integración y el comportamiento del sistema.

1.3 Planteamiento

Los problemas de planificación pueden ser formulados como problemas de optimización que buscan minimizar/maximizar un criterio eligiendo las acciones correctas, en este caso la trayectoria que maximiza la probabilidad de encontrar a un objetivo minimizando el tiempo para hacerlo. En este escenario, el problema de búsqueda en tiempo mínimo puede ser expresado como un problema de planificación que busca minimizar el número de pasos hasta encontrar el objetivo. Para ello se hace uso de las diferentes funciones objetivo propuestas, que a través de su optimización se tratará de escoger las trayectorias que más ganancia de información proporcionen en el menor número de pasos, tratando así de encontrar al objetivo en el menor tiempo posible.

En el entorno simulado, los drones podrán moverse libremente por el mapa, y cada trayectoria dará como resultado un valor dependiente de la función objetivo utilizada. Los algoritmos bioinspirados tendrán

la función de optimizar dicho valor en cada iteración hasta encontrar el valor mínimo (o máximo) posible de estas funciones, con el fin de optimizar las trayectorias.

En el caso particular de los algoritmos bioinspirados, también se estudiará el comportamiento de la función objetivo a lo largo de las iteraciones, con el fin de observar su proceso de convergencia y determinar el punto en el que el valor de dicha función se estabiliza, indicando una solución cercana al óptimo encontrada por cada una de las estrategias propuestas.

1.4 Organización del manuscrito

El presente Trabajo de Fin de Grado se organiza en cinco capítulos, estructurados de forma que lleva desde la motivación y contextualización del problema hasta la presentación de los resultados y las conclusiones finales.

- **Capítulo 1 – Introducción.** En este capítulo se presenta el problema principal abordado en este trabajo, la Búsqueda en Tiempo Mínimo (*Minimum Time Search*, MTS) en espacios con incertidumbre. Se describen los objetivos generales y específicos de la tesis, así como la motivación y la metodología seguida para abordarlos.
- **Capítulo 2 – Estado del arte.** Este capítulo revisa el estado del arte de los distintos conceptos del trabajo. Se introduce el contexto de la búsqueda probabilística, se describen los problemas NP-Hard, la búsqueda en tiempo mínimo y problemas relacionados, y se presentan las técnicas clásicas de búsqueda, incluyendo búsquedas exhaustivas (cortacésped, expansión) y búsquedas locales (voraz, voraz con heurística), indicando sus limitaciones. Se finaliza el capítulo con unas conclusiones respecto al marco teórico del trabajo.
- **Capítulo 3 – Formulación del problema de búsqueda.** En este capítulo se presenta la formulación matemática del problema de búsqueda. Se describe el espacio de búsqueda, el modelo de objetivo, la dinámica de los UAVs, el modelo de sensor y la fusión de información mediante el filtro recursivo bayesiano. Se incluye además la notación utilizada a lo largo del trabajo.
- **Capítulo 4 – Funciones objetivo en MTS.** Este capítulo presenta las distintas funciones objetivo utilizadas para resolver el problema de búsqueda en tiempo mínimo. Se describen en detalle las funciones *Expected Time* (ET), *Discounted Time Reward* (DTR), *Maximum Slope* y *Minimum Entropy*, analizando sus características y aplicabilidad en el contexto del problema.
- **Capítulo 5 – Algoritmos bioinspirados.** En este capítulo se describen los algoritmos bioinspirados empleados en el estudio: *Ant Colony Optimization* (ACO), *Artificial Bee Colony* (ABC) y *Black Hole Algorithm* (BHA). Se presenta una introducción general a este tipo de algoritmos y se detallan las particularidades y parámetros que utilizan cada uno de ellos.
- **Capítulo 6 – Framework.** Este capítulo describe el desarrollo del framework implementado para la evaluación de las estrategias de búsqueda. Se presenta la estructura del repositorio de GitHub desarrollado y se detallan los distintos módulos y componentes que conforman el entorno de experimentación.
- **Capítulo 7 – Experimentación.** Este capítulo presenta y analiza los resultados obtenidos a lo largo del estudio y se estructura en dos secciones diferenciadas. La primera sección corresponde a las pruebas básicas, donde se analizan escenarios con un indicio y un agente, dos indicios y un agente, y múltiples agentes con múltiples indicios, evaluando las trayectorias generadas y la

convergencia de las funciones objetivo. La segunda sección presenta las pruebas masivas, donde se ejecuta un conjunto amplio de experimentos en distintos entornos con el objetivo de obtener métricas cuantitativas (tasa de acierto, distancia recorrida, etc.) y realizar un análisis comparativo de los resultados.

- **Capítulo 8 – Conclusiones y líneas futuras.** En este último capítulo se resumen las principales conclusiones derivadas del trabajo realizado y se proponen diversas líneas de trabajo futuro que permiten ampliar y continuar el estudio desarrollado.

Capítulo 2

Estado del arte

En este capítulo se presenta una revisión del estado del arte sobre el problema de la búsqueda probabilística y las técnicas empleadas para su resolución. Este tipo de problemas se caracteriza por la incertidumbre inherente en la localización de un objetivo dentro de un entorno de búsqueda, lo que requiere el uso de modelos probabilísticos y estrategias de planificación adaptativas.

La búsqueda probabilística se utiliza para resolver problemas en los que la localización de un objetivo es incierta y se representa mediante una distribución de probabilidad sobre el espacio de búsqueda. Cada posición del entorno posee una probabilidad asociada de contener el objetivo, y el propósito del agente o conjunto de agentes es diseñar una trayectoria de exploración que maximice la probabilidad acumulada de detección, minimice el tiempo esperado hasta el hallazgo, o ambas. El número de posibles secuencias de movimientos crece exponencialmente con el tamaño del espacio y el horizonte temporal considerado (número de pasos que puede realizar el agente), lo que convierte la búsqueda probabilística en un problema *NP-Hard*.

En este capítulo se describen los fundamentos teóricos que sustentan el trabajo desarrollado. En primer lugar, se introduce el concepto de búsqueda probabilística, su evolución histórica y los modelos matemáticos empleados para representar la incertidumbre. Posteriormente, se analiza la complejidad computacional del problema y su clasificación como *NP-Hard*, lo que justifica el uso de técnicas aproximadas en lugar de métodos exactos. A continuación, se presenta el problema de búsqueda en tiempo mínimo (MTS) y su relación con otros problemas clásicos de optimización. Finalmente, se describen las técnicas clásicas de búsqueda, tanto exhaustivas como locales, destacando sus ventajas, limitaciones y el problema de la miopía que motiva el desarrollo de enfoques más avanzados.

2.1 Búsqueda probabilística

La búsqueda probabilística constituye un paradigma fundamental en la resolución de problemas de localización bajo incertidumbre. Se basa en la representación de la incertidumbre mediante distribuciones de probabilidad que modelan la posible localización del objetivo. Estas probabilidades son actualizadas dinámicamente conforme se obtiene nueva información del entorno, lo que permite adaptar las trayectorias de los agentes en tiempo real.

Uno de los principales desafíos asociados a la búsqueda probabilística es el denominado *problema de la miopía*. Este fenómeno se produce cuando la estrategia de búsqueda se centra únicamente en maximizar el beneficio inmediato, por ejemplo, la probabilidad de detección en el siguiente paso sin considerar el efecto que dicha decisión tiene sobre el futuro del proceso de búsqueda. Este comportamiento

miopo conduce, en muchos casos, a trayectorias subóptimas que exploran de forma local las regiones de mayor probabilidad pero dejan sin cubrir áreas potencialmente relevantes del mapa, ocasionando que los algoritmos se estancuen en zonas del espacio donde la distribución de probabilidad es uniforme o presenta valores nulos.

2.1.1 Antecedentes históricos

La búsqueda probabilística tiene sus orígenes en la teoría de la búsqueda desarrollada durante la Segunda Guerra Mundial, cuando el grupo de investigación antisubmarinos de la Marina de los Estados Unidos (ASWORG) formuló los primeros métodos sistemáticos para localizar objetivos bajo incertidumbre. Las técnicas desarrolladas en este periodo (1942–1945) fueron recopiladas por Bernard Koopman en su influyente informe *Search and Screening* [1], considerado el punto de partida de la teoría moderna de la búsqueda.

En sus trabajos posteriores [2], Koopman sentó las bases matemáticas para modelar la detección de objetivos mediante distribuciones de probabilidad, aunque sus primeras formulaciones asumían escenarios continuos e ideales, sin tener en cuenta las restricciones dinámicas de los buscadores ni las limitaciones reales de los sensores.

Durante la década de 1960, la teoría de la búsqueda demostró su utilidad práctica en dos casos emblemáticos: la localización de las bombas de hidrógeno perdidas en Palomares (Almería, España) en 1966, y la búsqueda del submarino USS *Scorpion*, desaparecido en 1968 en el Atlántico. En ambos casos se aplicaron métodos bayesianos para construir mapas de probabilidad basados en la información disponible y en hipótesis formuladas por expertos. La combinación de razonamiento probabilístico y evidencia observacional permitió reducir drásticamente el espacio de búsqueda y condujo al hallazgo exitoso de los objetivos [3], [4]. Estos episodios marcaron un hito al demostrar la efectividad del razonamiento bayesiano aplicado a la búsqueda de objetos en entornos reales.

A mediados de los años setenta, Lawrence D. Stone formalizó muchos de estos conceptos en su libro *Theory of Optimal Search* [5], que se convirtió en una referencia fundamental en el campo. Sus métodos estaban orientados principalmente a la búsqueda de objetivos estáticos y sentaron las bases para los desarrollos posteriores en planificación óptima. Sin embargo, estudios posteriores mostraron que, al introducir restricciones dinámicas, como por ejemplo limitar el movimiento del buscador a celdas vecinas en un entorno discreto, los métodos clásicos dejaban de ser aplicables directamente. Este descubrimiento llevó al uso de modelos basados en procesos de decisión parcialmente observables (POMDP), capaces de representar la incertidumbre y las limitaciones dinámicas de forma más realista.

En paralelo, durante los años setenta surgieron los primeros estudios sobre búsqueda de objetivos móviles, como los de Pollock (1970) [6] y Dobbie (1974) [7], que abordaron escenarios muy simplificados pero sentaron las bases para los modelos posteriores. Con el aumento de la capacidad computacional en la década de 1980, los enfoques analíticos fueron sustituidos gradualmente por métodos algorítmicos capaces de manejar escenarios más complejos. Trummel y Weisinger (1986) [8] demostraron que el problema de búsqueda con restricciones espaciales y temporales discretas es *NP-Hard*, lo que impulsó el uso de heurísticas y métodos aproximados en lugar de soluciones exactas.

A partir de los años 2000, la aparición de vehículos aéreos no tripulados (UAVs) de bajo coste y alta capacidad de cómputo revitalizó la investigación en búsqueda probabilística. Los UAVs ofrecían una plataforma ideal para aplicar modelos de búsqueda cooperativa y actualización dinámica de creencias. En este contexto, Bourgault, Furukawa y Durrant-Whyte [9] propusieron un marco bayesiano general para el problema de búsqueda, basado en filtros que permiten actualizar la creencia sobre la posición del objetivo

combinando modelos de movimiento y mediciones sensoriales heterogéneas. Este enfoque se convirtió en un estándar de facto en la literatura posterior.

Más recientemente, se han desarrollado trabajos centrados en la *búsqueda en tiempo mínimo* (MTS). Aunque la minimización del tiempo de detección fue mencionada en los primeros estudios (Pollock, 1970), la mayoría de los trabajos se han enfocado históricamente en maximizar la probabilidad de detección. Sin embargo, investigaciones recientes, como las de Sarmiento et al. (2009), han abordado directamente la optimización del tiempo esperado de detección en entornos estructurados. En esta misma línea, Lanillos et al. [10], [11] aplicaron métodos de optimización como la entropía cruzada y la optimización bayesiana para minimizar el tiempo de detección de objetivos estáticos y móviles. Estas contribuciones sentaron las bases para enfoques más avanzados, como los propuestos por Carabaza [12], que integran algoritmos bioinspirados y métricas probabilísticas (ET, DTR) para la planificación cooperativa *multi-UAV*.

2.1.2 Heurística en búsqueda probabilística

En el contexto de algoritmos de búsqueda, la heurística representa el conocimiento específico del problema que guía a los agentes hacia soluciones prometedoras. En misiones de búsqueda con UAVs, la heurística cuantifica el valor de moverse a cada posición candidata, permitiendo a los agentes tomar decisiones informadas sin explorar exhaustivamente todo el espacio de búsqueda.

Las heurísticas para misiones de búsqueda con UAVs han evolucionado desde enfoques locales simples hasta métodos sofisticados que consideran información global y planificación anticipada. Las estrategias más básicas, como la heurística local miope, seleccionan el siguiente movimiento basándose únicamente en la información inmediata del mapa de probabilidad, maximizando la ganancia de información en cada paso. Sin embargo, este enfoque sufre del problema de la miopía: los agentes pueden quedar atrapados en óptimos locales al no considerar el potencial de exploración a largo plazo.

Para superar estas limitaciones, se recurre a técnicas heurísticas y metaheurísticas más avanzadas que permiten obtener soluciones aproximadas en tiempos computacionales razonables. En particular, los algoritmos bioinspirados se han consolidado como una alternativa eficaz para abordar problemas complejos de optimización debido a su capacidad para equilibrar la exploración y la explotación del espacio de soluciones. Estos algoritmos reproducen comportamientos observados en sistemas naturales como la cooperación entre individuos, la comunicación indirecta o la adaptación colectiva y han demostrado gran eficacia en la resolución de problemas complejos de optimización.

2.2 Problemas NP-Hard

En el ámbito de la computación y la teoría de la complejidad, un problema se considera *NP-Hard* cuando su resolución es, al menos, tan difícil como los problemas más complejos de la clase *NP* (*Nondeterministic Polynomial time*). Formalmente, un problema es *NP-Hard* si todo problema de *NP* puede reducirse a él en tiempo polinómico. Esta definición implica que, salvo que se demuestre que $P = NP$, no existe un algoritmo determinista capaz de resolver cualquier instancia del problema en tiempo polinómico.

A diferencia de los problemas *NP-complete*, los problemas *NP-Hard* no tienen por qué pertenecer a la clase *NP*; es decir, sus soluciones no tienen por qué ser verificables en tiempo polinómico. En muchos casos, ni siquiera son problemas de decisión: pueden ser problemas de optimización, planificación o búsqueda, donde el objetivo es encontrar la mejor solución posible bajo un criterio concreto.

2.2.1 Complejidad del problema de búsqueda probabilística

En disciplinas como la robótica, la investigación operativa y la optimización de trayectorias, los problemas *NP-Hard* aparecen de forma natural. La combinación de múltiples variables, restricciones dinámicas y espacios de búsqueda de alta dimensionalidad conduce a escenarios donde el número de posibles soluciones crece exponencialmente con el tamaño del problema. Por ello, abordar estos problemas mediante técnicas exactas resulta computacionalmente inviable en la práctica, especialmente cuando se requiere operar en tiempo real o con recursos limitados.

El problema estudiado en este trabajo se enmarca precisamente dentro de esta categoría. Encontrar la ruta que minimice el tiempo de detección del objetivo (o que maximice la exploración del mapa de probabilidad) constituye un problema combinatorio de gran complejidad. Cada trayectoria posible corresponde a una secuencia de posiciones sobre la matriz de probabilidad, y el número total de secuencias crece de manera exponencial con la cantidad de pasos y el tamaño del espacio.

Trummel y Weisinger (1986) [8] demostraron formalmente que el problema de búsqueda con restricciones espaciales y temporales discretas es *NP-Hard*. Esta demostración tiene importantes implicaciones prácticas: significa que no se conoce ningún algoritmo exacto que pueda resolver todas las instancias del problema en tiempo polinómico, y que es altamente improbable que tal algoritmo exista.

2.2.2 Justificación del uso de algoritmos aproximados

Debido a esta complejidad inherente, la literatura científica propone el uso de algoritmos de optimización para aproximar soluciones de buena calidad en tiempos razonables. Entre las técnicas más utilizadas destacan:

- **Genetic Algorithms (GA):** Basados en la evolución biológica, utilizan operadores de selección, cruce y mutación para evolucionar poblaciones de soluciones.
- **Particle Swarm Optimization (PSO):** Inspirado en el comportamiento social de bandadas de aves, donde las partículas exploran el espacio de búsqueda influenciadas por su experiencia individual y la del colectivo.
- **Simulated Annealing (SA):** Basado en el proceso de recocido de metales, permite aceptar temporalmente soluciones peores para escapar de óptimos locales.
- **Ant Colony Optimization (ACO):** Inspirado en el comportamiento de las hormigas, utiliza comunicación estigmérica mediante feromonas para construir soluciones de forma cooperativa.

En este trabajo se emplean tres algoritmos bioinspirados: *Ant Colony Optimization* (ACO), *Artificial Bee Colony* (ABC) y *Black Hole Algorithm* (BHA). Estos algoritmos permiten explorar el espacio de soluciones de forma probabilística, reforzando las trayectorias prometedoras y evitando quedar atrapados en mínimos locales mediante mecanismos específicos de exploración y explotación.

En conclusión, clasificar el problema abordado como *NP-Hard* justifica el uso de algoritmos aproximados como ACO, ABC y BHA, y contextualiza la dificultad computacional de obtener soluciones óptimas exactas. Además, refuerza la elección metodológica y el enfoque experimental del presente trabajo, orientado a comparar métricas, visualizar trayectorias y evaluar estrategias de búsqueda eficientes para UAVs en entornos probabilísticos complejos.

2.3 Búsqueda en tiempo mínimo y problemas relacionados

La búsqueda en tiempo mínimo (*Minimum Time Search*, MTS) en entornos con incertidumbre ha sido ampliamente estudiada en el contexto de sistemas *multi-UAV*. Este tipo de problemas requiere estrategias de planificación y coordinación que permitan optimizar la exploración de áreas desconocidas o parcialmente conocidas, minimizando el tiempo hasta localizar un objetivo o completar una misión.

2.3.1 Relación con problemas clásicos de optimización

El problema MTS está estrechamente relacionado con otros problemas clásicos de planificación y optimización. Aunque comparten elementos comunes, cada uno presenta características distintivas que los diferencian:

- **Path Planning:** El objetivo consiste en encontrar la trayectoria óptima entre un punto inicial y un destino evitando obstáculos o zonas peligrosas. En cambio, el MTS busca optimizar trayectorias en función del tiempo de detección de un objetivo cuya localización es incierta, aprovechando información probabilística sobre su posible ubicación.
- **Coverage Problem:** La meta es explorar completamente un área sin límite de tiempo estricto. En MTS, los recursos y el tiempo de misión son limitados, lo que obliga a priorizar las regiones más relevantes según su probabilidad de contener el objetivo.
- **Art Gallery Problem:** Aborda la cobertura estática mediante sensores fijos para vigilar un espacio. MTS, por el contrario, requiere planificación dinámica de trayectorias móviles en entornos probabilísticos.
- **Travelling Salesman Problem (TSP):** Optimiza recorridos cerrados entre múltiples puntos conocidos. MTS se diferencia por incorporar incertidumbre sobre la localización del objetivo y restricciones temporales, lo que lo convierte en un problema más dinámico y complejo.

2.3.2 Contribuciones recientes en MTS

Sara Pérez Carabaza [12] ha trabajado en la aplicación de algoritmos bioinspirados para la planificación de trayectorias en MTS, mostrando cómo técnicas basadas en colonias de hormigas (ACO) pueden generar rutas eficientes bajo incertidumbre en mapas probabilísticos. Su enfoque combina la optimización de funciones objetivo como el tiempo esperado de búsqueda (ET) y la recompensa temporal descontada (DTR), demostrando mejoras significativas frente a estrategias heurísticas tradicionales.

El rendimiento de una trayectoria de búsqueda se evalúa mediante diferentes métricas, que cuantifican criterios como la rapidez de detección, la eficiencia en la exploración o la ganancia de información. En este contexto, las funciones objetivo más utilizadas son:

- **Expected Time (ET):** Minimiza el tiempo esperado hasta la detección del objetivo.
- **Discounted Time Reward (DTR):** Maximiza la recompensa acumulada considerando un factor de descuento temporal.

Estas métricas permiten evaluar distintos aspectos del proceso de búsqueda y guían el desarrollo de algoritmos que mitigan el problema de la miopía, promoviendo estrategias de búsqueda más equilibradas entre la explotación local y la exploración global del entorno.

2.4 Técnicas clásicas de búsqueda

Las técnicas clásicas de búsqueda constituyen la base sobre la que se han desarrollado métodos más avanzados. Estas técnicas pueden clasificarse en dos grandes categorías: búsquedas exhaustivas, que garantizan la cobertura completa del espacio, y búsquedas locales, que toman decisiones basadas en información inmediata. A continuación se describen los principales algoritmos de cada categoría, destacando sus características, ventajas y limitaciones.

2.4.1 Búsquedas exhaustivas

Las búsquedas exhaustivas recorren sistemáticamente el espacio de búsqueda siguiendo patrones geométricos predefinidos. Aunque garantizan la exploración completa del área, no aprovechan información probabilística sobre la localización del objetivo, lo que puede resultar ineficiente cuando existe conocimiento previo sobre su posible ubicación.

2.4.1.1 Algoritmo de cortacésped (Lawnmower Search)

El *algoritmo de cortacésped* (*Lawnmower Search*, LS) es una estrategia sistemática de exploración que recorre el mapa siguiendo un patrón similar al de un cortacésped real: el agente avanza en líneas rectas paralelas y, al llegar al final de cada tramo, se desplaza una distancia fija para comenzar un nuevo barrido en sentido contrario. Este patrón garantiza la exploración exhaustiva del espacio, permitiendo cubrir toda el área sin dejar zonas sin revisar.

El procedimiento comienza determinando cuál es la esquina accesible más cercana a la posición inicial del agente. Para ello, se consideran las cuatro esquinas del mapa, desplazadas por un margen de seguridad definido por la distancia máxima de detección. El agente se desplaza hacia dicha esquina siguiendo movimientos horizontales y verticales elementales hasta situarse en el punto de inicio desde el que comenzará el barrido sistemático.

Una vez alcanzada la esquina inicial, el agente recorre el mapa siguiendo trayectorias horizontales. Dentro de cada tramo, el agente avanza en línea recta con un incremento fijo, definido por el parámetro de movimiento. Cuando llega al límite permitido en la dirección horizontal, el agente asciende o desciende verticalmente una cantidad fija (definida por el parámetro de separación) y comienza un nuevo tramo en el sentido opuesto. Este movimiento en zigzag alterna el sentido horizontal en cada iteración, cubriendo progresivamente toda la región del mapa comprendida entre los márgenes establecidos.

En cada posición visitada, el mapa de creencias se actualiza mediante el filtro probabilístico correspondiente, incorporando la información obtenida por el sensor del agente. El algoritmo también registra las trayectorias seguidas (coordenadas x , y y z), así como la evolución temporal de los mapas de probabilidad. El proceso continúa hasta que el agente detecta el objetivo o se alcanza el número máximo de pasos permitidos.

2.4.1.1.1 Ventajas y limitaciones El algoritmo de cortacésped es especialmente útil en escenarios donde se desea garantizar la cobertura completa del mapa, ya que no depende de la estructura de la distribución probabilística ni de heurísticas que puedan conducir a mínimos locales. Su principal **ventaja** es la garantía de cobertura total del área de búsqueda.

Sin embargo, presenta importantes **limitaciones**:

- No aprovecha información probabilística previa sobre la localización del objetivo.

- Puede resultar muy ineficiente en términos de tiempo cuando existe conocimiento sobre regiones de alta probabilidad.
- Mantiene un movimiento rígido y sistemático que no se adapta dinámicamente al entorno.
- En espacios grandes, el tiempo requerido para completar el barrido puede ser prohibitivo.

2.4.1.2 Algoritmo de expansión cuadrada (Expanding Square Search)

El *Expanding Square Search* (ESS) es un algoritmo clásico en tareas de búsqueda donde se desconoce la distribución espacial del objetivo. Su funcionamiento se basa en recorrer el entorno mediante cuadrados concéntricos cada vez más amplios, comenzando desde una posición central. Este método garantiza que el agente expanda de forma sistemática su área de cobertura, manteniendo una trayectoria ordenada y fácilmente reproducible.

En primer lugar, el agente se desplaza desde su posición inicial hasta el centro del mapa. Este punto actúa como origen de la expansión. Una vez centrado, el agente comienza a recorrer un cuadrado alrededor del origen, moviéndose en las cuatro direcciones principales: derecha, arriba, izquierda y abajo. Cada vuelta completa incrementa el tamaño del cuadrado, lo que permite cubrir zonas cada vez más alejadas del centro.

Durante el avance, tras cada movimiento el agente actualiza el mapa de probabilidades mediante el filtro correspondiente. Si el objetivo se encuentra dentro del radio de detección, el filtro lo identifica y el algoritmo detiene la búsqueda inmediatamente. Si no se detecta el objetivo, el agente continúa recorriendo los lados del cuadrado, aumentando la longitud de los tramos de forma progresiva para expandir la búsqueda.

A pesar de su simplicidad, su **rendimiento puede ser inferior** al de algoritmos más informados cuando existen distribuciones de probabilidad no uniformes. Al igual que el algoritmo de cortacésped, ESS no aprovecha el conocimiento probabilístico previo y puede invertir recursos en explorar regiones con baja probabilidad de contener el objetivo. Sin embargo, su estructura geométrica lo convierte en una referencia habitual en búsquedas sistemáticas y sirve como base para comparaciones con métodos más avanzados.

2.4.2 Búsquedas locales

Las búsquedas locales, a diferencia de las exhaustivas, toman decisiones basadas en información inmediata sobre el estado actual del sistema. Utilizan heurísticas para evaluar las posiciones candidatas y seleccionar el siguiente movimiento sin realizar una planificación completa del futuro. Aunque son computacionalmente eficientes, pueden sufrir del problema de la miopía.

2.4.2.1 Algoritmo voraz

El *algoritmo voraz* (*greedy*) consiste en que el agente selecciona, en cada paso, el movimiento que maximiza una heurística local, sin considerar la calidad futura de la trayectoria ni realizar planificación a medio o largo plazo. Este enfoque se basa exclusivamente en la información disponible en el estado actual y en las creencias probabilísticas del mapa en ese instante.

El agente parte de una posición inicial y, en cada iteración, genera el conjunto de posiciones sucesoras posibles a partir del conjunto de movimientos permitidos. Estos sucesores se generan en orden aleatorio

para evitar que la decisión quede sesgada por un orden fijo en la lista de movimientos. A continuación, se filtran aquellos sucesores que no son válidos por encontrarse fuera de los límites del mapa.

Sobre los sucesores válidos, el algoritmo evalúa una heurística que estima la conveniencia de cada posición siguiente. Esta heurística puede basarse en criterios como la probabilidad estimada de encontrar el objetivo, la distancia al mismo, la probabilidad acumulada en la región adyacente, el gradiente de la distribución probabilística o cualquier métrica diseñada para favorecer movimientos más prometedores. La función heurística puede combinar múltiples factores con el fin de orientar mejor la selección del siguiente movimiento, incorporando información adicional del mapa de probabilidad, de la estructura del entorno o del comportamiento de los agentes.

La evaluación heurística asigna un valor a cada sucesor y el agente selecciona aquel que maximiza dicha función. Una vez realizado el movimiento, se actualiza el mapa de creencias mediante un filtro probabilístico, que ajusta la distribución en función de la nueva posición del agente y de la posible detección del objetivo. El proceso se repite para cada agente (en caso de búsqueda multiagente) y en cada iteración se registran tanto las trayectorias generadas como la evolución temporal del mapa de probabilidad.

El algoritmo finaliza cuando alguno de los agentes detecta el objetivo o cuando se alcanza el número máximo de pasos definido.

Debido a su naturaleza de búsqueda local, este método presenta una **limitación inherente que lo convierte en miope**: al tomar decisiones basadas únicamente en el beneficio inmediato, el algoritmo puede:

- Quedar atrapado en óptimos locales, explorando repetidamente regiones con probabilidad moderada sin aventurarse hacia áreas más prometedoras pero distantes.
- Estancarse en zonas donde la distribución de probabilidad es uniforme o presenta valores nulos.
- Ignorar el valor informativo a largo plazo de explorar ciertas regiones.
- No considerar la coordinación entre múltiples agentes de forma anticipada.

A pesar de estas limitaciones, el algoritmo voraz es eficiente computacionalmente y permite un comportamiento reactivo ante cambios en el entorno probabilístico, lo que lo hace útil como línea base de comparación y en escenarios donde se requieren decisiones en tiempo real con recursos computacionales limitados.

2.4.2.1.1 Trabajos relevantes Lanillos et al. [10], [11] desarrollaron heurísticas avanzadas basadas en la minimización de la entropía del mapa de creencias y en la maximización de la ganancia de información esperada. Sus trabajos demostraron que heurísticas bien diseñadas pueden mejorar significativamente el rendimiento respecto a estrategias puramente miopes, especialmente en escenarios con múltiples agentes y objetivos móviles.

Por su parte, Carabaza [12] exploró el uso de heurísticas combinadas con algoritmos bioinspirados, mostrando que la integración de información probabilística en la construcción de trayectorias permite obtener soluciones de mayor calidad que los enfoques voraces tradicionales.

Capítulo 3

Formulación del problema de búsqueda

En este capítulo se define formalmente el problema de búsqueda considerado en este trabajo, estableciendo los modelos matemáticos y probabilísticos necesarios para su resolución. El objetivo principal consiste en localizar uno o varios objetivos cuya posición es desconocida, empleando uno o varios vehículos aéreos no tripulados (UAVs) que exploran una región delimitada bajo condiciones de incertidumbre.

La formulación adoptada se apoya en un enfoque probabilístico, donde tanto la posición del objetivo como la información obtenida a través de los sensores se representan mediante distribuciones de probabilidad. Este marco permite integrar de manera coherente la información previa disponible, la dinámica del objetivo y las observaciones realizadas durante la misión, proporcionando una base sólida para el diseño de estrategias de búsqueda eficientes.

El problema se ha modelado basándose en el modelo Global Deterministic Solution: Constraint Programming, descrito en [13], en el que se toma un enfoque de uso de restricciones.

3.1 Espacio de búsqueda

El área de interés donde se desarrolla la misión de búsqueda se denota como $\Omega \subset \mathbb{R}^2$. Con el fin de facilitar su tratamiento computacional, dicha región se discretiza en una cuadrícula regular G_Ω formada por celdas cuadradas de tamaño uniforme $w_x \times w_y$. Cada celda representa una posible localización del objetivo.

Se asume que el objetivo se encuentra dentro del área de búsqueda, por lo que el conjunto de probabilidades asociadas a todas las celdas debe sumar la unidad:

$$\sum_{\nu^0 \in G_\Omega} P(\nu^0) = 1 \quad (3.1)$$

Esta discretización permite trabajar con un mapa de creencias que asigna a cada celda una probabilidad que cuantifica la plausibilidad de que el objetivo se encuentre en dicha posición.

3.1.1 Generación del mapa de probabilidad inicial

La búsqueda del objetivo no se hace completamente a ciegas; antes de empezar se parte de cierta información sobre la posible ubicación o ubicaciones donde es más probable que se encuentre el objetivo. Con esta información se crea un mapa de probabilidad, denotado como $b(\nu^t)$, que asigna a cada punto de Ω la probabilidad de que el objetivo se encuentre en esa posición en el instante de tiempo t .

Siguiendo el modelo propuesto en [14], esta información se obtiene a partir de J fuentes de información (indicios), cada una aportando una capa de probabilidad $b_j(\nu^0)$, y combinándose todas las fuentes ponderando la creencia con un peso w_j :

$$b(\nu^0) = \frac{1}{\sum_{j=1}^J w_j} \sum_{j=1}^J w_j \cdot b_j(\nu^0) \quad (3.2)$$

En nuestro caso, cada fuente o indicio se modela como una distribución de probabilidad gaussiana, generando una función de probabilidad alrededor de un punto τ y especificando una matriz de covarianza Σ . Entonces, para cada punto $P = (x, y)$ de la capa de probabilidad inicial del indicio j , se calcula la probabilidad de que el objetivo se encuentre en ese punto usando la siguiente fórmula:

$$b_{j,P}(\nu^0) = \frac{1}{2\pi \det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(P - \tau)^T \cdot \Sigma^{-1} \cdot (P - \tau)\right) \quad (3.3)$$

Usando este modelo se pueden recrear escenarios muy variados, como asignar diferentes pesos a distintos indicios o modificar la matriz de covarianza para crear distribuciones de probabilidad no circulares.

Una vez generado el espacio de probabilidad inicial, si no se especifica explícitamente una posición para el objetivo, esta se escoge de forma aleatoria alrededor de uno de los indicios. Se utiliza un intervalo de confianza de $n = 3\sigma$, lo que equivale a un nivel de confianza del 99,74 %. Ya que asumimos que nuestras fuentes de información son fiables, el objetivo se encuentra no muy lejos de alguno de los indicios marcados por nuestras fuentes de información.

3.2 Modelo de objetivo

La posición del objetivo en el instante t se modela mediante una variable aleatoria ν^t , cuyo dominio es el conjunto de celdas G_Ω . Esta variable indica que la posición del objetivo puede no ser fija ni es conocida con certeza, sino que la distribución de probabilidad de su posición está repartida por el espacio Ω .

La incertidumbre sobre su localización se representa a través de un mapa de probabilidad o creencia, definido como:

$$b(\nu^t) = P(\nu^t) \quad (3.4)$$

Para modelar la información sobre la posición del objetivo se usa un modelo dinámico que describe la probabilidad de que el objetivo se desplace desde una posición en $t-1$, ν^{t-1} , a ν^t dando un paso de tiempo de duración Δ , expresado como $P(\nu^t | \nu^{t-1})$. Usando este modelo se puede actualizar la distribución de la creencia sobre la posición del objetivo a medida que se realiza la búsqueda.

Para describir este modelo dinámico del objetivo hemos usado un modelo basado en procesos de Markov. En un modelo de objetivo markoviano, el objetivo no tiene memoria de sus posiciones anteriores

y sus acciones solo están definidas por su estado actual. Usando la ecuación de Chapman-Kolmogorov [15], podemos calcular cuál será la distribución de probabilidad en el tiempo t , $\hat{b}(\nu^t)$, actualizando la creencia en $t - 1$, $b(\nu^{t-1})$, con el modelo dinámico del objetivo $P(\nu^t|\nu^{t-1})$:

$$\hat{b}(\nu^t) = \sum_{\nu^{t-1} \in G_\Omega} P(\nu^t|\nu^{t-1})b(\nu^{t-1}) \quad (3.5)$$

Este modelo de transición puede ser todo lo simple o complicado que queramos, incluyendo restricciones del entorno o del movimiento, como direcciones preferidas o límites de velocidad. En los espacios discretos, el modelo de transición se puede implementar a través de una matriz de transición P_{kernel} que incluye la probabilidad de que el objetivo se mueva desde su posición actual a una de sus celdas vecinas. Para aplicar este modelo de objetivo basado en la matriz de transición, se usarán convoluciones, que permiten actualizar y redistribuir las probabilidades hacia celdas vecinas según las probabilidades definidas en la matriz de transición P_{kernel} .

Un ejemplo de las matrices de transición que usamos sería la siguiente 3.6, una matriz de 3×3 , donde el valor central contiene la probabilidad de permanecer en la misma celda en que ya está (0.2), y el resto de valores (0.15 y 0.05) la probabilidad de moverse a las celdas vecinas en las direcciones cardinales o en diagonal:

$$P_{\text{kernel}} = \begin{bmatrix} 0,05 & 0,15 & 0,05 \\ 0,15 & 0,2 & 0,15 \\ 0,05 & 0,15 & 0,05 \end{bmatrix} \quad (3.6)$$

Como en este trabajo nos enfocamos en el caso de un objetivo estático, el modelo probabilístico de transición sería el siguiente:

$$P_{\text{kernel}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (3.7)$$

Aplicando este modelo estático para el objetivo, la ecuación 3.5 para actualizar la creencia sobre la posición del objetivo en el momento t a partir de la creencia en $t - 1$, quedaría así:

$$\hat{b}(\nu^t) = P(\nu^t|\nu^{t-1})b(\nu^{t-1}) = b(\nu^{t-1}) \quad (3.8)$$

3.3 Dinámica de los UAVs

Los UAVs que participan en la búsqueda toman decisiones de movimiento en instantes discretos de tiempo. La dinámica de cada UAV u se modela de forma determinista mediante la ecuación:

$$s_u^{t+1} = f(s_u^t, c_u^t) \quad (3.9)$$

donde s_u^t representa la posición del UAV u en el instante t , y c_u^t es la acción de control aplicada en dicho instante. En nuestro caso, el conjunto de acciones de control corresponde a desplazamientos hacia celdas vecinas de la cuadrícula, siguiendo direcciones cardinales y diagonales:

$$c_u^t \in \{N, NE, E, SE, S, SW, W, NW\} \quad (3.10)$$

Este modelo dinámico permite definir las trayectorias de búsqueda como secuencias de estados o, alternativamente, como secuencias de acciones de control, garantizando que los movimientos respetan las restricciones cinemáticas impuestas al sistema.

3.4 Modelo de sensor

Los UAVs tienen incorporado un sensor con el que son capaces de interactuar con el espacio de búsqueda. Utilizando este sensor realizan *observaciones*, denotando la observación realizada por el agente u en el momento t como z_t^u .

En el problema de búsqueda en espacios con incertidumbre se considera únicamente que estas observaciones pueden ser: o bien de detección del objetivo, $z_u^t = D$, en cuyo caso la búsqueda terminará; o de no detección $z_u^t = \overline{D}$ y la búsqueda del objetivo continúa.

Para simular el comportamiento del sensor se usa la función de verosimilitud $P(z_u^t | \nu^t, s_u^t)$ que, teniendo en cuenta la posición del agente s_u^t y la posición del objetivo ν^t en el momento t , devuelve la probabilidad de que se dé una cierta observación. Usando este modelo, la información de la creencia de la posición del objetivo se puede actualizar a medida que el espacio de búsqueda se explora.

Nosotros asumimos un sensor ideal que detectará siempre al objetivo si este está dentro del área de detección, y que nunca producirá falsas alarmas. El área de detección se modela como un cuadrado de tamaño constante posicionado siempre bajo el agente, inspirado en las cámaras que se usan en los drones reales.

Denotando por $A_{\text{sensor}}(s_u^t)$ el área cubierta por el sensor desde la posición s_u^t , el modelo de sensor ideal se define como:

$$P(z_u^t = D | \nu^t, s_u^t) = \begin{cases} 1 & \text{si } \nu^t \in A_{\text{sensor}}(s_u^t) \\ 0 & \text{si } \nu^t \notin A_{\text{sensor}}(s_u^t) \end{cases} \quad (3.11)$$

De forma equivalente, la probabilidad de no detección es complementaria:

$$P(z_u^t = \overline{D} | \nu^t, s_u^t) = 1 - P(z_u^t = D | \nu^t, s_u^t) \quad (3.12)$$

Dado que en algunos casos vamos a trabajar con más de un agente para realizar la búsqueda, cada uno realizando sus propias observaciones, estas deben ponerse en común. Hemos optado por un enfoque centralizado, de forma que todos los agentes conocen la misma información en todo momento y nos es más fácil razonar sobre qué está ocurriendo durante la búsqueda.

3.5 Fusión de información: Filtro recursivo bayesiano

Si ahora ponemos en común este modelado del problema, llegamos a un enfoque probabilístico que, tomando un modelo dinámico del objetivo y las mediciones de los sensores, devuelve la distribución de probabilidad con la posición del objetivo para el instante t . Este modelo es el *Filtro Bayesiano Recursivo* [16] (RBF por sus siglas en inglés), un algoritmo que consta de dos pasos que se repetirán hasta que el objetivo se detecte o cuando se alcance el límite temporal T .

El proceso consta de dos etapas principales:

Predicción: usando el modelo probabilístico de movimiento del objetivo, se ajusta el mapa de probabilidad. Usando la ecuación 3.5 podemos calcular la distribución de la posición del objetivo en el instante t , basándonos en el modelo del objetivo $P(\nu^t|\nu^{t-1})$ y la creencia $b(\nu^{t-1})$:

$$\hat{b}(\nu^t) = \sum_{\nu^{t-1} \in G_\Omega} P(\nu^t|\nu^{t-1})b(\nu^{t-1}) \quad (3.13)$$

Actualización: redistribuir la probabilidad a priori sobre el mapa a medida que el agente se mueve. Para calcular la creencia de la posición en el instante t se toma la creencia anterior y se actualiza usando la información obtenida de los sensores. En concreto, se usa la probabilidad de observación en el momento t , z_u^t , dado el estado del agente, s_u^t , y el del objetivo, ν^t . Además se añade una constante de normalización, ξ^{-1} , para que la suma de todas las probabilidades sea 1, dando como resultado la siguiente expresión [13]:

$$b(\nu^t) = \xi^{-1} \prod_{u=1}^U P(z_u^t|\nu^t, s_u^t) \hat{b}(\nu^t) \quad (3.14)$$

donde ξ es un factor de normalización que garantiza que:

$$\sum_{\nu^t \in G_\Omega} b(\nu^t) = 1 \quad (3.15)$$

3.6 Notación

A continuación se resume la notación empleada a lo largo de este capítulo, ordenada por orden de aparición:

- Ω : Área de búsqueda.
- G_Ω : Conjunto de celdas que representan la cuadrícula (malla) del área de búsqueda Ω discretizada.
- w_x, w_y : Dimensiones de la malla G_Ω .
- U : Número de UAVs que participan en la búsqueda.
- ν^t : Posición del objetivo en el instante t , modelada como una variable aleatoria.
- $b(\nu^t) = P(\nu^t)$: Mapa de probabilidad que describe la creencia sobre la ubicación del objetivo en el tiempo t .
- $b(\nu^0)$: Probabilidad inicial del objetivo en cada celda, combinando diferentes fuentes de información.
- J : Número de fuentes de información o indicios.
- $b_j(\nu^0)$: Capa de probabilidad de la fuente de información j .
- w_j : Peso asignado a la fuente de información j .
- τ : Punto central de la distribución gaussiana del indicio.
- Σ : Matriz de covarianza de la distribución gaussiana del indicio.
- $P(\nu^t|\nu^{t-1})$: Modelo dinámico del objetivo, que describe la probabilidad de transición del objetivo de ν^{t-1} a ν^t en un intervalo de tiempo Δ .

- P_{kernel} : Kernel bidimensional que representa las probabilidades de transición hacia celdas vecinas.
- s_u^t : Posición del UAV u en el tiempo t .
- c_u^t : Acción de control aplicada por el UAV u en el tiempo t .
- $c_u^t \in \{N, NE, E, SE, S, SW, W, NW\}$: Conjunto de direcciones cardinales y diagonales que el UAV puede tomar como acciones de control.
- Δ : Intervalo de tiempo entre dos mediciones consecutivas.
- z_u^t : Medición del sensor a bordo del UAV u en el instante t , con valores $z_u^t = D$ (detección) o $z_u^t = \bar{D}$ (no detección).
- $P(z_u^t | \nu^t, s_u^t)$: Modelo del sensor, que describe la probabilidad de obtener una medición z_u^t dada la posición del objetivo ν^t y la posición del agente s_u^t .
- $A_{\text{sensor}}(s_u^t)$: Área observable por el sensor desde la posición s_u^t del UAV.
- $\hat{b}(\nu^t)$: Creencia predicha sobre la ubicación del objetivo en el tiempo t .
- ξ : Factor de normalización que asegura que $\sum_{\nu^t \in G_\Omega} b(\nu^t) = 1$.
- T : Límite temporal de la búsqueda.

Capítulo 4

Funciones objetivo en MTS

Las funciones objetivo constituyen herramientas esenciales en los campos de la inteligencia artificial, el aprendizaje automático y la investigación operativa. Su función principal es evaluar el rendimiento de los algoritmos de optimización y servir como guía para el desarrollo de métodos más eficientes. En particular, las funciones de prueba para optimización (*Test Functions for Optimization* [17]) permiten estudiar y comparar el comportamiento de distintos algoritmos en la búsqueda de soluciones óptimas.

Una función de optimización puede clasificarse como de un solo objetivo o multiobjetivo. En el caso de las funciones de un solo objetivo, el propósito es identificar un mínimo o máximo único dentro de un espacio de búsqueda definido. Estas funciones se emplean habitualmente para evaluar la eficacia de algoritmos metaheurísticos.

Dado que los algoritmos bioinspirados son, por su naturaleza, técnicas de optimización, requieren la definición de funciones objetivo que permitan cuantificar y comparar la calidad de las soluciones generadas. En este trabajo se han considerado cuatro funciones objetivo distintas que representan diferentes criterios de búsqueda: *Expected Time (ET)*, *Discounted Time Reward (DTR)*, *Maximum Slope (MS)* y *Minimum Entropy (ME)*.

Cada una de estas métricas captura un aspecto específico de la eficacia en la exploración del mapa de probabilidad, proporcionando así una evaluación integral del rendimiento de las trayectorias generadas. Las funciones objetivo empleadas en este trabajo se han adaptado específicamente al problema planteado. Cada función ha sido implementada como un método independiente dentro de un archivo, acompañado de un *wrapper* que permite su invocación de manera uniforme.

4.1 Expected Time (ET)

El *Expected Time (ET)* o tiempo esperado representa el valor medio del tiempo necesario para detectar el objetivo en función de la probabilidad acumulada sobre las celdas visitadas. El objetivo del algoritmo es **minimizar** este valor, de manera que se prioricen trayectorias que exploren primero las regiones con mayor probabilidad de contener el objetivo.

Matemáticamente, puede expresarse como:

$$ET(s_{0:t}^{1:U}) = \sum_{t=0}^{\infty} P(z_{1:t}^{0:t} = \bar{D} \mid s_{1:t}^{0:t}) = \bar{D} \sum_{t=0}^{\infty} \tilde{b}_t(\nu^t) \quad (4.1)$$

donde

$$\tilde{b}_t(\nu^t) = \prod_{u=1}^U P(z_t^u = \bar{D} \mid s_t^u) \sum_{\nu^{t-1} \in G_R} P(\nu^t \mid \nu^{t-1}) \tilde{b}_{t-1}(\nu^{t-1}) \quad (4.2)$$

Este modelo permite calcular el tiempo esperado en pasos discretos de medición, considerando la evolución de la probabilidad de detección en cada celda visitada por los UAVs. Esta métrica se utiliza habitualmente en problemas de búsqueda y rescate o localización de objetivos, donde el tiempo hasta la detección es un factor crítico de rendimiento. Su simplicidad y carácter intuitivo la hacen una referencia base para comparar estrategias de búsqueda probabilística.

4.1.1 Ejemplo ET

Veamos un ejemplo práctico del cálculo del *Expected Time* (ET) en un mapa de probabilidad pequeño. Consideremos una matriz de probabilidades normalizada de tamaño 3×3 , donde cada celda contiene una fracción de la probabilidad total, aplanada para representar la matriz en una dimensión, y dos trayectorias:

$$\text{trayectoria_A} = [1, 4, 5, 2, 3] \quad (4.3)$$

$$\text{trayectoria_B} = [1, 2, 5, 4, 7] \quad (4.4)$$

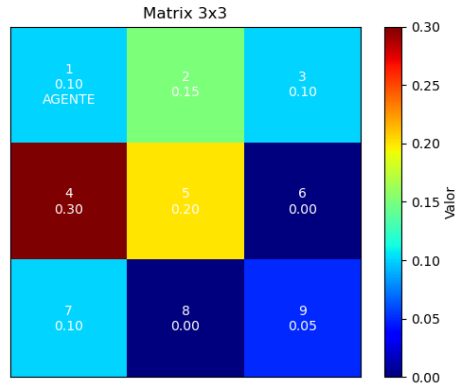


Figura 4.1: Matriz 3×3 utilizada para demostrar el funcionamiento del ET.

Recordemos que el valor de *ET* en cada paso se calcula como el valor de *ET* anterior más la probabilidad restante menos la probabilidad de la celda actual:

$$ET_i = ET_{i-1} + (P_{\text{restante}} - P_{\text{celda}_i}) \quad (4.5)$$

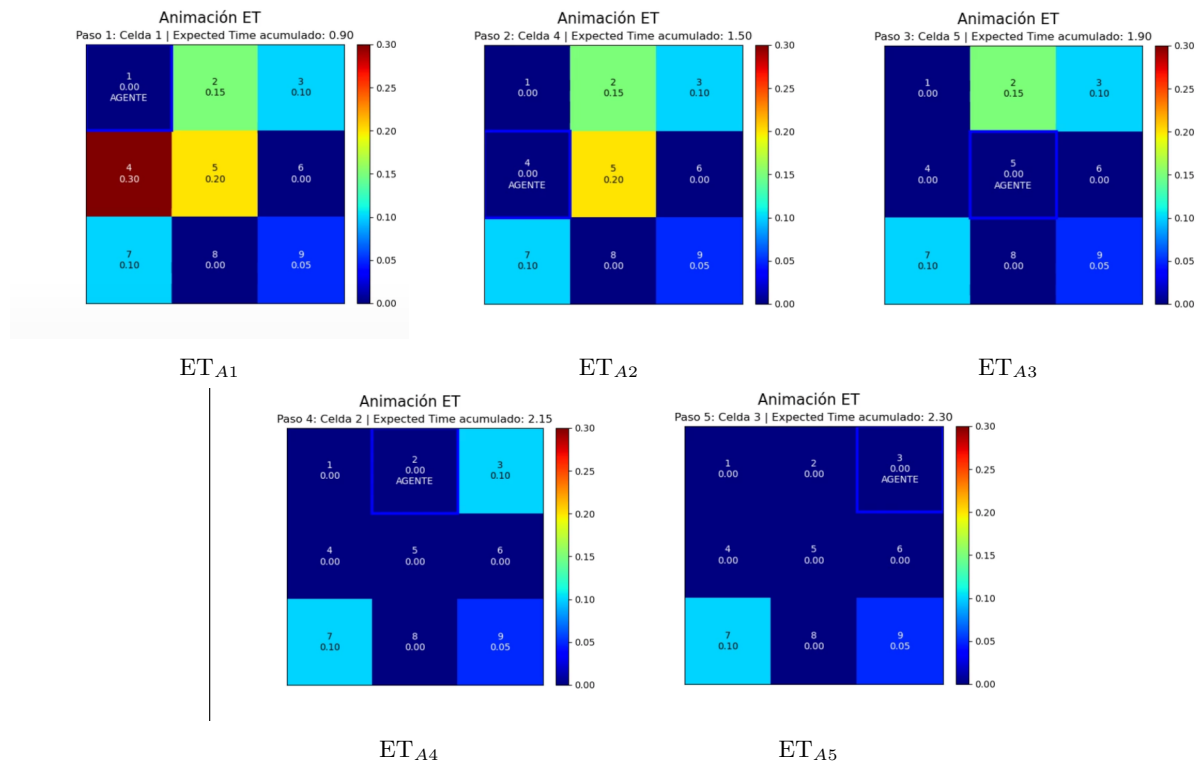


Figura 4.2: Evolución del Expected Time (ET) para la trayectoria A.

A continuación, se muestra el cálculo paso a paso para la trayectoria A:

Tabla 4.1: Cálculo manual de la evolución del ET para la trayectoria A.

Paso	Celda	P(celda)	ET Acumulado A
1	1	0.10	$0 + (1 - 0,1) = 0,9$
2	4	0.30	$0,9 + (0,9 - 0,3) = 1,5$
3	5	0.20	$1,5 + (0,6 - 0,2) = 1,9$
4	2	0.15	$1,9 + (0,4 - 0,15) = 2,15$
5	3	0.10	$2,15 + (0,25 - 0,1) = 2,3$
ET Total			2.3

Para la trayectoria B:

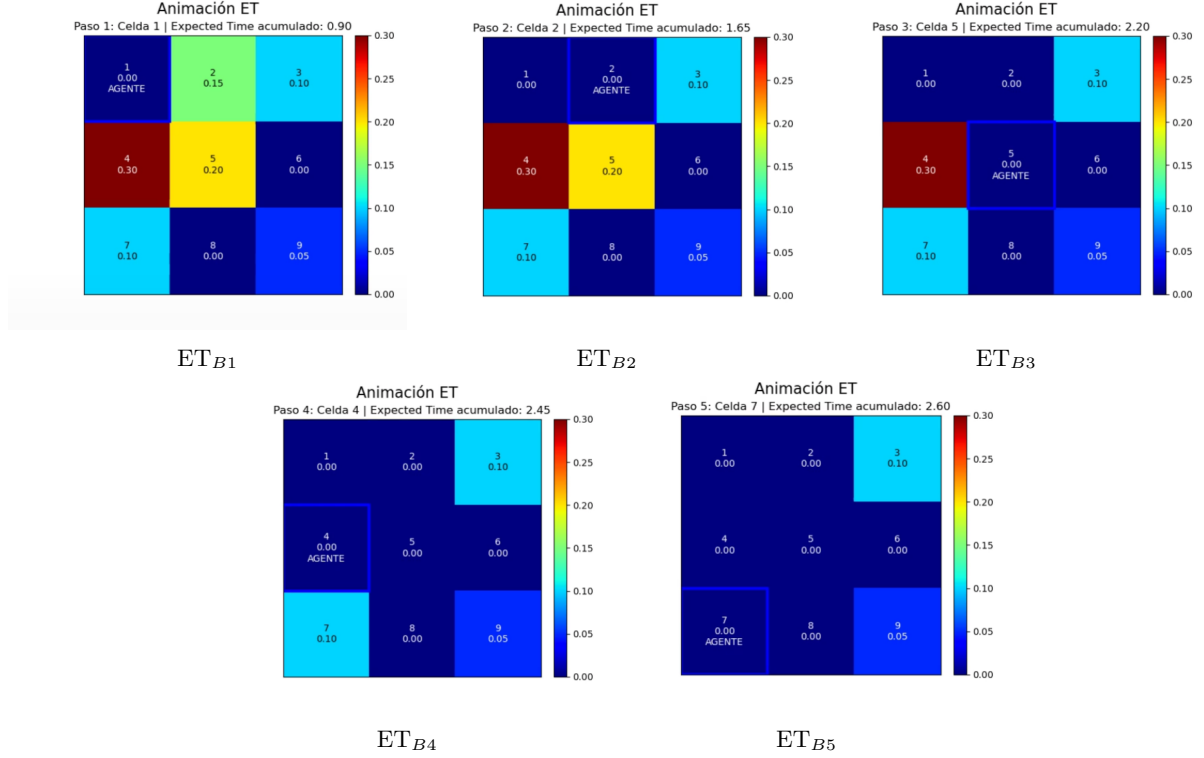


Figura 4.3: Evolución del Expected Time (ET) para la trayectoria B.

A mano, el cálculo del ET acumulado para cada paso se detalla en la siguiente tabla:

Tabla 4.2: Cálculo manual de la evolución del ET para la trayectoria B.

Paso	Celda	P(celda)	ET Acumulado B
1	1	0.10	$0 + (1 - 0,1) = 0,9$
2	2	0.15	$0,9 + (0,9 - 0,15) = 1,65$
3	5	0.20	$1,65 + (0,75 - 0,2) = 2,2$
4	4	0.30	$2,2 + (0,55 - 0,3) = 2,45$
5	7	0.10	$2,45 + (0,25 - 0,1) = 2,6$
ET Total			2.6

Como se puede apreciar, la trayectoria B no minimiza tanto ET como la trayectoria A, pues da el paso hacia la celda con mayor probabilidad (celda 4, con probabilidad = 0.3) más tarde que en la primera trayectoria, penalizando el valor de la función objetivo. Con este principio, las trayectorias generadas por los algoritmos bioinspirados buscarán explotar trayectorias que minimicen esta función objetivo, pues de esta manera los agentes visitarán antes las zonas donde más probabilidad hay de que esté el objetivo.

4.2 Discounted Time Reward (DTR)

La métrica *Discounted Time Reward* (DTR) tiene como objetivo **maximizar** la recompensa acumulada en el tiempo, considerando un factor de descuento γ^{t-1} que penaliza los hallazgos tardíos. De esta forma, las trayectorias que concentran su búsqueda en zonas de alta probabilidad durante los primeros pasos obtienen una recompensa mayor.

Matemáticamente, el *Discounted Time Reward* puede definirse como:

$$DTR(s_{0:t}^{1:U}) = \sum_{t=0}^{\infty} \gamma^{t-1} P(z_t = \bar{D} \mid s_t) \quad (4.6)$$

o, de forma equivalente, utilizando la creencia $\tilde{b}_t(\nu^t)$ en cada instante de tiempo:

$$DTR(s_{0:t}^{1:U}) = \sum_{t=0}^{\infty} \gamma^{t-1} \sum_{\nu^t \in G_R} \tilde{b}_t(\nu^t) \quad (4.7)$$

donde:

$$\tilde{b}_t(\nu^t) = \prod_{u=1}^U P(z_t^u = \bar{D} \mid s_t^u) \sum_{\nu^{t-1} \in G_R} P(\nu^t \mid \nu^{t-1}) \tilde{b}_{t-1}(\nu^{t-1}) \quad (4.8)$$

Siendo $\gamma \in (0, 1]$ el factor de descuento temporal, y $P(z_t^u = \bar{D} \mid s_t^u)$ la probabilidad de detección en la posición s_t^u del UAV u en el instante t .

En la implementación práctica, esta métrica se calcula acumulando la probabilidad de cada celda visitada, descontada en el tiempo según el valor de γ definido por el usuario, que se representa como γ^{t-1} . Esta función recompensa trayectorias que concentran la búsqueda inicial en zonas de alta probabilidad, ya que el valor de las detecciones decrece exponencialmente con el tiempo.

4.2.1 Ejemplo DTR

Para el ejemplo con ambas trayectorias se usará $\gamma = 0,95$:

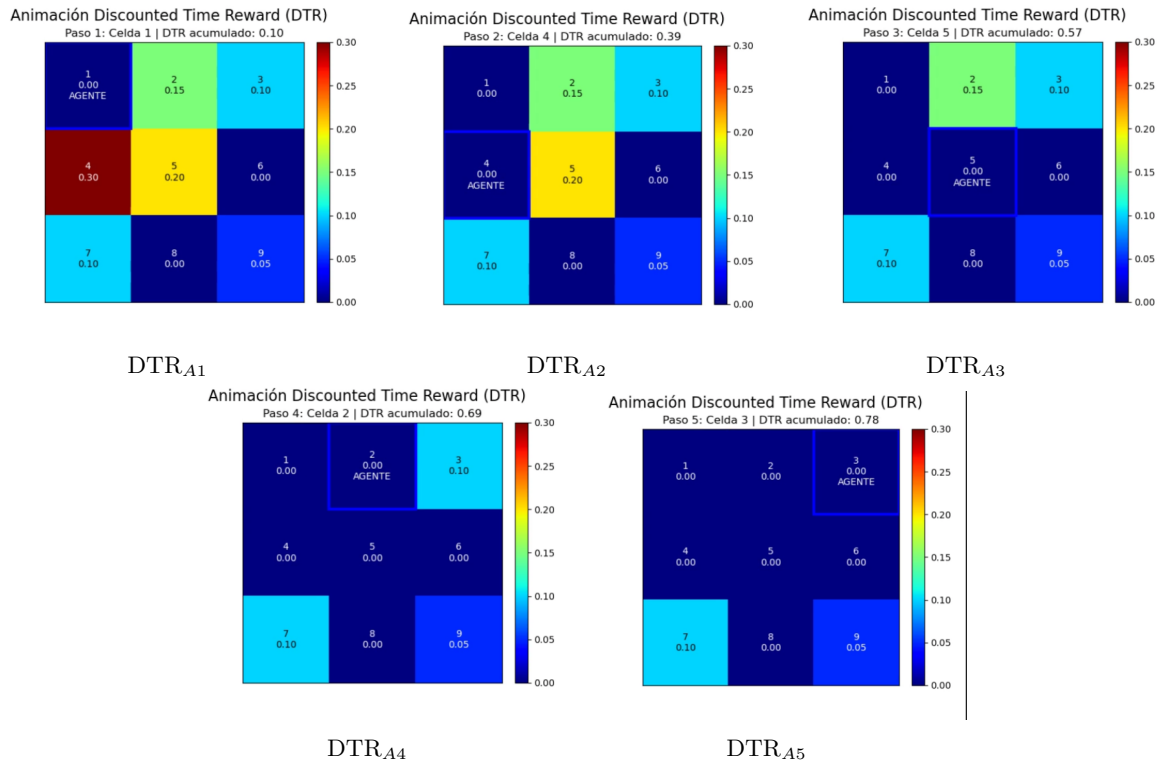


Figura 4.4: Evolución del DTR para la trayectoria A.

Tabla 4.3: Cálculo manual del Discounted Time Reward (DTR) para la trayectoria A.

Paso	Celda	P(celda)	DTR Acumulado
1	1	0.10	$1,0 \cdot 0,10 = 0,10$
2	4	0.30	$0,95 \cdot 0,30 = 0,285$
3	5	0.20	$0,95^2 \cdot 0,20 \approx 0,1805$
4	2	0.15	$0,95^3 \cdot 0,15 \approx 0,1286$
5	3	0.10	$0,95^4 \cdot 0,10 \approx 0,0814$
DTR Total			0.7755

Para la trayectoria B:

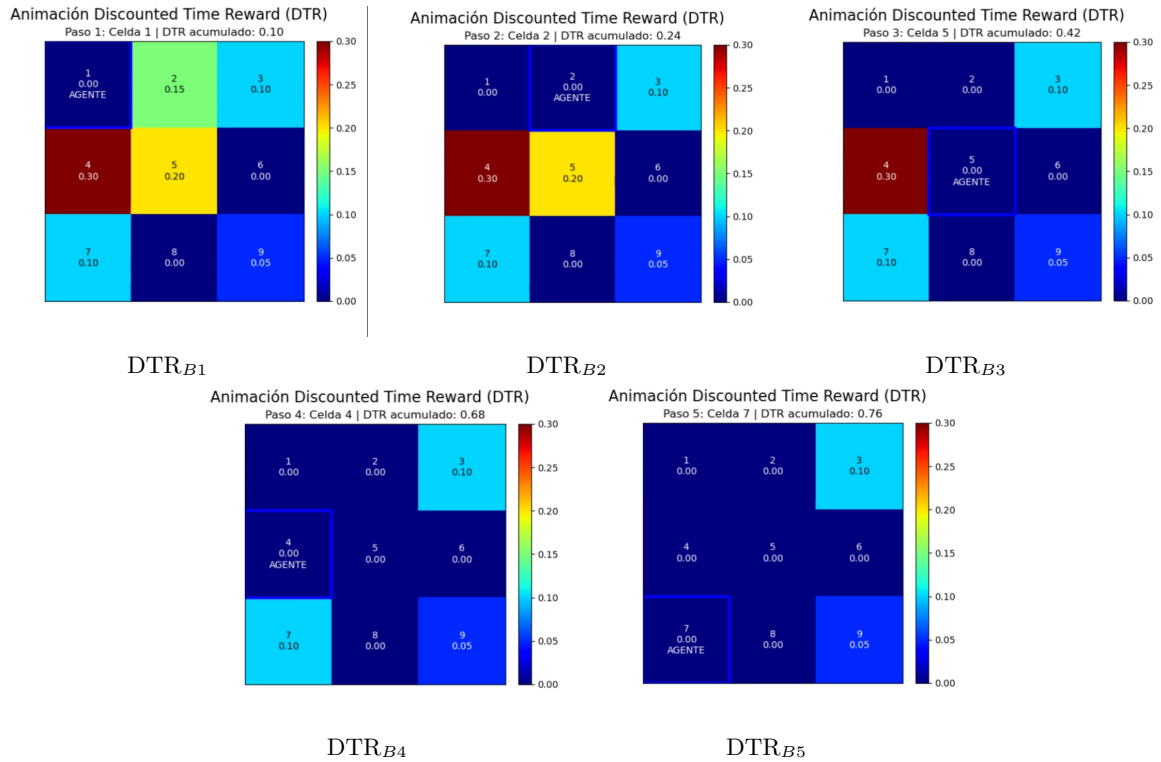


Figura 4.5: Evolución del DTR para la trayectoria B.

Tabla 4.4: Cálculo manual del Discounted Time Reward (DTR) para la trayectoria B.

Paso	Celda	P(celda)	DTR Acumulado
1	1	0.10	$1,0 \cdot 0,10 = 0,10$
2	2	0.15	$0,95 \cdot 0,15 = 0,1425$
3	5	0.20	$0,95^2 \cdot 0,20 \approx 0,1805$
4	4	0.30	$0,95^3 \cdot 0,30 \approx 0,2572$
5	7	0.10	$0,95^4 \cdot 0,10 \approx 0,0814$
DTR Total			0.7616

Ambas trayectorias presentan valores finales muy similares, ya que el factor de descuento γ se encuentra cercano a 1, lo que provoca que las visitas tardías a celdas con mayor probabilidad reciban únicamente una penalización leve. No obstante, se observa una diferencia entre la trayectoria A y la trayectoria B: en

esta última, la celda con mayor probabilidad se visita más tarde, lo que reduce su contribución al valor total, evidenciando la importancia de maximizar esta función para generar trayectorias más eficientes. Aunque estas variaciones puedan parecer pequeñas, en mapas extensos tienen un impacto significativo en la configuración y calidad de las trayectorias.

4.3 Maximum Slope (MS)

La métrica *Maximum Slope* (MS) busca **maximizar** la suma de los cambios máximos en el mapa de creencias a lo largo de la trayectoria. Esta medida refleja la sensibilidad acumulada del sistema ante nuevas observaciones, incentivando trayectorias que produzcan actualizaciones significativas y consistentes de la creencia en cada paso temporal.

El valor del *Maximum Slope* puede expresarse como:

$$MS(s_{0:t}^{1:U}) = \sum_{t=1}^T \max_{\forall(i,j)} |b_t(i,j) - b_{t-1}(i,j)| \quad (4.9)$$

y, de forma análoga al modelo general basado en $\tilde{b}_t(\nu^t)$, puede representarse como:

$$MS(s_{0:t}^{1:U}) = \sum_{t=1}^T \max_{\nu^t \in G_R} |\tilde{b}_t(\nu^t) - \tilde{b}_{t-1}(\nu^{t-1})| \quad (4.10)$$

donde:

- $b_t(i,j)$ es la probabilidad asociada a la celda (i,j) en el instante t ,
- $\tilde{b}_t(\nu^t)$ representa la distribución de creencias global sobre el conjunto de celdas G_R ,
- la operación $\max$ identifica el cambio más pronunciado entre dos pasos consecutivos,
- la sumatoria acumula estos gradientes máximos a lo largo de toda la trayectoria.

A diferencia de métricas que consideran el cambio global o promedio, MS se enfoca en el **pico de variación** en cada instante, premiando trayectorias donde al menos una región del espacio de creencias experimenta una actualización dramática. Este enfoque fomenta exploraciones agresivas que priorizan acciones con alto impacto local en la distribución de probabilidad, típicas de estrategias que buscan regiones con mayor contraste probabilístico.

4.3.1 Ejemplo MS

Veamos cómo cambia esta función con las trayectorias A y B propuestas anteriormente.

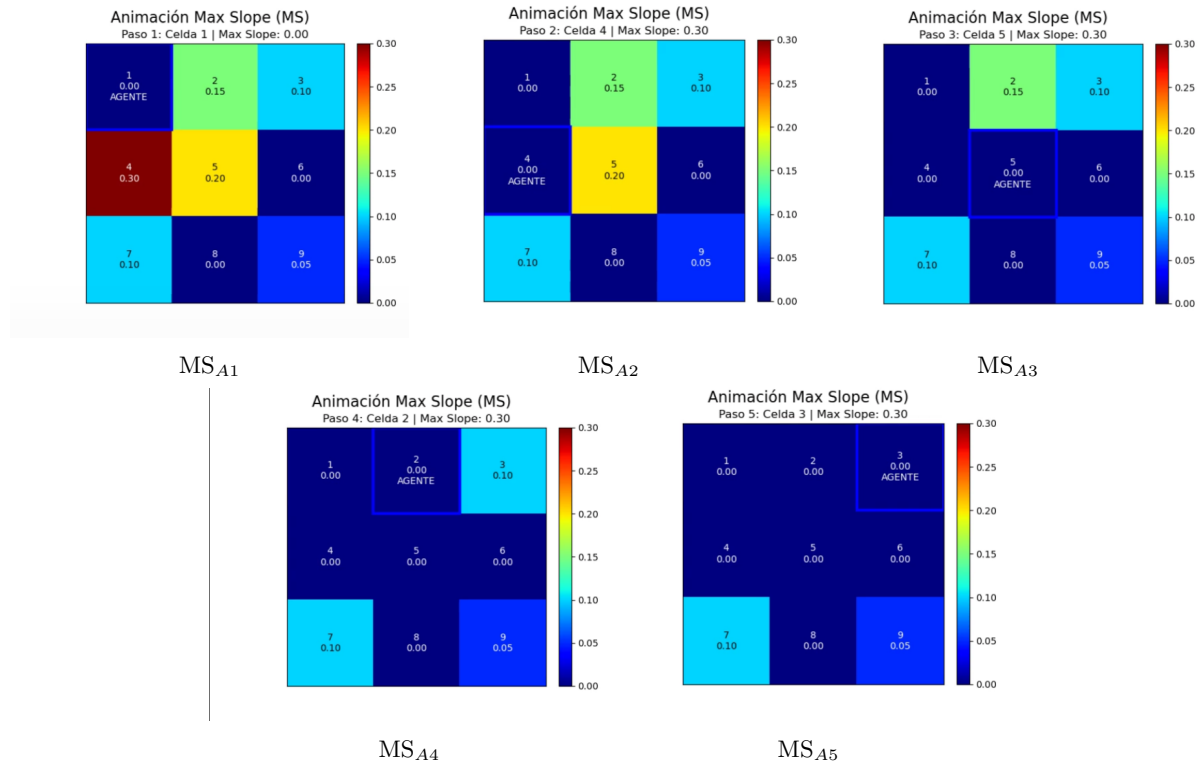


Figura 4.6: Evolución del MS para la trayectoria A.

Tabla 4.5: Cálculo manual del Maximum Slope (MS) para la trayectoria A.

Paso	Celda	P(celda)	MS Total A
1	1	0.10	$\max(0,0,0,0) = 0,0$
2	4	0.30	$\max(0,0, 0,3 - 0,0) = 0,3$
3	5	0.20	$\max(0,3, 0,2 - 0,0) = 0,3$
4	2	0.15	$\max(0,3, 0,15 - 0,0) = 0,3$
5	3	0.10	$\max(0,3, 0,10 - 0,0) = 0,3$

Para la trayectoria B:

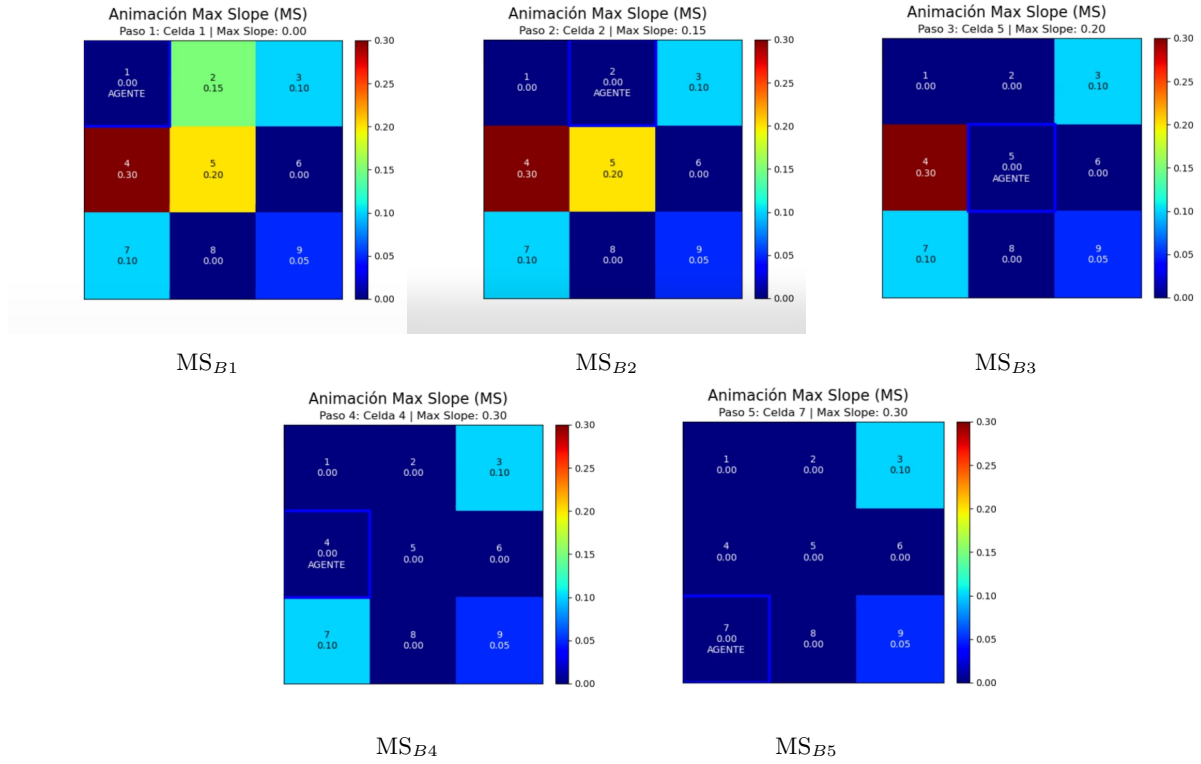


Figura 4.7: Evolución del MS para la trayectoria B.

Tabla 4.6: Cálculo manual del Maximum Slope (MS) para la trayectoria B.

Paso	Celda	P(celda)	MS Total B
1	1	0.10	$\max(0,0,0,0) = 0,0$
2	2	0.15	$\max(0,0, 0,15 - 0,0) = 0,15$
3	5	0.20	$\max(0,15, 0,20 - 0,0) = 0,20$
4	4	0.30	$\max(0,20, 0,30 - 0,0) = 0,30$
5	7	0.10	$\max(0,30, 0,10 - 0,0) = 0,30$

Como se puede observar, el Maximum Slope es idéntico en ambas trayectorias, ya que ambas visitan la celda con mayor probabilidad, aunque en momentos distintos. Es importante destacar que, al no considerar actualizaciones de las celdas, el Maximum Slope siempre corresponderá al valor asociado a la visita de la celda con mayor probabilidad. Además, del análisis de esta métrica se desprende que no existe penalización por visitar más tarde las celdas de mayor probabilidad, lo que limita su utilidad en problemas de búsqueda orientados a minimizar el tiempo. Por tanto, sería necesario modificar esta función objetivo para que pueda aplicarse adecuadamente en este contexto.

4.4 Minimum Entropy (ME)

La métrica *Minimum Entropy* (ME) tiene como objetivo **minimizar** la incertidumbre del mapa de creencias al final de la trayectoria, expresada en términos de entropía de Shannon. Una entropía final baja indica un alto grado de certeza acerca de la posición del objetivo, lo que refleja una trayectoria eficiente desde el punto de vista informativo.

La entropía del mapa de creencias b_t se define como:

$$H(b_t) = - \sum_{\nu^t \in G_R} b_t(\nu^t) \ln b_t(\nu^t) \quad (4.11)$$

por lo que la función objetivo a minimizar puede expresarse como:

$$ME(s_{0:t}^{1:U}) = \min H(b_T) = \min \left(- \sum_{\nu^T \in G_R} \tilde{b}_T(\nu^T) \ln \tilde{b}_T(\nu^T) \right) \quad (4.12)$$

donde:

- $b_t(\nu^t)$ o $\tilde{b}_t(\nu^t)$ representan la distribución de creencias en el instante t ,
- $H(b_T)$ corresponde a la entropía final del mapa tras ejecutar toda la trayectoria,
- $\ln$ denota el logaritmo natural, utilizado en la implementación práctica de la entropía de Shannon,
- el objetivo es minimizar la incertidumbre residual del sistema al término de la exploración.

A diferencia de métricas que acumulan información paso a paso, ME evalúa el **estado final de incertidumbre**, permitiendo valorar la eficacia global de la trayectoria en concentrar la distribución de probabilidad. Esta métrica premia estrategias que, independientemente del camino seguido, logran reducir la dispersión probabilística sobre el espacio de búsqueda hasta alcanzar una creencia altamente concentrada. En el caso ideal, cuando el objetivo es encontrado, la entropía alcanza su valor mínimo ($H(b_T) = 0$) al concentrarse toda la probabilidad en una única celda.

De esta manera, ME proporciona una medida directa de la efectividad informativa terminal de la exploración, complementando métricas que consideran la ganancia de información durante el proceso.

4.4.1 Ejemplo ME

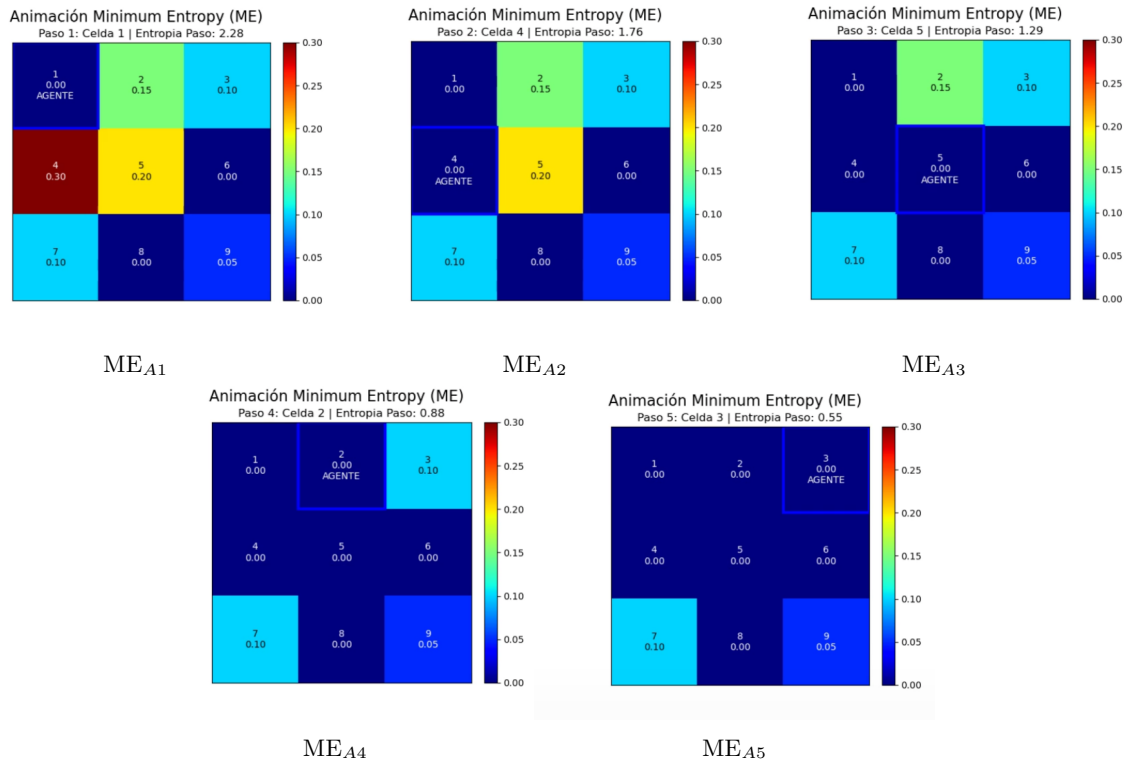


Figura 4.8: Evolución del ME para la trayectoria A.

Tabla 4.7: Cálculo manual del Minimum Entropy (ME) para la trayectoria A.

Paso	Celda	P(celda)	ME Total
1	1	0.10	$0,4105 + 0,5211 + 0,4644 + 0,3322 + 0,2161 = 2,28$
2	4	0.30	$0,4105 + 0,3322 + 0,4644 + 0,3322 + 0,2161 = 1,76$
3	5	0.20	$0,4105 + 0,3322 + 0,3322 + 0,2161 = 1,29$
4	2	0.15	$0,3322 + 0,3322 + 0,2161 = 0,88$
5	3	0.10	$0,3322 + 0,2161 = 0,55$

Para la trayectoria B:

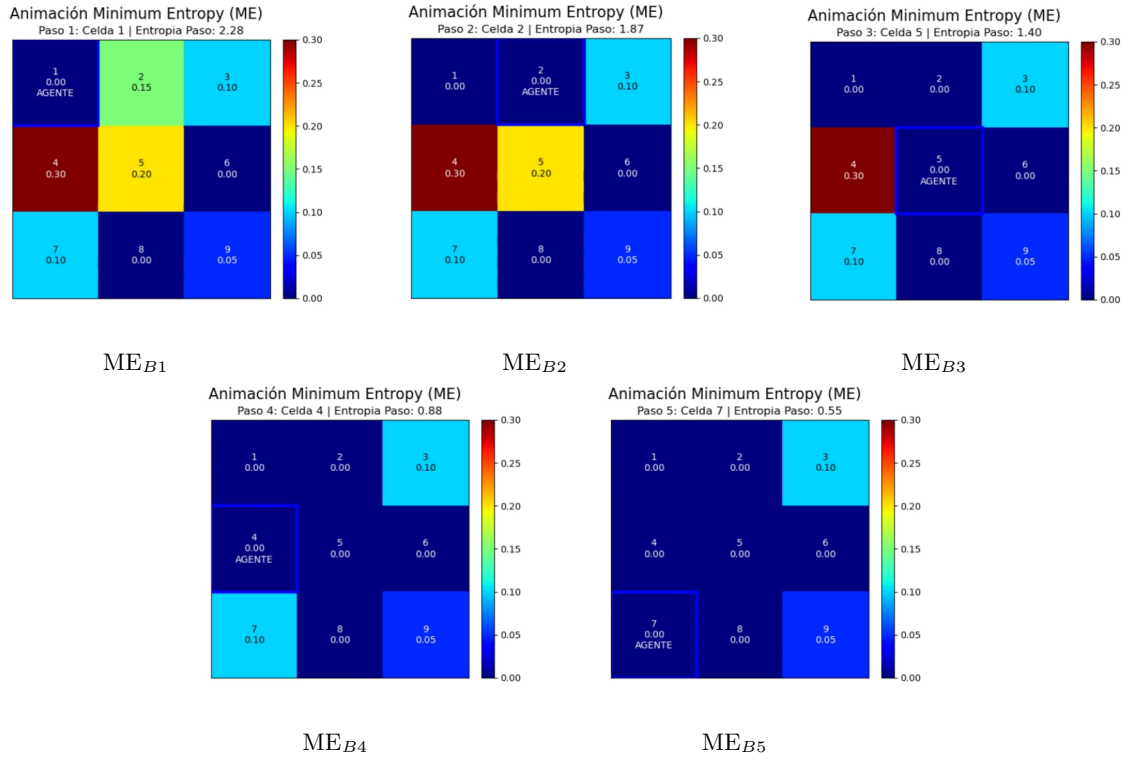


Figura 4.9: Evolución del ME para la trayectoria B.

Tabla 4.8: Cálculo manual del Minimum Entropy (ME) para la trayectoria B.

Paso	Celda	P(celda)	ME Total
1	1	0.10	$0,4105 + 0,3322 + 0,5211 + 0,4644 + 0,3322 + 0,2161 = 2,28$
2	2	0.15	$0,3322 + 0,5211 + 0,4644 + 0,3322 + 0,2161 = 1,87$
3	5	0.20	$0,3322 + 0,5211 + 0,3322 + 0,2161 = 1,40$
4	4	0.30	$0,3322 + 0,3322 + 0,2161 = 0,88$
5	7	0.10	$0,3322 + 0,2161 = 0,55$

Finalmente, para ambas trayectorias se obtiene la misma entropía final, ya que la probabilidad restante en las celdas es idéntica en cada caso. Teniendo esto en cuenta, se presenta una situación similar a la observada con la función Maximum Slope: la métrica no distingue entre recorrer antes o después celdas con mayor probabilidad, por lo que no contribuye de manera efectiva a minimizar el tiempo de búsqueda. Por esta razón, al igual que ocurre con la métrica MS, es necesario modificar esta función objetivo para que pueda aplicarse de manera adecuada al problema.

Capítulo 5

Algoritmos bioinspirados

Los algoritmos bioinspirados constituyen una familia de metaheurísticas de optimización que toman como referencia comportamientos observados en sistemas naturales, tales como colonias de insectos o fenómenos físicos. Estos algoritmos han demostrado ser especialmente efectivos para resolver problemas complejos de optimización combinatoria, donde los métodos exactos resultan inviables debido a su elevado coste computacional.

En este trabajo se emplean tres algoritmos bioinspirados: *Ant Colony Optimization* (ACO), *Artificial Bee Colony* (ABC) y *Black Hole Algorithm* (BHA) adaptados a un entorno discreto para resolver un problema de búsqueda probabilística multi-UAV. La elección de estos tres algoritmos se debe a que representan diferentes paradigmas de inspiración biológica y física, permitiendo evaluar y comparar distintos enfoques de optimización en el contexto del problema planteado.

El entorno de búsqueda se modela como un mapa discreto bidimensional, donde cada celda representa una posición de interés potencial. Las trayectorias de los agentes evolucionan iterativamente según las reglas propias de cada algoritmo y en función de distintas funciones objetivo, combinando mecanismos de exploración y explotación para mitigar el problema de la miopía inherente a la búsqueda local.

5.1 Ant Colony Optimization (ACO)

El algoritmo de *Optimización por Colonia de Hormigas* (*Ant Colony Optimization*, ACO) es una metaheurística inspirada en el comportamiento cooperativo de las hormigas durante la búsqueda de alimento. Este algoritmo se fundamenta en la capacidad de las hormigas reales para encontrar caminos óptimos entre su nido y las fuentes de alimento mediante el depósito y seguimiento de rastros de feromonas.

ACO ha sido ampliamente utilizado para resolver problemas *NP-Hard*, especialmente en optimización combinatoria, como el problema del viajante (TSP), el problema de programación de talleres, el problema de enrutamiento de vehículos (VRP) o el problema de la mochila [18], [19].

5.1.1 Principio de funcionamiento

En la naturaleza, las hormigas depositan feromonas en los caminos que recorren, creando un mecanismo de comunicación indirecta conocido como estigmergia. Los caminos más cortos acumulan mayor concentración de feromonas debido a que son recorridos más frecuentemente, lo que guía a otras hormigas hacia rutas prometedoras.

En la adaptación propuesta, ACO se aplica al problema de búsqueda multi-UAV mediante una matriz de feromonas bidimensional $\tau \in \mathbb{R}^{m \times n}$ asociada directamente al mapa de búsqueda. Cada hormiga (UAV) construye su trayectoria paso a paso, seleccionando movimientos en función de la feromona acumulada y de una heurística basada en el mapa de creencias bayesiano.

La probabilidad de que un agente se desplace desde la celda i a una celda adyacente j viene dada por:

$$p_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{k \in N} [\tau_{ik}]^\alpha \cdot [\eta_{ik}]^\beta} \quad (5.1)$$

donde:

- τ_{ij} representa la cantidad de feromona en la celda j ,
- η_{ij} es la información heurística (típicamente basada en el mapa de creencias),
- α y β controlan la influencia relativa de la feromona y la heurística, respectivamente,
- N representa el conjunto de celdas vecinas factibles.

Para evitar la convergencia prematura, se emplea el esquema *Max-Min Ant System* (MMAS), que impone límites inferiores y superiores a la feromona ($\tau_{\min}$, $\tau_{\max}$), manteniendo la diversidad de soluciones y evitando que la búsqueda quede atrapada en óptimos locales.

5.1.2 Fases del algoritmo

El algoritmo se ejecuta durante un número fijo de iteraciones y, en cada una de ellas, se distinguen las siguientes fases:

1. **Construcción de trayectorias:** Cada una de las n_{agents} hormigas construye su trayectoria de manera incremental usando la regla de transición probabilística definida anteriormente. Durante esta fase se aplica evaporación local de feromona (ρ_{local}) para fomentar la exploración y evitar que las hormigas sigan ciegamente los rastros anteriores.
2. **Actualización del mapa de creencias:** En cada paso, el filtro bayesiano actualiza la distribución de probabilidad del objetivo en función de las observaciones de los agentes, modificando dinámicamente la heurística η_{ij} . Esta actualización permite que el algoritmo se adapte a la información obtenida durante la búsqueda.
3. **Evaluación de la solución:** Al finalizar la trayectoria, se evalúa su calidad mediante la función objetivo seleccionada (ET, DTR, MS o ME) y se compara con la mejor solución global conocida hasta el momento.
4. **Actualización global de feromonas:** Se aplica evaporación global (ρ) sobre toda la matriz de feromonas según la expresión:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} \quad (5.2)$$

Posteriormente, se deposita feromona únicamente en las celdas pertenecientes a la mejor solución global encontrada:

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij}^{\text{best}} \quad (5.3)$$

donde $\Delta\tau_{ij}^{\text{best}}$ es proporcional a la calidad de la mejor solución. Finalmente, se imponen los límites MMAS para mantener todos los valores de feromona dentro del rango $[\tau_{\min}, \tau_{\max}]$.

5.1.3 Parámetros del algoritmo

La configuración de ACO requiere la especificación de diversos parámetros que influyen en el equilibrio entre exploración y explotación:

Tabla 5.1: Parámetros de configuración de ACO.

Parámetro	Descripción
α	Peso de la feromona en la selección de movimientos. Valores altos favorecen la explotación.
β	Peso de la heurística basada en el mapa de creencias. Valores altos favorecen la exploración guiada.
ρ	Tasa de evaporación global de feromonas ($\rho \in [0, 1]$).
ρ_{local}	Tasa de evaporación local durante la construcción de trayectorias.
Q	Constante de depósito de feromona según la calidad de la solución.
$\tau_{\text{mín}}, \tau_{\text{máx}}$	Límites inferior y superior de feromona (esquema MMAS).
Iteraciones	Número total de generaciones de hormigas.
max_steps	Número máximo de pasos por trayectoria.
n_{agents}	Número de UAVs (hormigas) en la búsqueda coordinada.

5.2 Artificial Bee Colony (ABC)

El algoritmo *Artificial Bee Colony* (ABC) es una metaheurística inspirada en el comportamiento de forrajeo de las abejas melíferas. Este algoritmo se basa en el modelo de división del trabajo observado en colonias reales, donde diferentes grupos de abejas desempeñan roles específicos en la búsqueda y explotación de fuentes de néctar.

En este enfoque, cada solución se modela como una fuente de alimento cuya calidad determina la atención que recibe por parte de la colonia [20], [21]. El algoritmo simula el proceso de búsqueda inteligente mediante la cooperación entre diferentes tipos de abejas, cada una con un comportamiento específico.

5.2.1 Principio de funcionamiento

La población se divide en tres tipos de agentes con roles diferenciados:

- **Abejas empleadas:** Cada abeja empleada está asociada a una fuente de alimento específica y es responsable de explotarla y compartir información sobre su calidad.
- **Abejas observadoras:** Permanecen en la colmena y seleccionan fuentes de alimento basándose en la información compartida por las abejas empleadas, favoreciendo las fuentes más prometedoras.
- **Abejas exploradoras:** Abandonan fuentes de alimento agotadas y buscan nuevas fuentes de manera aleatoria, manteniendo la diversidad de la búsqueda.

En la adaptación al problema de búsqueda multi-UAV, cada fuente de alimento se representa mediante una matriz tridimensional $F \in \mathbb{R}^{m \times n \times k}$ que asigna una preferencia a cada combinación de celda (i, j) y

movimiento posible k . Esta matriz codifica el conocimiento acumulado sobre qué movimientos son más prometedores en cada posición del mapa.

La probabilidad de seleccionar un movimiento j desde la posición actual i viene dada por:

$$p_j = \frac{F_{ij}^{1/T} \cdot \eta_j}{\sum_{k \in M} F_{ik}^{1/T} \cdot \eta_k} \quad (5.4)$$

donde:

- F_{ij} es el valor de preferencia para el movimiento j desde la celda i ,
- η_j es la información heurística (basada en el mapa de creencias),
- T es un parámetro de temperatura que regula el compromiso entre exploración y explotación,
- M es el conjunto de movimientos posibles.

5.2.2 Fases del algoritmo

El algoritmo ABC opera mediante ciclos iterativos, cada uno compuesto por las siguientes fases:

1. **Fase de abejas empleadas:** Cada abeja empleada genera una solución vecina mediante la perturbación de su fuente de alimento actual. La perturbación se realiza modificando aleatoriamente algunos valores de la matriz F . Si la nueva solución tiene mejor fitness que la actual, la fuente de alimento se actualiza; en caso contrario, se mantiene la solución original. Se incrementa un contador de intentos sin mejora para cada fuente.
2. **Fase de abejas observadoras:** Las abejas observadoras seleccionan fuentes de alimento mediante un mecanismo de ruleta proporcional a la calidad (fitness) de cada fuente. Las fuentes con mejor fitness tienen mayor probabilidad de ser seleccionadas y explotadas. Una vez seleccionada una fuente, la abeja observadora intenta mejorarla mediante perturbación, similar a la fase de abejas empleadas.
3. **Fase de abejas exploradoras:** Las fuentes de alimento que no han experimentado mejora durante un número determinado de intentos (definido por el parámetro *limit*) se consideran agotadas y son abandonadas. Estas fuentes se reemplazan por soluciones completamente nuevas generadas aleatoriamente, lo que introduce diversidad en la población y evita el estancamiento en óptimos locales.
4. **Actualización de la mejor solución global:** Se compara el fitness de todas las fuentes de alimento actuales y se actualiza la mejor solución global encontrada hasta el momento.

5.2.3 Parámetros del algoritmo

La configuración de ABC implica la definición de parámetros que controlan el comportamiento de las abejas y el equilibrio entre exploración y explotación:

Tabla 5.2: Parámetros del algoritmo ABC.

Parámetro	Descripción
n_{employed}	Número de abejas empleadas, equivalente al número de fuentes de alimento en la población.
n_{onlooker}	Número de abejas observadoras que seleccionan y explotan las mejores fuentes.
limit	Límite de intentos sin mejora antes de abandonar una fuente de alimento.
ε	Epsilon para evitar divisiones por cero.
exploration_rate	Probabilidad de exploración aleatoria durante la construcción de trayectorias.
T	Parámetro de temperatura para la selección probabilística de movimientos.
Iteraciones	Número de ciclos completos del algoritmo.
max_steps	Máximo número de pasos por trayectoria.
n_{agents}	Número de UAVs participantes en la búsqueda.

5.3 Black Hole Algorithm (BHA)

El *Black Hole Algorithm* (BHA) es una metaheurística inspirada en la dinámica gravitacional de los agujeros negros en el universo. Este algoritmo se fundamenta en el concepto físico de que un agujero negro ejerce una fuerza gravitacional tan intensa que atrae toda la materia circundante, incluida la luz.

En la metáfora del algoritmo, cada solución candidata se representa como una estrella en el espacio de búsqueda, mientras que la mejor solución global actúa como un agujero negro que atrae al resto de las estrellas [22], [23]. Este mecanismo de atracción permite que las soluciones converjan gradualmente hacia regiones prometedoras del espacio de búsqueda, mientras que el horizonte de eventos proporciona un mecanismo de diversificación.

5.3.1 Principio de funcionamiento

En BHA, cada estrella representa una solución completa al problema de búsqueda multi-UAV. La estrella con el mejor fitness se designa como el agujero negro, ejerciendo una influencia gravitacional sobre el resto de la población.

Cada estrella se modela mediante una matriz tridimensional $S \in \mathbb{R}^{m \times n \times k}$ que codifica las preferencias de movimiento en cada celda del mapa. La selección de movimientos combina la información almacenada en la estrella con la heurística del mapa de creencias:

$$p_j = \frac{(S_{ij}^{\text{norm}})^{w_s} \cdot (\eta_j^{\text{norm}})^{w_h}}{\sum_{k \in M} (S_{ik}^{\text{norm}})^{w_s} \cdot (\eta_k^{\text{norm}})^{w_h}} \quad (5.5)$$

donde:

- S_{ij}^{norm} es el valor normalizado de preferencia de la estrella,
- η_j^{norm} es la información heurística normalizada,
- w_s y w_h son pesos que controlan la influencia de cada componente,
- M es el conjunto de movimientos posibles.

Las estrellas se mueven hacia el agujero negro mediante una actualización de su matriz de preferencias:

$$S_i^{\text{nuevo}} = S_i^{\text{actual}} + r \cdot (S_{\text{BH}} - S_i^{\text{actual}}) \quad (5.6)$$

donde:

- S_i^{actual} es la matriz de la estrella i en la iteración actual,
- S_{BH} es la matriz del agujero negro (mejor solución),
- r es un número aleatorio en $[0, 1]$ que introduce estocasticidad en el movimiento.

5.3.2 Horizonte de eventos y exploración

Un elemento distintivo de BHA es el concepto de horizonte de eventos, que define una región alrededor del agujero negro. El radio del horizonte de eventos se calcula como:

$$R_{\text{EH}} = \frac{|f_{\text{BH}}|}{\sum_{i=1}^{n_{\text{stars}}} |f_i| + \epsilon} \quad (5.7)$$

donde:

- f_{BH} es el fitness del agujero negro,
- f_i es el fitness de la estrella i ,
- ϵ es una constante pequeña para evitar división por cero,
- n_{stars} es el número total de estrellas.

La distancia entre una estrella y el agujero negro se calcula como la norma de Frobenius de la diferencia entre sus matrices:

$$d_i = \|S_i - S_{\text{BH}}\|_F \quad (5.8)$$

Las estrellas cuya distancia al agujero negro es inferior al radio del horizonte de eventos ($d_i < R_{\text{EH}}$) se consideran absorbidas y se reinician aleatoriamente en nuevas posiciones del espacio de búsqueda. Este mecanismo cumple dos funciones importantes:

- Mantiene la diversidad de la población al evitar que todas las soluciones converjan prematuramente.
- Permite escapar de óptimos locales al reintroducir soluciones aleatorias en la población.

5.3.3 Fases del algoritmo

El algoritmo BHA opera mediante los siguientes pasos en cada iteración:

1. **Generación de trayectorias:** Cada estrella genera una trayectoria utilizando su matriz de preferencias S_i y la información heurística del mapa de creencias.
2. **Evaluación:** Se calcula el fitness de cada estrella mediante la función objetivo seleccionada.
3. **Identificación del agujero negro:** La estrella con el mejor fitness se designa como el nuevo agujero negro.

4. **Movimiento de estrellas:** Cada estrella actualiza su matriz de preferencias moviéndose hacia el agujero negro según la ecuación de atracción.
5. **Absorción y reinicialización:** Se calcula el radio del horizonte de eventos y se reinician las estrellas absorbidas.

5.3.4 Parámetros del algoritmo

La configuración de BHA requiere la especificación de parámetros que determinan el comportamiento de las estrellas y la influencia de la heurística en la búsqueda:

Tabla 5.3: Parámetros del algoritmo BHA.

Parámetro	Descripción
n_{stars}	Número de estrellas en la población.
w_s	Peso de la información de la estrella en la selección de movimientos.
w_h	Peso de la heurística del mapa de creencias en la selección de movimientos.
Iteraciones	Número de ciclos completos del algoritmo.
max_steps	Máximo número de pasos por trayectoria.
n_{agents}	Número de UAVs participantes en la búsqueda.

5.4 Comparación entre algoritmos

Los tres algoritmos presentados comparten el objetivo común de optimizar trayectorias de búsqueda multi-UAV, pero difieren significativamente en sus mecanismos de funcionamiento y características:

- **ACO** se basa en la comunicación indirecta a través de feromonas, permitiendo la emergencia de comportamiento colectivo. Su fortaleza radica en la capacidad de explotar soluciones prometedoras mientras mantiene diversidad mediante la evaporación de feromonas.
- **ABC** organiza la búsqueda mediante roles diferenciados (empleadas, observadoras, exploradoras), combinando explotación intensiva de buenas soluciones con exploración mediante el abandono de fuentes agotadas.
- **BHA** utiliza un mecanismo de atracción gravitacional centralizado, donde todas las soluciones convergen hacia la mejor, pero mantiene diversidad mediante el horizonte de eventos que reinicializa soluciones demasiado cercanas al óptimo actual.

Los algoritmos ACO, ABC y BHA permiten generar trayectorias adaptativas que equilibran exploración y explotación en entornos de búsqueda probabilística. Aunque son algoritmos de aproximación (no garantizan el óptimo global), su capacidad para evitar la miopía de la búsqueda local y adaptarse dinámicamente al entorno los hace especialmente adecuados para problemas complejos de búsqueda multi-UAV.

La principal ventaja de estos algoritmos es su flexibilidad y robustez frente a la incertidumbre, permitiendo obtener soluciones de buena calidad en tiempo razonable. El coste de esta flexibilidad es un mayor esfuerzo computacional comparado con métodos greedy, y la necesidad de múltiples iteraciones para refinar las soluciones. Además, el rendimiento de estos algoritmos depende críticamente de la configuración adecuada de sus parámetros, lo que puede requerir un proceso de ajuste experimental.

Capítulo 6

Framework

En este capítulo se detalla el proceso de desarrollo llevado a cabo para la implementación y evaluación del sistema propuesto. El trabajo se centra en el estudio de tres algoritmos bioinspirados —*Ant Colony Optimization* (ACO), *Artificial Bee Colony* (ABC) y *Black Hole Algorithm* (BHA)— aplicados al problema de búsqueda en tiempo mínimo en entornos con incertidumbre.

Para abordar este estudio, se ha desarrollado un framework integral en Python que integra tanto las estrategias bioinspiradas como técnicas clásicas de búsqueda, conformando un entorno de pruebas controlado que permite la experimentación sistemática y la comparación rigurosa entre diferentes enfoques. Este framework incluye la implementación de múltiples componentes: la generación de mapas de probabilidad, la simulación de la dinámica de los UAVs, la actualización bayesiana de creencias, y la evaluación de trayectorias mediante diversas funciones objetivo.

Se describe la metodología seguida para la adaptación de los algoritmos bioinspirados al contexto específico del problema de búsqueda con UAVs, así como la definición de las funciones objetivo que permiten evaluar cuantitativamente la calidad de las trayectorias generadas. Además, se expone el diseño del sistema de experimentación, incluyendo la estructura del repositorio de código desarrollado, que ha posibilitado realizar diferentes pruebas y comparar los resultados obtenidos bajo diversas condiciones operativas, variando parámetros como el número de agentes, indicios y configuraciones iniciales del entorno.

El capítulo se estructura en varios apartados. En primer lugar, se describe el entorno de pruebas empleado para la realización de las simulaciones. A continuación, se presentan los algoritmos y métodos implementados en dicho entorno, profundizando especialmente en los algoritmos bioinspirados, así como en las funciones objetivo consideradas. Posteriormente, se detallan los parámetros configurables que pueden recibir los algoritmos. Finalmente, el capítulo concluye con una breve descripción del cuaderno utilizado para la ejecución de las pruebas, la forma en que se visualizan gráficamente los resultados y las conclusiones derivadas de este capítulo.

6.1 Entorno de simulación

El entorno de simulación desarrollado proporciona los elementos necesarios para modelar escenarios de búsqueda probabilística con agentes autónomos. Entre los componentes principales se incluyen:

- **Heurísticas de búsqueda:** Se implementan distintas heurísticas para guiar los algoritmos en la exploración del espacio de búsqueda.

- **Algoritmos de búsqueda clásicos:**
 - *Greedy Search*.
 - *Lawnmower*.
 - *Expanding Square Search (ESS)*.
- **Actualización de mapas de probabilidad:** Se emplea un *Filtro Bayesiano Recursivo (RBF)* para actualizar dinámicamente la información sobre la localización del objetivo.
- **Configuración inicial:** Generador de archivos `.json` que contienen la configuración inicial del entorno.
- **Modelo de sensor:** Sensor probabilístico del dron, cuya capacidad de detección depende de la distancia y del nivel de incertidumbre.
- **Visualización:** Generador de animaciones que representan las trayectorias de los agentes y la evolución del mapa de probabilidad en el tiempo.

El desarrollo del sistema de experimentación se ha centrado en la adaptación e integración de tres algoritmos bioinspirados: *Ant Colony Optimization (ACO)*, *Artificial Bee Colony (ABC)* y *Black Hole Algorithm (BHA)*. Estos algoritmos fueron implementados en Python y adaptados para operar sobre mapas bidimensionales de probabilidad que modelan la incertidumbre sobre la localización del objetivo. Cada algoritmo se ha configurado para generar trayectorias que maximizan o minimizan distintas *funciones objetivo*, que evalúan aspectos como:

- *Expected Time (ET)*: Tiempo esperado para localizar el objetivo.
- *Discounted Time Reward (DTR)*: Recompensa temporal acumulada descontada.
- *Maximum Slope (MS)*: Cambios en la distribución de probabilidad a lo largo de la trayectoria.
- *Minimum Entropy (ME)*: Reducción de la incertidumbre en el mapa de probabilidad.

El entorno permite ejecutar múltiples pruebas experimentales modificando parámetros tanto del escenario como de los algoritmos. Entre los factores considerados se incluyen:

- **Tamaño del mapa:** Afecta directamente la complejidad del espacio de búsqueda.
- **Número y posición de los indicios:** Condiciona el nivel de información inicial disponible y la dificultad de localización del objetivo.

Estas configuraciones permiten analizar la robustez, convergencia y eficiencia de cada algoritmo frente a escenarios con distinto nivel de información previa e incertidumbre.

A continuación se muestra un diagrama de componentes que representa las dependencias de los archivos hasta el cuaderno de pruebas:

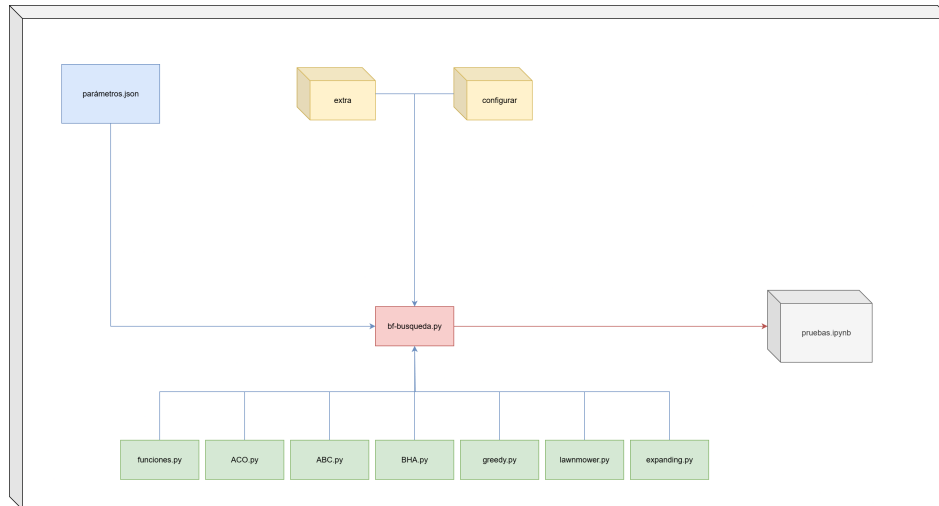


Figura 6.1: Diagrama de componentes del entorno de experimentación.

En el diagrama se incluyen:

- Archivos `.py` correspondientes a los algoritmos de búsqueda y las funciones objetivo.
- Módulos auxiliares presentes en las carpetas `extra` y `configurar`, tales como `rbf.py`, `heurísticas.py` y `sensores.py`.
- Archivo principal `bf_busqueda.py`, que integra los algoritmos y módulos auxiliares, así como los parámetros en formato JSON.

Una vez configurado el entorno y seleccionado el algoritmo de búsqueda en `bf_busqueda.py`, la simulación se ejecuta desde el cuaderno `pruebas.ipynb`, desde el cual es posible cargar cualquier archivo de configuración JSON.

6.2 Parámetros de entrada de los algoritmos y métodos

Estos parámetros se encuentran definidos en un archivo JSON y son utilizados por el código para ejecutar el algoritmo correspondiente, extrayendo únicamente aquellos parámetros que son necesarios para cada ejecución.

Parámetros del entorno

Parámetro	Descripción
version	Versión del formato JSON utilizado para la configuración.
cov	Matriz de covarianza empleada para generar distribuciones gaussianas o indicios.
indicios	Lista de coordenadas que indican la posición de los indicios iniciales.
obj_pos	Posición del objetivo real dentro del entorno.
indicio	Índice del indicio activo para las actualizaciones probabilísticas.
semilla	Semilla aleatoria que garantiza la reproducibilidad del experimento.
height	Altura de vuelo o parámetro vertical utilizado por ciertas métricas o modelos de sensor.
init_pos	Lista de posiciones iniciales de los agentes; puede ser null para asignación aleatoria.
mov_delta	Magnitud del desplazamiento elemental que realiza el agente en cada paso.
posible_moves	Número de direcciones de movimiento permitidas (“4 dir” o “8 dir”).
min_dist	Distancia mínima de separación entre agentes para evitar solapamiento.
itersteps	Intervalo de pasos tras el cual se actualizan gráficos, animaciones o métricas evolutivas.

Tabla 6.1: Parámetros generales de configuración del entorno.

Parámetros del modelo probabilístico

Parámetro	Descripción
λ	Parámetro de penalización empleado en funciones como la métrica DTR.
pdmax	Probabilidad máxima detectable por el sensor.
dmax	Distancia máxima considerada por el modelo de sensor.
σ	Desviación estándar utilizada en modelos gaussianos para el ruido o la dispersión.

Tabla 6.2: Parámetros del modelo probabilístico.

Parámetros comunes de los algoritmos

Parámetro	Descripción
grid_size (X, Y)	Dimensiones de la grilla de búsqueda. Aumentar amplía el espacio de búsqueda y el coste computacional; reducirlo limita el dominio pero acelera la convergencia.
b _k	Mapa de creencias inicial que representa la probabilidad de presencia del objetivo en cada celda. Un mapa concentrado favorece explotación y convergencia rápida; uno uniforme exige mayor exploración.
target (x, y)	Posición real del objetivo utilizada por el filtro de actualización. No afecta la dinámica, pero sirve para simulación y validación de resultados.
heur	Función heurística que calcula la atractividad de los movimientos según el mapa de creencias. Heurísticas más informadas dirigen la búsqueda; heurísticas débiles fomentan la exploración guiada por el algoritmo.
filter	Función de actualización bayesiana del mapa tras cada observación. Filtro agresivo acelera la reducción de incertidumbre; conservador mantiene mayor diversidad.
dmax	Distancia máxima de movimiento permitida. Aumentarla permite desplazamientos largos; reducirla limita a zonas locales.
init_pos	Posiciones iniciales (x, y, z) de cada UAV. Posiciones próximas a zonas de alta probabilidad mejoran rendimiento; alejadas requieren más pasos.
moves	Conjunto de movimientos posibles en el plano (X, Y). Más movimientos incrementan diversidad de trayectorias y coste; menos movimientos simplifican la decisión.
n_agents	Número de UAVs o agentes cooperativos. Aumentarlo mejora cobertura espacial pero incrementa el coste; reducirlo simplifica el problema pero reduce exploración.
separation	Distancia mínima entre UAVs. Valores altos fomentan cobertura amplia; valores bajos permiten concentración en regiones.
max_steps	Longitud máxima de cada trayectoria. Aumentarla permite recorridos más extensos; reducirla acelera el proceso pero limita cobertura.
algoritmo_búsqueda	Algoritmo de búsqueda empleado en el entorno. Puede ser voraz-heur, voraz-myope, lawnmower, expanding_sq, ACO, ABC o BHA. Determina la estrategia de exploración del mapa de probabilidad.
funcion_objetivo	Función para evaluar la calidad de la trayectoria (ET, DTR, MS, ME). Modifica la naturaleza de la optimización (minimización o maximización). Específico de algoritmos bioinspirados.
show_evolution	Si es True, muestra y guarda la evolución de la función objetivo durante la ejecución. Específico de algoritmos bioinspirados.

Tabla 6.3: Parámetros de entrada comunes a los algoritmos de búsqueda.

6.3 Cuaderno de pruebas

El módulo de pruebas permite definir, ejecutar y registrar automáticamente cada experimento. Este componente gestiona los parámetros de simulación, los resultados de las funciones objetivo y los datos necesarios para su posterior análisis gráfico y comparativo. De esta forma, el sistema asegura la repetibilidad de los experimentos y facilita la obtención de resultados consistentes bajo las mismas condiciones iniciales. En el entorno este módulo ha sido desarrollado en un cuaderno de jupyter donde es

posible hacer anotaciones e insertar bloques de código, de forma que hace muy sencillo la implementación de módulos independientes para un único fin, en este caso probar y comparar cada uno de los algoritmos.

El entorno dispone de la capacidad de transformar los datos de las trayectorias generadas por cada algoritmo, junto con la matriz de probabilidades, los indicios iniciales y la posición del objetivo en una animación visual. Esto permite al usuario observar de manera más intuitiva el recorrido del agente. La animación se ejecuta automáticamente al finalizar el algoritmo y puede visualizarse en un navegador web, ya que se despliega en el equipo mediante un servidor web local a través de un puerto disponible.

Además, en el caso de los algoritmos bioinspirados, se lanzará una gráfica donde se puede ver la optimización de las funciones objetivo y como se van estabilizando a través de cada iteración.

6.4 Visualización de la salida del framework

En las pruebas realizadas, la visualización gráfica tanto de las trayectorias como de los agentes presenta el siguiente aspecto:

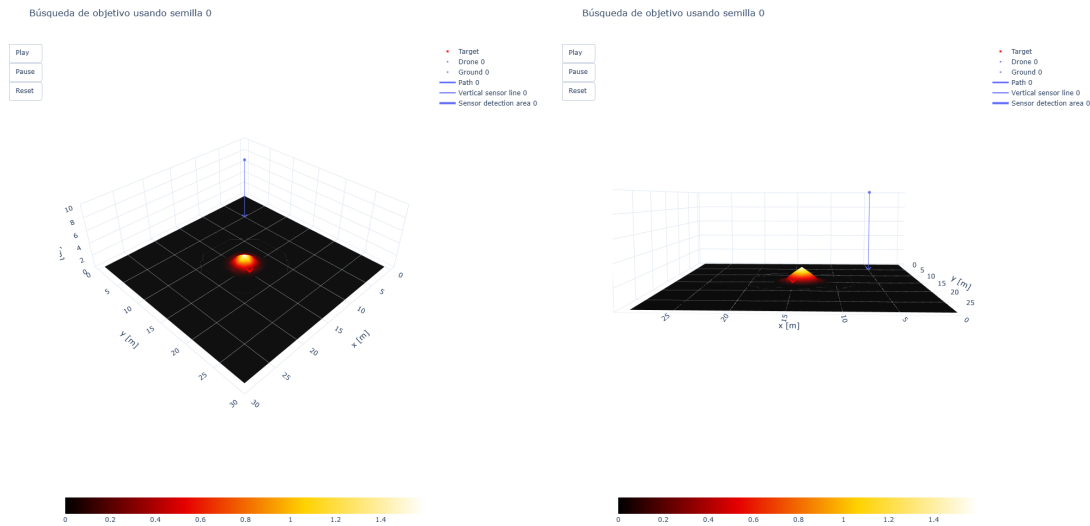


Figura 6.2: Visualización de la animación del entorno.

Como se puede observar, se muestra el mapa de probabilidades junto con los indicios, que representan las zonas del entorno con mayor probabilidad de encontrar el objetivo. También es posible visualizar a los agentes; en este caso, únicamente hay uno, situado en la posición (5,5), y un indicio en la posición (15,15) que contiene al objetivo. Cuando el agente genere una trayectoria, esta podrá visualizarse en la animación para comparar las trayectorias generadas por cada algoritmo de búsqueda.

6.5 Repositorio de GitHub

Para realizar una gestión adecuada de las versiones del software desarrollado en el *framework*, se ha utilizado GitHub como sistema de control de versiones. Se ha creado una nueva rama denominada *algoritmos-bioinspirados* sobre el repositorio GitHub previamente existente del *framework*. Esta estrategia ha permitido mantener el nuevo código desarrollado de forma organizada y separada, facilitando así el desarrollo ordenado del trabajo y garantizando la trazabilidad de

Capítulo 7

Experimentación

En este capítulo se presentan los resultados más relevantes obtenidos en el desarrollo de este trabajo. Para ello, se pone en funcionamiento el conjunto completo de elementos del entorno descrito en el capítulo anterior, con el objetivo de evaluar el desempeño global de los algoritmos implementados.

Cada algoritmo será evaluado variando la configuración definida en el archivo JSON, lo que permite compararlos en mapas de distintos tamaños, con diferente número de indicios y de agentes. Además, los algoritmos bioinspirados se probarán con todas las funciones objetivo, con el fin de determinar cuál combina mejor con cada función. Las pruebas son acordes a las pruebas descritas en el capítulo de desarrollo.

7.1 Pruebas básicas

En esta sección se describen las configuraciones utilizadas para evaluar los algoritmos en las pruebas básicas:

- **Tamaño del mapa:** $[30 \times 30]$, $[50 \times 50]$, $[100 \times 100]$

Para el mapa de 30×30 se emplearán todos los algoritmos disponibles en el entorno: algoritmo voraz miope, algoritmo voraz con heurística, algoritmo de cortacésped, algoritmo de expansión cuadrada, algoritmo de colonia de hormigas, algoritmo de colonia de abejas artificiales y algoritmo de agujero negro.

En el mapa de 50×50 se evaluarán también todos estos algoritmos, considerando dos configuraciones: una con un único indicio y otra con dos indicios colocados de forma simétrica.

Para el mapa de 100×100 se utilizarán igualmente todos los algoritmos. En este caso, se dispondrán 10 indicios distribuidos uniformemente y tres agentes exploradores. Debido al mayor tamaño del mapa, los agentes comenzarán alineados y deberán localizar los 10 indicios siguiendo trayectorias que simulen situaciones realistas.

La posición del objetivo se inicializa aleatoriamente cerca de uno de los indicios en todos los escenarios.

Los agentes cuentan con un número máximo de pasos según el tamaño del mapa: 100 pasos para el mapa pequeño, 300 para el mediano y 500 para el grande. Los parámetros de los algoritmos bioinspirados se han ajustado para mantener un equilibrio entre exploración y explotación durante la búsqueda.

Además, en los algoritmos bioinspirados se emplean las siguientes funciones objetivo:

- **Funciones:** [ET, DTR, MS, ME]

A continuación, se muestran las posiciones iniciales de los agentes y los indicios para cada mapa:

Mapa 30×30 :

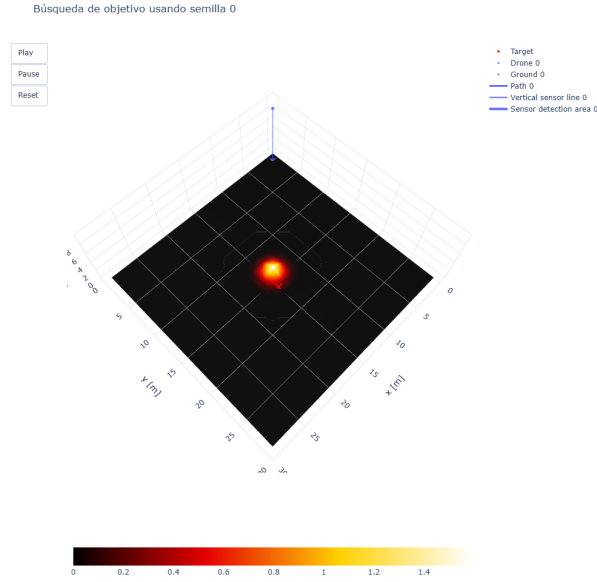


Figura 7.1: Mapa del entorno (30×30).

El indicio se ubica en la celda (15,15), mientras que el agente inicia su búsqueda desde la posición (1,1).

Mapa 50×50 :

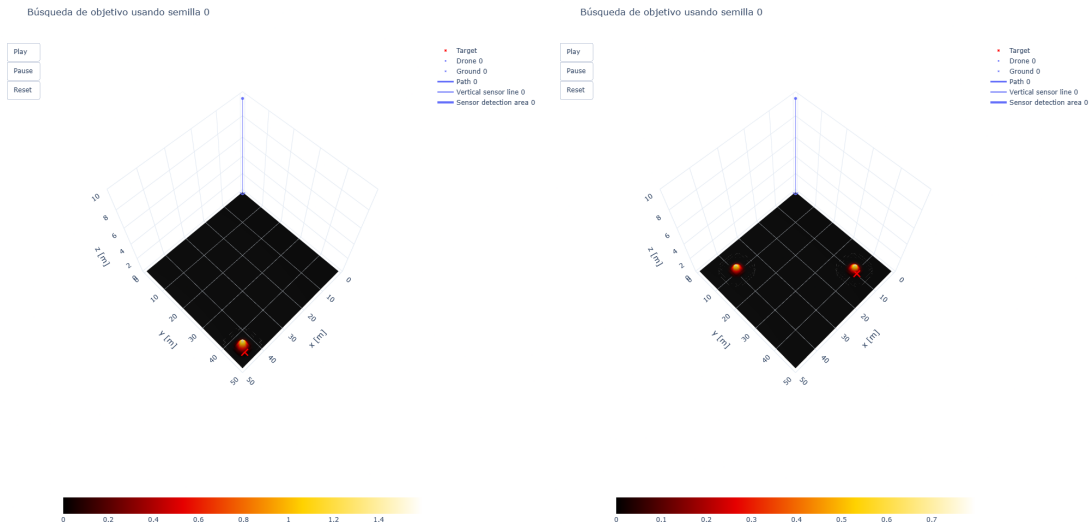


Figura 7.2: Mapas del entorno (50×50).

En este caso se consideran dos escenarios. En el primero, un único indicio se coloca en (45,45), uno de los puntos más alejados del agente, para evaluar la capacidad de localización desde una distancia significativa. En el segundo, dos indicios se sitúan de manera simétrica en (10,40) y (40,10) con el fin de

analizar la miopía de los algoritmos heurísticos. En ambos escenarios, el agente inicia su búsqueda desde $(1, 1)$.

Mapa 100×100 :

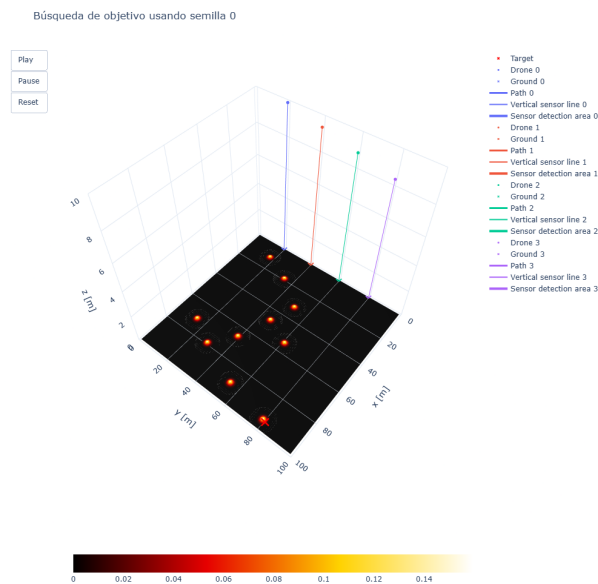


Figura 7.3: Mapa del entorno (100×100).

Los agentes comienzan en $(1, 20)$, $(1, 40)$ y $(10, 60)$. Se colocan 10 indicios distribuidos uniformemente y el objetivo se sitúa en el indicio más alejado. De esta forma, los agentes deben recorrer y analizar todos los indicios para localizar el objetivo.

7.1.1 Mapa 30×30

Voraz miope

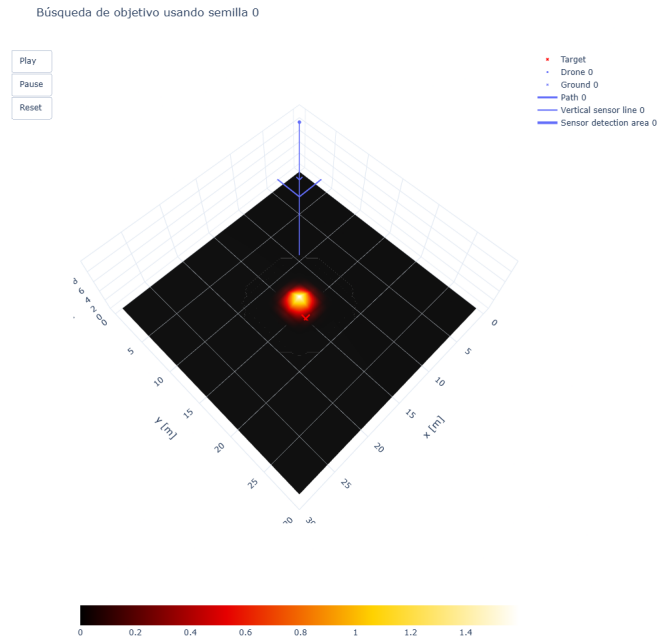


Figura 7.4: Trayectoria obtenida con el algoritmo voraz miope en el mapa 30×30 .

Voraz con heurística

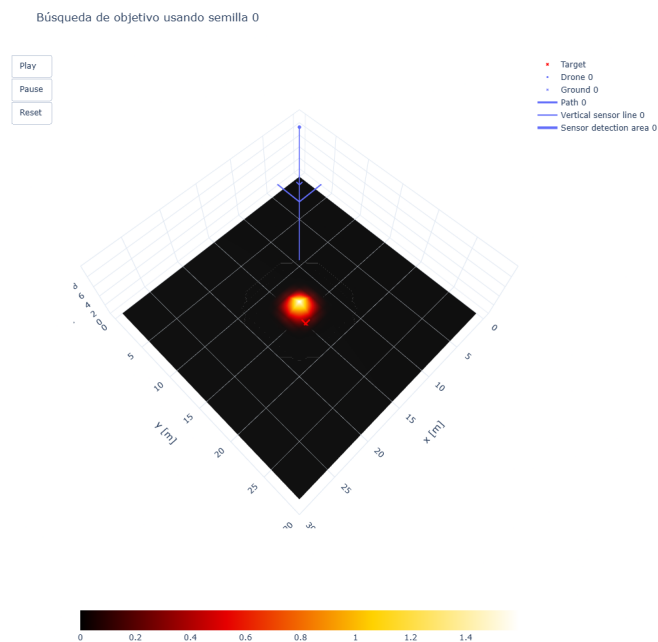


Figura 7.5: Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 30×30 .

Para este mapa el algoritmo voraz miope no tiene ningún problema en encontrar al objetivo ya que el agente está lo suficientemente cerca del indicio para poder explorarlo. Para su versión con heurística pasa exactamente lo mismo.

Cortacesped

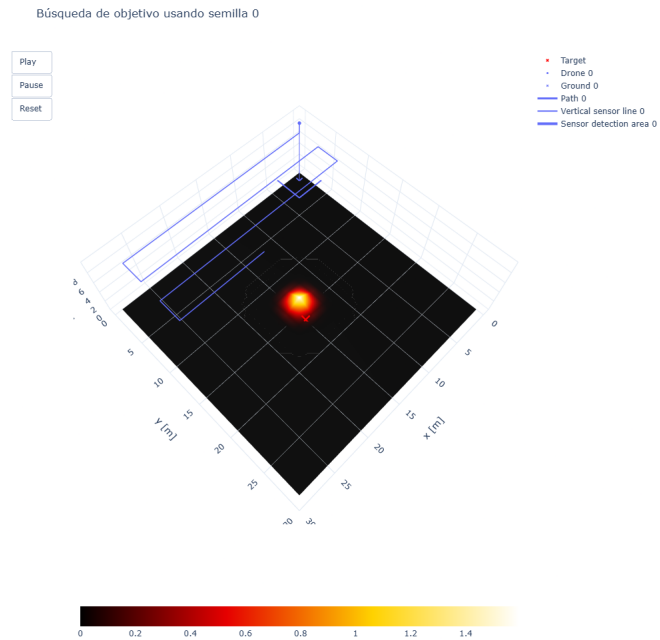


Figura 7.6: Trayectoria obtenida con el algoritmo de cortacésped en el mapa 30×30 .

Expansión cuadrada

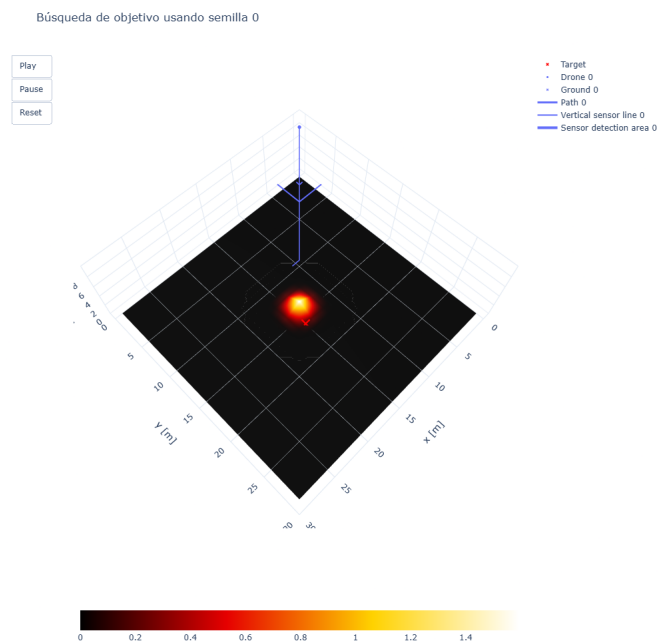


Figura 7.7: Trayectoria obtenida con el algoritmo de expansión cuadrada en el mapa 30×30 .

Respecto a estos 2 algoritmos no hay mucho que analizar, el algoritmo de cortacésped no dispone de la cantidad de pasos suficientes para encontrar al objetivo, y el algoritmo de expansión cuadrada lo encuentra al coincidir su ruta hacia el centro del mapa con la posición del objetivo.

ACO

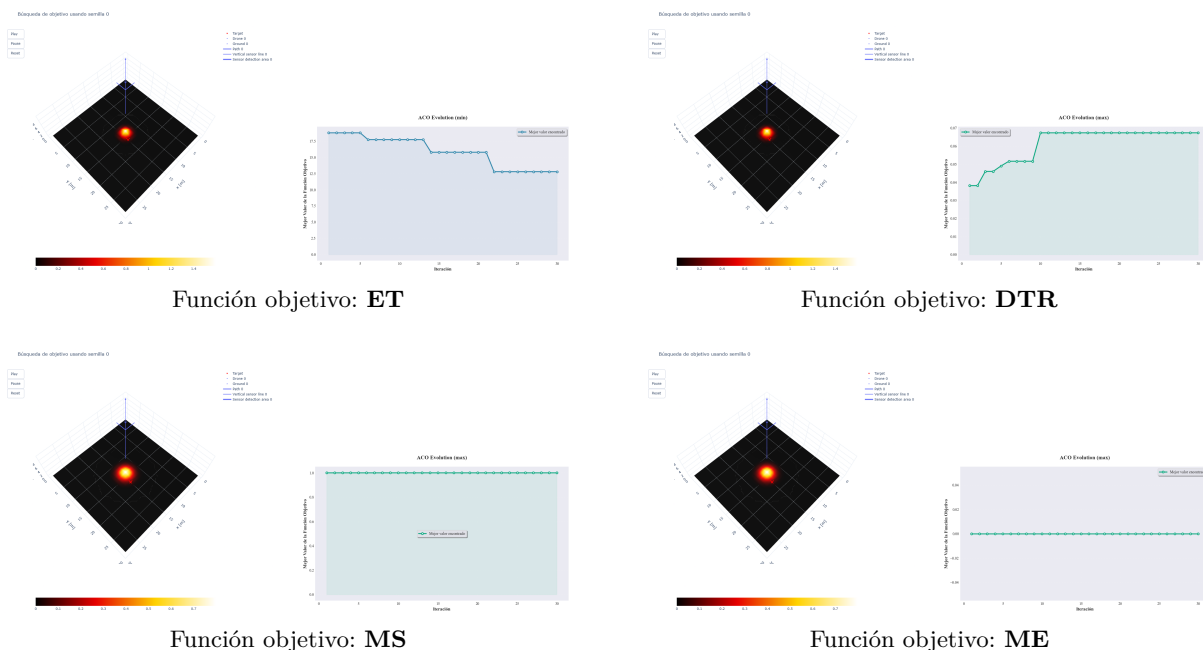


Figura 7.8: Resultados del algoritmo ACO para las cuatro funciones objetivo en el mapa 30×30 : trayectorias (izquierda) y sus gráficas asociadas (derecha).

En este mapa, el algoritmo ACO no presenta dificultades para localizar el objetivo al aplicar las cuatro funciones objetivo consideradas. En el caso de MS y ME, se observa que, siguiendo la heurística definida, el objetivo se alcanza ya en la primera iteración. Dado que, tras su localización, la métrica MS adopta directamente el valor 1.0 y la métrica ME el valor 0.0, las gráficas asociadas permanecen constantes a lo largo de las iteraciones. En los cuatro casos analizados, el algoritmo ACO es capaz de encontrar la ruta óptima hasta el objetivo.

ABC

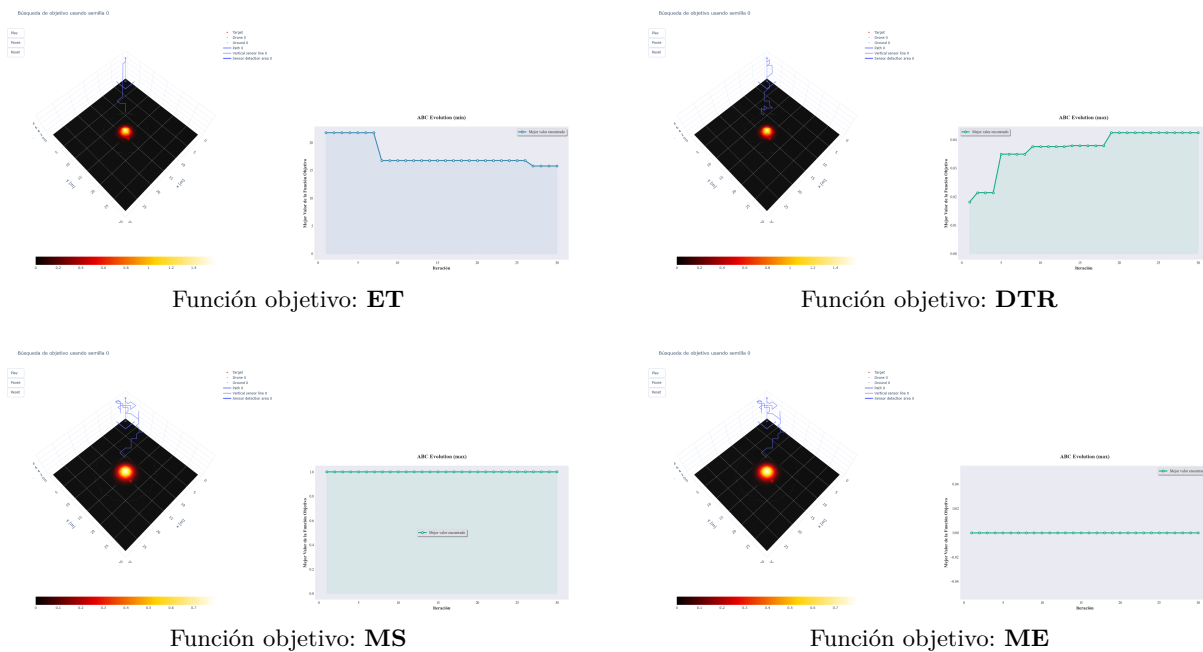


Figura 7.9: Resultados del algoritmo ABC para las cuatro funciones objetivo en el mapa 30×30 : trayectorias (izquierda) y sus gráficas asociadas (derecha).

El algoritmo ABC no presenta una convergencia tan rápida como la observada en ACO; sin embargo, es capaz de localizar el objetivo en los cuatro casos correspondientes a las funciones objetivo consideradas. Las trayectorias generadas por ABC muestran un mayor grado de aleatoriedad en comparación con ACO y, en el caso de las funciones MS y ME, se observa que el algoritmo realiza recorridos más prolongados antes de estabilizarse. No obstante, dado que MS se maximiza y ME se minimiza completamente ya en la primera iteración, la trayectoria resultante permanece invariable en las iteraciones posteriores. Parece que también para MS y ME se realiza la misma trayectoria.

BHA

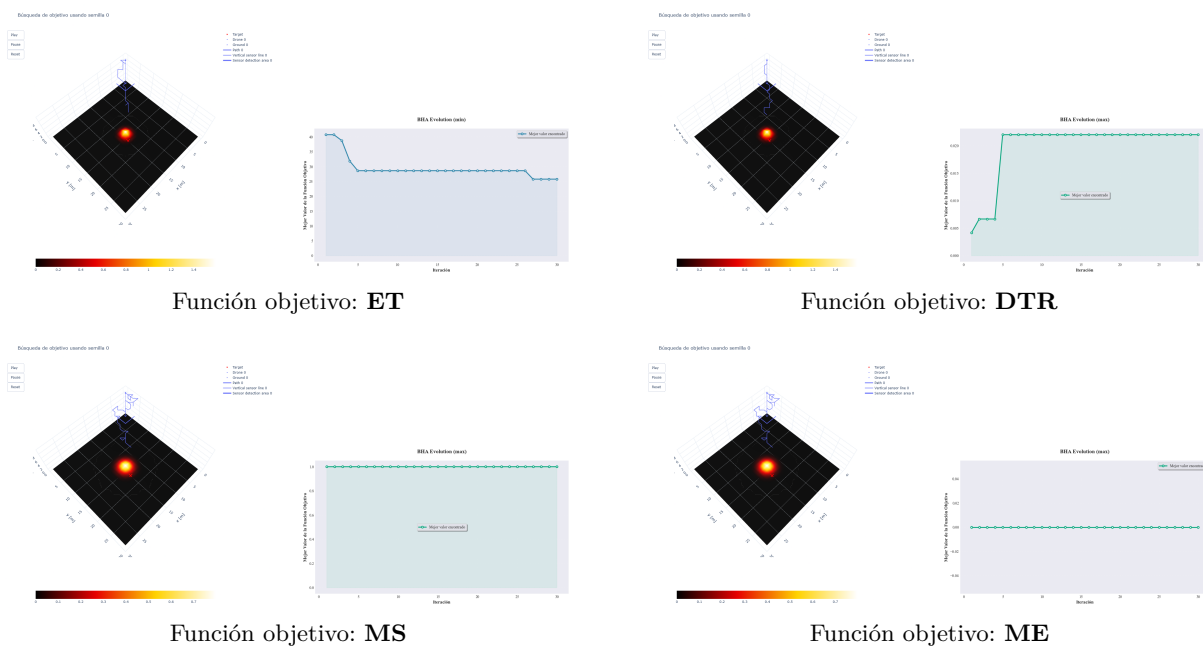


Figura 7.10: Resultados del algoritmo BHA para las cuatro funciones objetivo en el mapa 30×30 : trayectorias (izquierda) y sus gráficas asociadas (derecha).

Para BHA igual que para ACO y para ABC se consigue encontrar al objetivo para las 4 funciones. Al igual que para ABC parece que en MS y ME se realiza la misma trayectoria.

Comparación evoluciones de las funciones objetivo

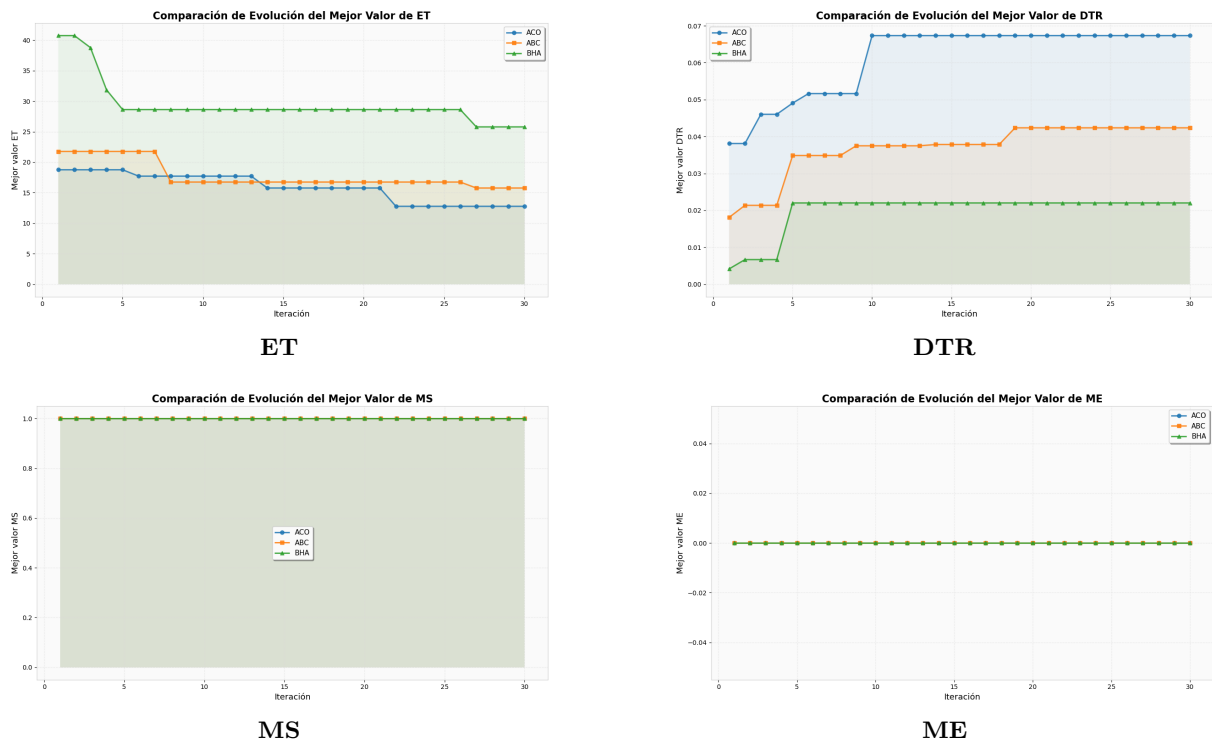


Figura 7.11: Comparación de la evolución de las cuatro funciones objetivo con ACO, ABC y BHA para el mapa 30x30.

De los tres BHA es el que pero a maximizado/minimizado las funciones objetivo pero en general los 3 presentan muy buen desempeño para las rutas de estas situaciones. ACO es el que mejor lo ha hecho de los 3 para este mapa.

7.1.2 Mapa 50x50

Voraz miope

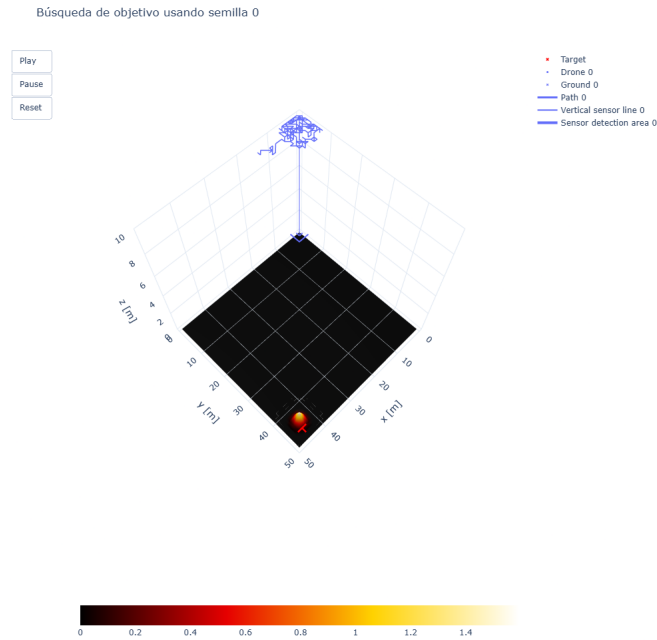


Figura 7.12: Trayectoria obtenida con el algoritmo voraz miope en el mapa 50×50 con 1 indicio.

Voraz con heurística

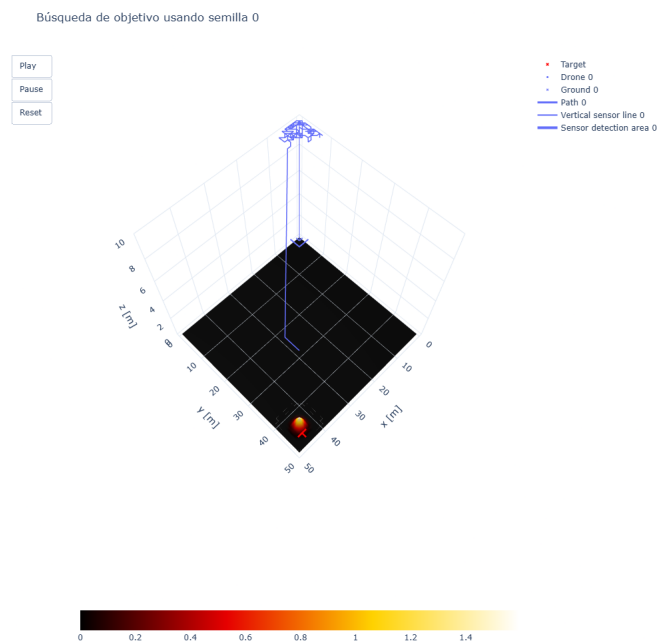


Figura 7.13: Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 50×50 con 1 indicio.

1 Indicio El algoritmo voraz miope es incapaz de explorar el mapa de forma eficaz y se queda estancado en el punto de salida, a diferencia de su parte con heurística que en un paso consigue encontrar un punto donde la heurística empieza a tener efecto sobre su trayectoria y va directamente a explorar el indicio.

Cortacesped

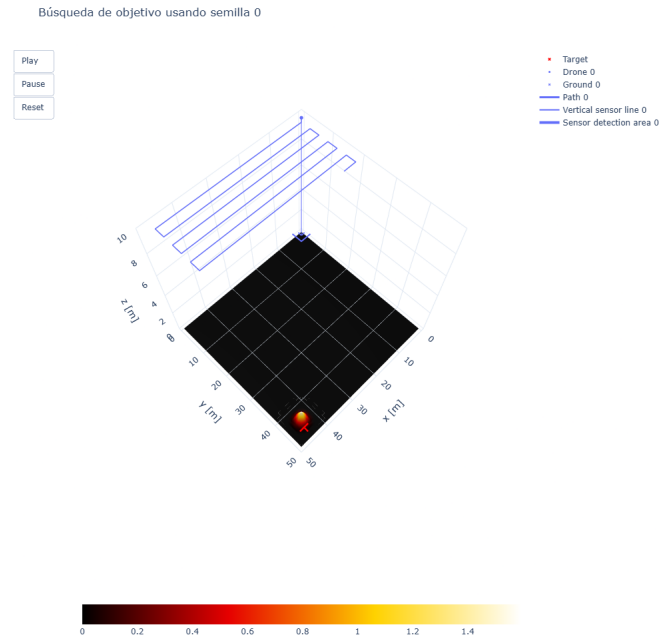


Figura 7.14: Trayectoria obtenida con el algoritmo de cortacésped en el mapa 50×50 con 1 indicio.

Expansión cuadrada

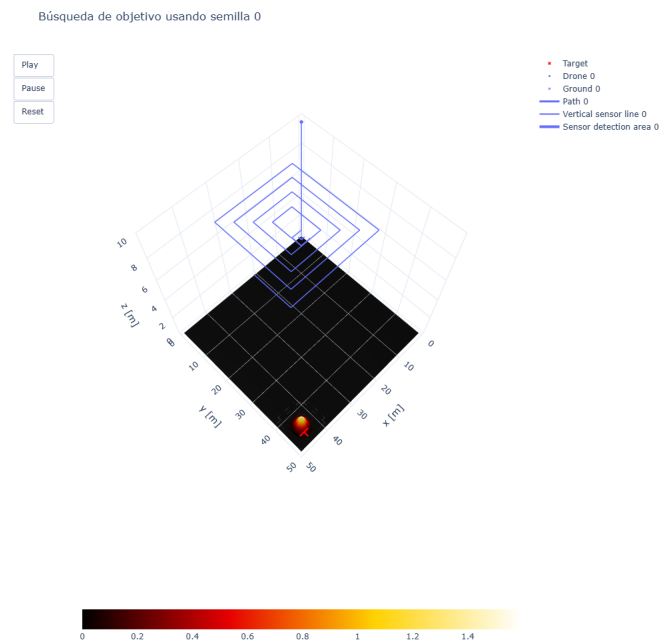


Figura 7.15: Trayectoria obtenida con el algoritmo de expansión cuadrada en el mapa 50×50 con 1 indicio.

Ninguno de estos dos algoritmos consigue encontrar al objetivo por el mismo problema que el algoritmo de cortacésped en la situación anterior, no disponen de suficientes pasos para encontrar al objetivo, que es el único parámetro utilizable por estos algoritmos.

ACO

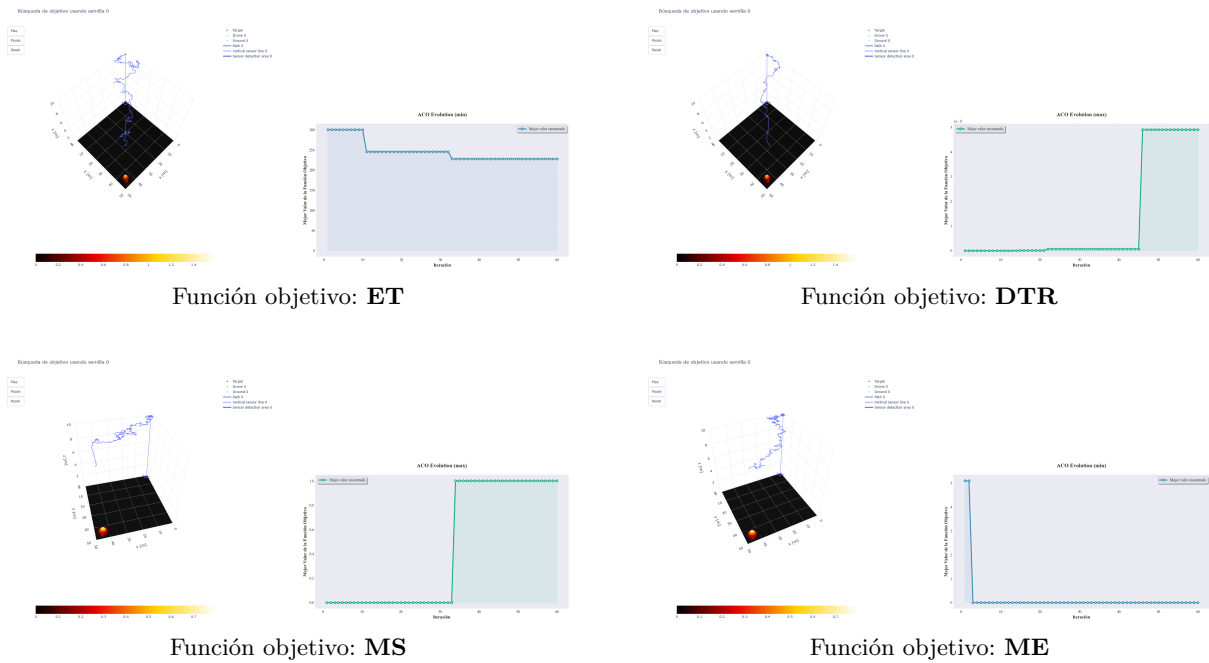


Figura 7.16: Resultados del algoritmo ACO para las cuatro funciones objetivo en el mapa 50×50 con 1 indicio: trayectorias (izquierda) y sus gráficas asociadas (derecha).

En este nuevo escenario, el algoritmo ACO presenta ciertas dificultades iniciales para avanzar hacia el indicio; sin embargo, finalmente logra localizarlo en todos los casos analizados. Cabe destacar el comportamiento de la métrica DTR, que en mapas de gran tamaño experimenta variaciones muy pequeñas debido a su factor de descuento. Aun así, ha sido capaz de identificar la trayectoria más corta entre las cuatro funciones evaluadas.

ABC

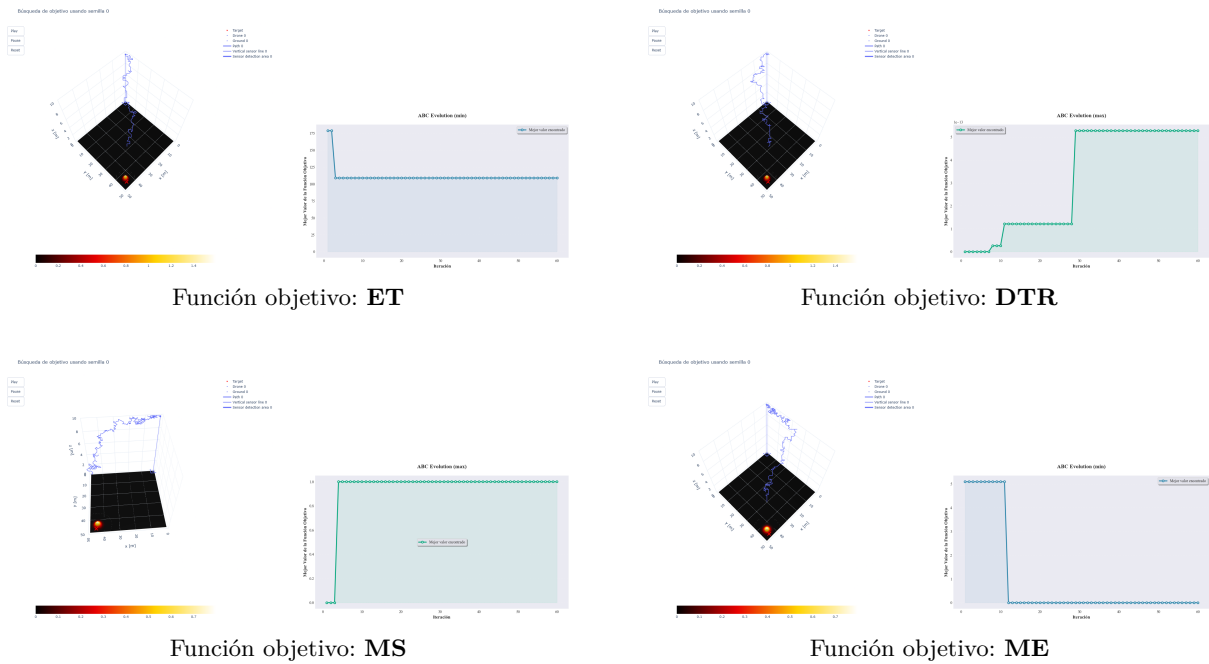


Figura 7.17: Resultados del algoritmo ABC para las cuatro funciones objetivo en el mapa 50×50 con 1 indicio: trayectorias (izquierda) y sus gráficas asociadas (derecha).

El algoritmo ABC no presenta dificultades significativas para dirigirse hacia el indicio. En términos generales, genera trayectorias de alta calidad y logra optimizar las funciones con un rendimiento comparable al obtenido mediante ACO. Cabe destacar la calidad de la trayectoria generada con ET.

BHA

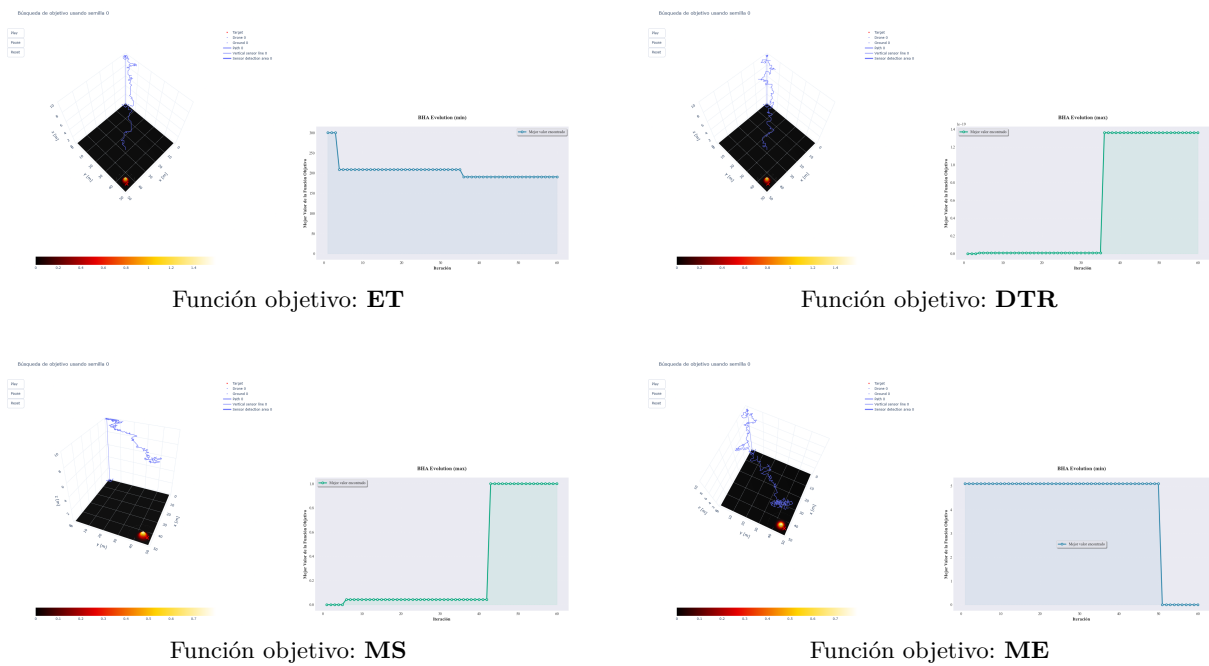


Figura 7.18: Resultados del algoritmo BHA para las cuatro funciones objetivo en el mapa 50×50 con 1 indicio: trayectorias (izquierda) y sus gráficas asociadas (derecha).

Al igual que los otros dos algoritmos, BHA genera trayectoria de alta calidad al cabo de unas cuantas iteraciones.

Comparación evoluciones de las funciones objetivo

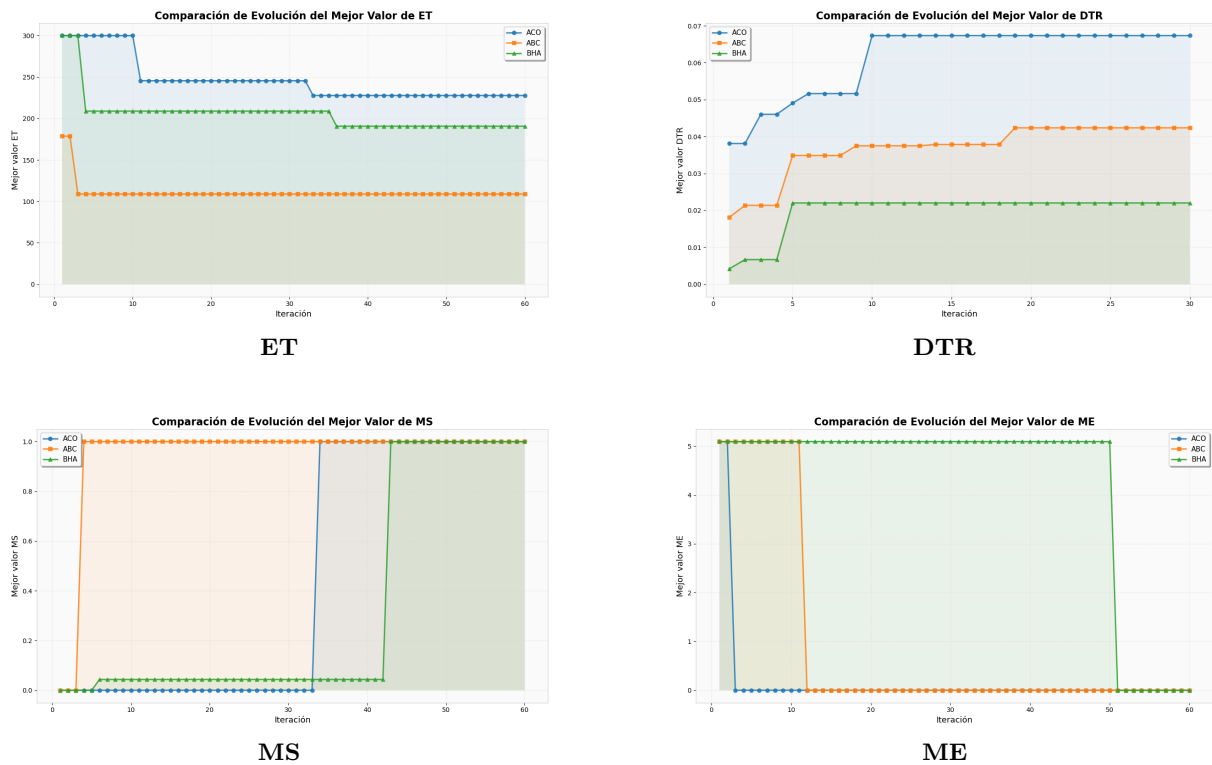


Figura 7.19: Comparación de la evolución de las cuatro funciones objetivo con ACO, ABC y BHA para el mapa 50x50 con 1 indicio.

En este caso, el algoritmo ACO obtiene trayectorias de mejor calidad según las métricas ET, DTR y ME. No obstante, en la métrica MS destaca el algoritmo ABC, ya que con un número reducido de iteraciones es capaz de localizar el objetivo de forma eficiente. En términos generales, ambos enfoques generan trayectorias de alta calidad que permiten una exploración satisfactoria del mapa de probabilidad.

2 Indicios

En este escenario se evidencia uno de los mayores problemas de los algoritmos voraces: estos solo eligen el paso que les aporta el mayor beneficio inmediato, sin considerar si dicha elección será la mejor a nivel global, quedando así atrapados en mínimos locales. En este sentido, los algoritmos bioinspirados destacan, ya que presentan un enfoque que equilibra la exploración y la explotación de soluciones. Al ser algoritmos de optimización, buscan encontrar soluciones que mejoren el resultado a nivel global, representando una mejora significativa frente a los métodos voraces.

Voraz miope

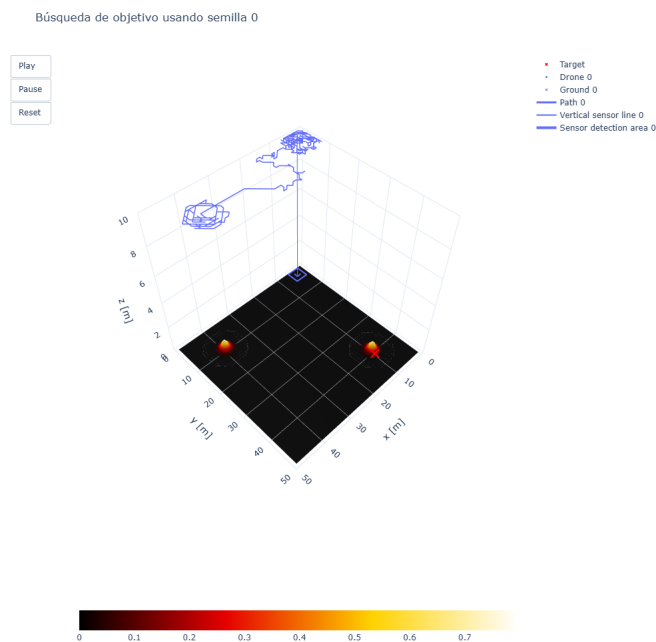


Figura 7.20: Trayectoria obtenida con el algoritmo voraz miope en el mapa 50×50 con 2 indicios.

Voraz con heurística

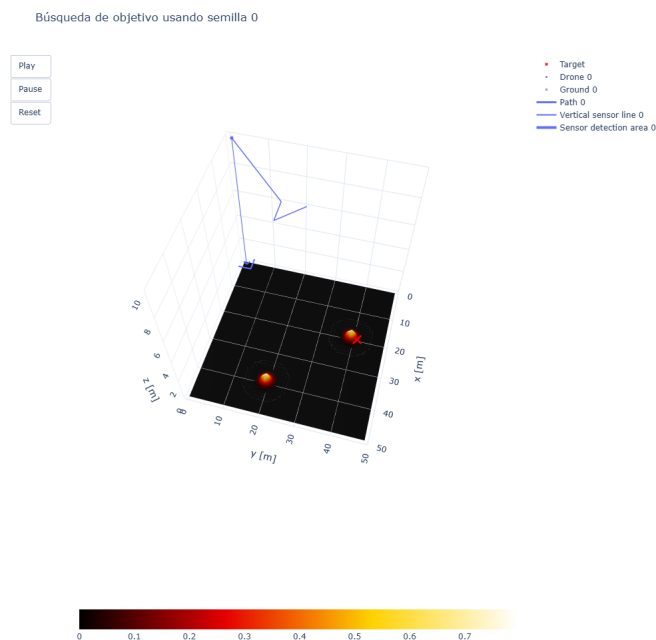


Figura 7.21: Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 50×50 con 2 indicios.

En este nuevo escenario, el comportamiento del algoritmo voraz, tanto en su versión miope como en la versión con heurística, difiere notablemente respecto a los métodos anteriores. En el caso del algoritmo miope, únicamente logra explorar uno de los indicios, recorriendo por completo el otro sin llegar a localizar el objetivo. Este resultado muestra por primera vez en este trabajo el impacto de la miopía, ya que

el algoritmo queda atrapado en un mínimo local. Por otro lado, la versión con heurística emplea una estrategia de corrección de miopía; sin embargo, debido a su propia naturaleza, concentra la exploración en una zona intermedia entre ambos indicios, impidiendo que llegue a explorar cualquiera de los dos y quedando igualmente atrapado. Este escenario pone de manifiesto las limitaciones de los algoritmos voraces ante situaciones de esta complejidad.

Cortacesped

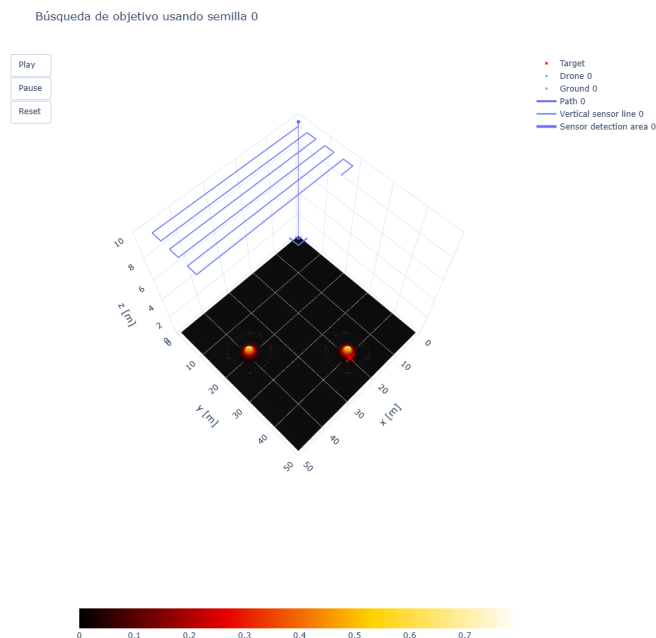


Figura 7.22: Trayectoria obtenida con el algoritmo de cortacesped en el mapa 50×50 con 2 indicios.

Expansión cuadrada

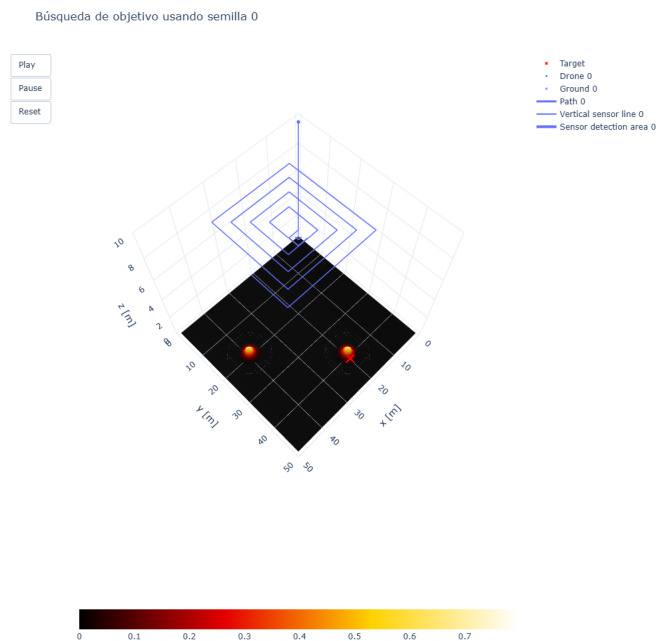


Figura 7.23: Trayectoria obtenida con el algoritmo de expansión cuadrada en el mapa 50×50 con 2 indicios.

De manera similar a lo observado en la situación anterior, estos dos algoritmos no cuentan con pasos suficientes para explorar el mapa hasta alcanzar el objetivo.

ACO

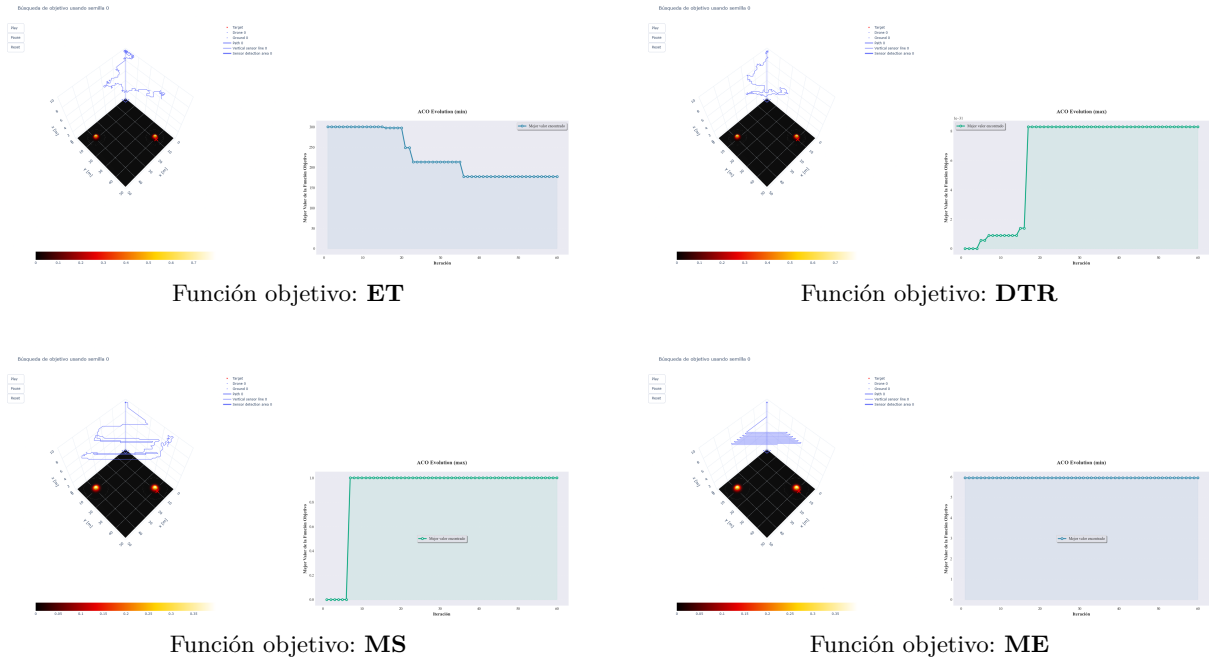


Figura 7.24: Resultados del algoritmo ACO para las cuatro funciones objetivo en el mapa 50×50 con 2 indicios: trayectorias (izquierda) y sus gráficas asociadas (derecha).

En esta sección se presentan diversas situaciones. Al aplicar la heurística de corrección de miopía, la cual dirige la búsqueda hacia la intersección de los dos indicios, se observa que ACO inicia la exploración desde el centro del mapa. En los casos de ET y MS, el algoritmo logra localizar el objetivo; sin embargo, para DTR y ME no es capaz de encontrarlo, mostrando especialmente dificultades significativas en el caso de ME. Cabe destacar que la elección de la heurística en este escenario concreto influye mucho, y para obtener resultados mejores bastaría con reducir el peso de la heurística bajando el valor de el parámetro beta.

ABC

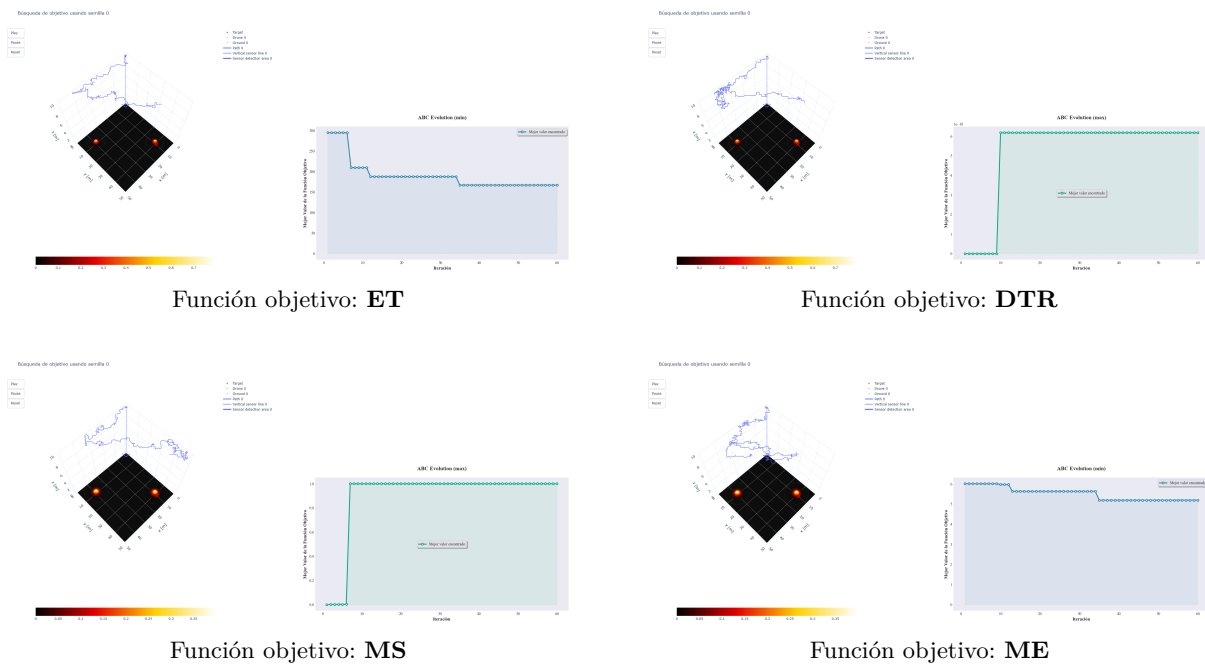


Figura 7.25: Resultados del algoritmo ABC para las cuatro funciones objetivo en el mapa 50×50 con 2 indicios: trayectorias (izquierda) y sus gráficas asociadas (derecha).

El algoritmo ABC presenta una dificultad ligeramente menor al explorar el mapa de probabilidades. A diferencia de ACO, ABC logra localizar el objetivo en los casos de ET y MS; sin embargo, no consigue hacerlo en el caso de ME.

BHA

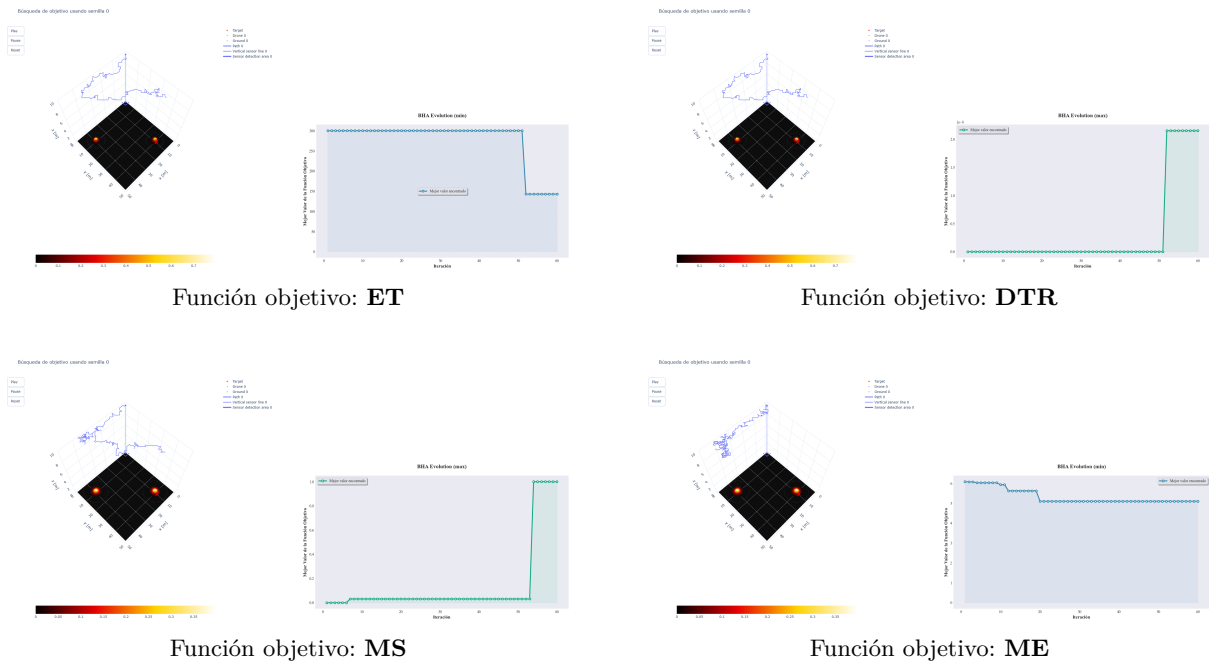


Figura 7.26: Resultados del algoritmo BHA para las cuatro funciones objetivo en el mapa 50×50 con 2 indicios: trayectorias (izquierda) y sus gráficas asociadas (derecha).

El algoritmo BHA logra localizar el objetivo al utilizar las métricas ET, DTR y MS, generando trayectorias de alta calidad. Sin embargo, al emplear la métrica ME, no consigue explorar de manera efectiva el segundo indicio.

Comparación evoluciones de las funciones objetivo

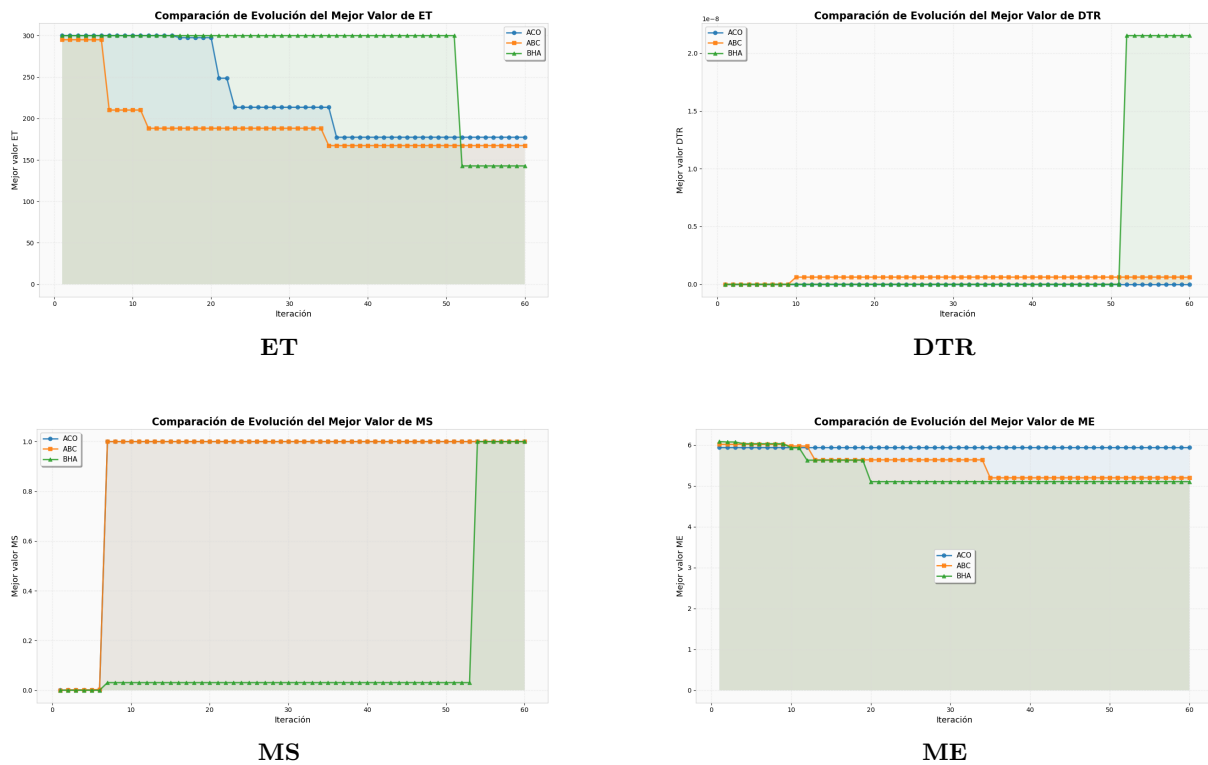


Figura 7.27: Comparación de la evolución de las cuatro funciones objetivo con ACO, ABC y BHA para el mapa 50x50 con 2 indicio.

En este escenario, los tres algoritmos logran alcanzar optimizaciones muy similares y presentan el mismo problema con la métrica ME, lo que evidencia que esta métrica no es adecuada para este tipo de situaciones. En cambio, para ET y MS, los tres algoritmos consiguen localizar el objetivo. Es importante destacar que, para distintos mapas, el desempeño de los algoritmos bioinspirados depende en gran medida de los ajustes de sus parámetros, así como de la heurística empleada, la cual influye significativamente en su rendimiento.

7.1.3 Mapa 100x100

Para este mapa en el caso de las funciones objetivo solo se utilizará la función ET por comodidad ya que más adelante con las pruebas masivas se analizarán todas las funciones más en profundidad.

Voraz miope

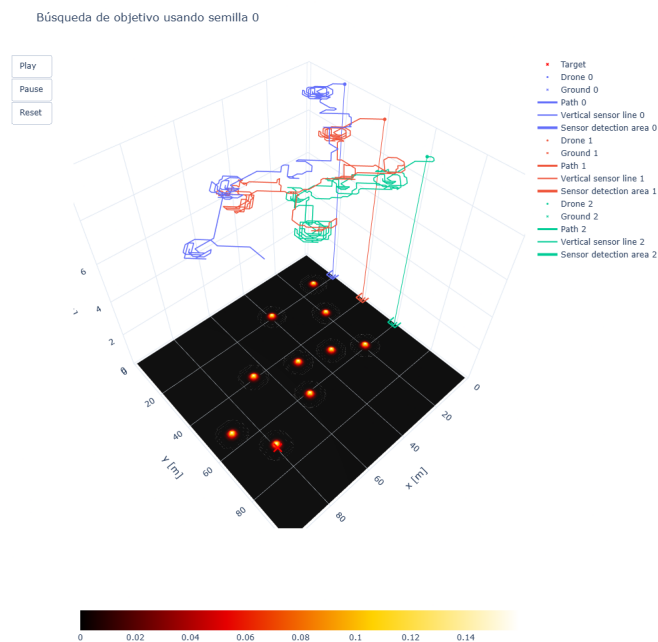


Figura 7.28: Trayectoria obtenida con el algoritmo voraz miope en el mapa 50×50 con 1 indicio.

Voraz con heurística

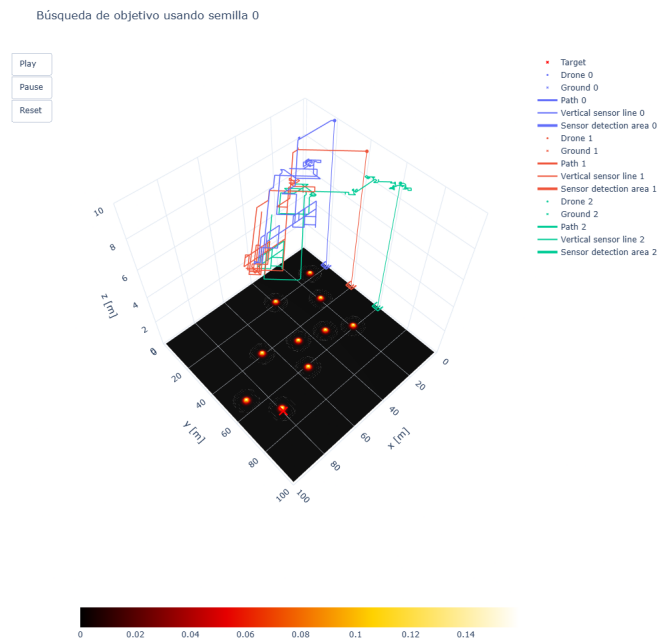


Figura 7.29: Trayectoria obtenida con el algoritmo voraz con heurística en el mapa 100×100 con 10 indicios y 3 agentes.

El algoritmo voraz miope logra generar una trayectoria muy eficiente en este mapa tan complejo, explorando con éxito los indicios presentes y encontrando finalmente el objetivo. En su versión con heurística, parece tener un rendimiento ligeramente inferior debido al elevado contenido de indicios y el uso de la heurística de corrección de miopía.

Cortacesped

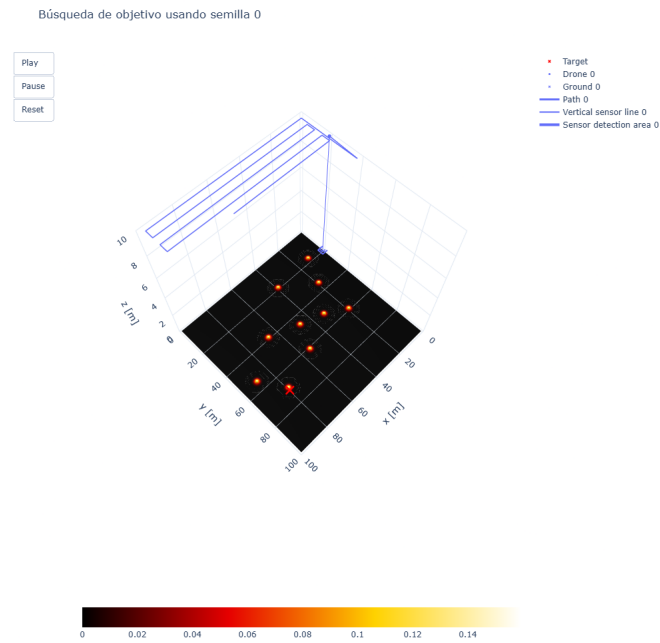


Figura 7.30: Trayectoria obtenida con el algoritmo de cortacesped en el mapa 100×100 con 10 indicios y 3 agentes.

Expansión cuadrada

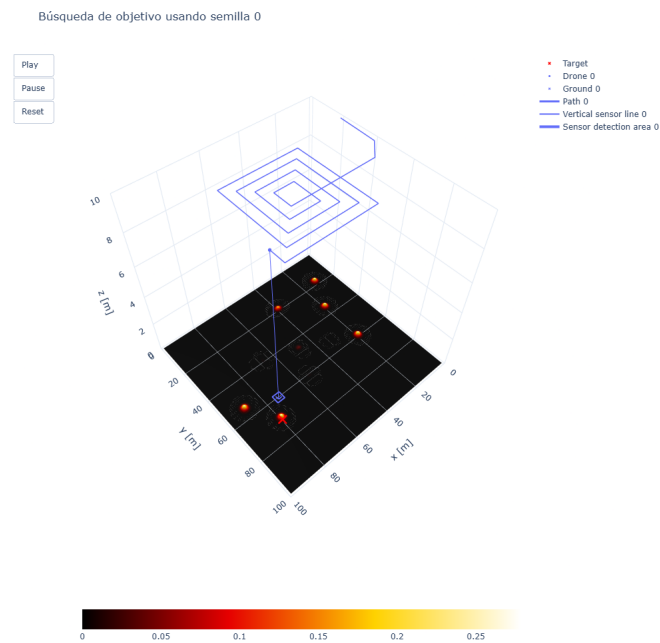


Figura 7.31: Trayectoria obtenida con el algoritmo de expansion cuadrada en el mapa 100×100 con 10 indicios y 3 agentes.

El algoritmo de cortacésped y el de expansión consiguen el mismo resultado que en los mapas anteriores.

ACO

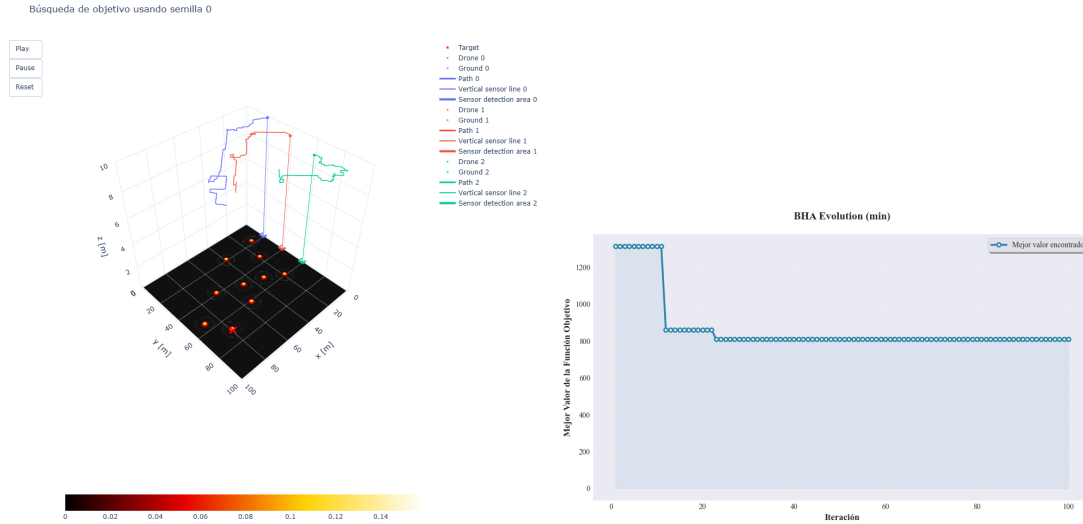


Figura 7.32: Trayectoria obtenida con el algoritmo ACO junto con la función ET en el mapa 100×100 con 10 indicios y 3 agentes.

El algoritmo ACO logra optimizar la función hasta localizar la función objetivo en el mapa. Sin embargo, después de unas pocas iteraciones deja de minimizar el ET, posiblemente debido a que la feromona no se evapora con suficiente rapidez para este mapa, destacando la importancia de ajustar los parámetros de ACO adecuadamente para cada escenario. Aquí es importante notar que los algoritmos bioinspirados son algoritmos de optimización, y para que funcionen requieren siempre de varias iteraciones, por lo que cuentan con cierta 'ventaja' respecto a algoritmos de búsqueda tradicionales pues recorren el mapa más veces, y aquí se puede notar que los agentes van casi directamente a la posición del objetivo ya que es la mejor forma de reducir ET.

ABC

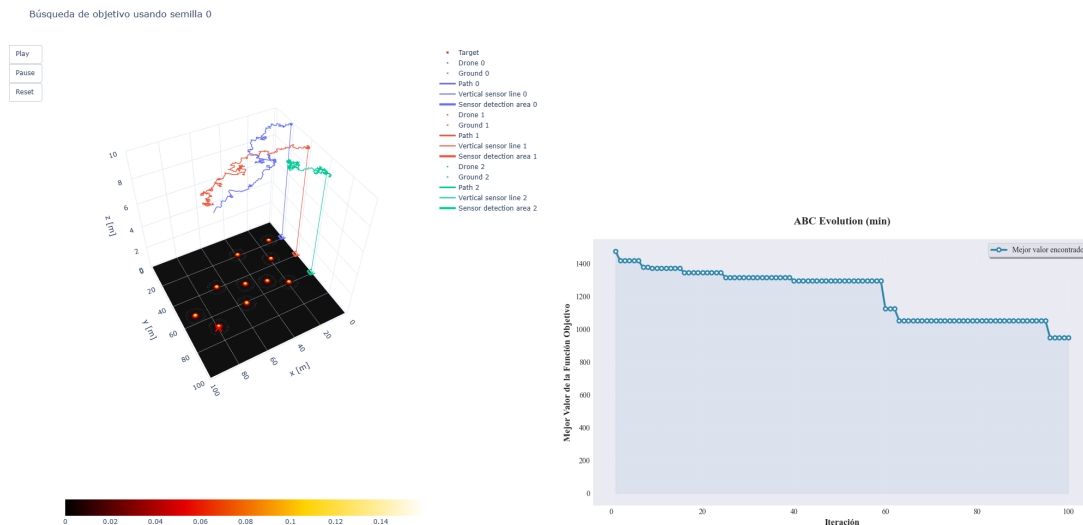


Figura 7.33: Trayectoria obtenida con el algoritmo ABC junto con la función ET en el mapa 100×100 con 10 indicios y 3 agentes.

ABC igualmente explora bien el mapa y consigue encontrar al objetivo, pero optimizando menos ET, aunque a diferencia de ACO consigue seguir minimizando la función pasadas las primeras iteraciones.

BHA

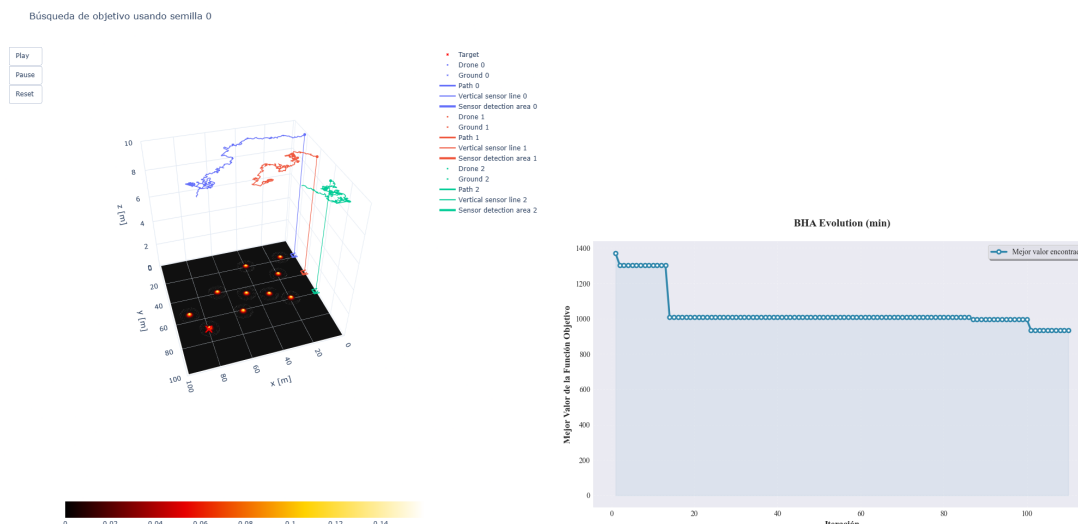


Figura 7.34: Trayectoria obtenida con el algoritmo BHA junto con la función ET en el mapa 100×100 con 10 indicios y 3 agentes.

Con BHA ocurre un comportamiento similar; sin embargo, el agente que sigue la trayectoria azul parece no llegar a explorar completamente el mapa. Aunque logra reducir significativamente el valor de la función objetivo, lo hace en menor medida que ACO.

Comparación evoluciones de ET

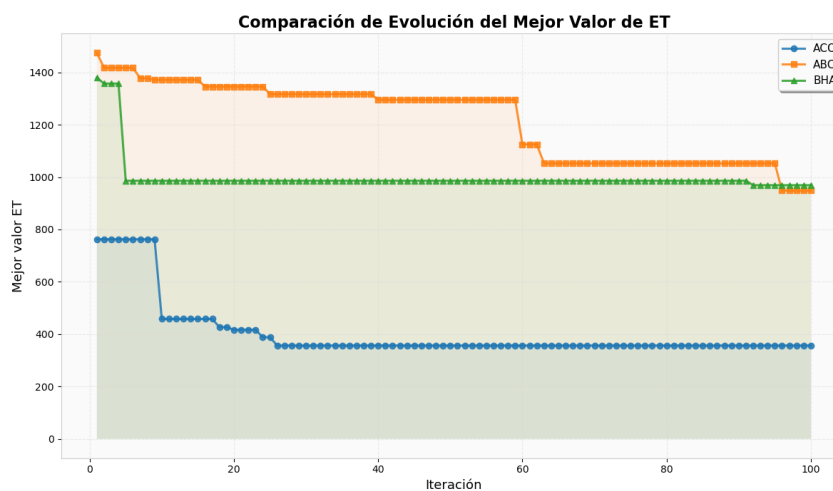


Figura 7.35: Comparación de la evolución de ET con ACO, ABC y BHA para el mapa 100x100.

Para este mapa se evidencia un claro ganador: ACO, ya que logra reducir significativamente ET en comparación con ABC y BHA, los cuales presentan una evolución casi similar.

7.2 Pruebas masivas

Las pruebas a alto nivel se han dividido en cuatro escenarios representativos, en función del número de agentes e indicios involucrados:

1. **Un solo agente y un solo indicio:** Escenario base para evaluar el comportamiento de los algoritmos en la situación más simple. Se utilizó:
 - **Agentes:** 1
 - **Indicios:** 1
 - **Número de iteraciones:** 1000
2. **Un solo agente con varios indicios:** Se analiza la capacidad de un único agente para localizar el objetivo entre múltiples indicios. Se probaron:
 - **Agentes:** 1
 - **Indicios:** 1, 2, 4, 6, 8, 10
 - **Número de iteraciones:** 1000
3. **Varios agentes con un solo indicio:** Este escenario permite estudiar la eficiencia de varios agentes trabajando sobre un único indicio. Se evaluaron:
 - **Agentes:** 5, 10, 15, 20, 25, 30
 - **Indicios:** 1
 - **Número de iteraciones:** 1000
4. **Varios agentes y varios indicios:** Escenario más complejo, en el que múltiples agentes deben localizar múltiples indicios. Para simplificar los tiempos de ejecución, se eligieron configuraciones representativas:
 - **Agentes:** 1, 5
 - **Indicios:** 1, 5
 - **Número de iteraciones:** 1000

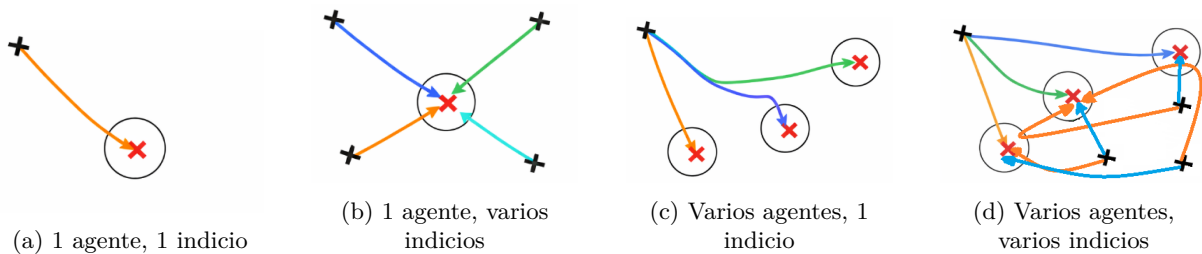


Figura 7.36: Representación visual de los cuatro escenarios de pruebas masivas.

Para todos los escenarios se evalúan dos mapas distintos: uno mediano de 60×60 y uno grande de 120×120 . Es decir, cada algoritmo se ejecuta en ambos mapas para cada configuración de agentes e indicios, evaluando todas las métricas en los dos tamaños de mapa.

En todos los escenarios, las posiciones de los agentes y de los indicios se generan de manera completamente aleatoria, y el objetivo se sitúa dentro de uno de los indicios seleccionados aleatoriamente.

Se evalúan todas las combinaciones de agentes e indicios descritas en cada escenario y mapa para cada algoritmo considerado: *voraz-myope*, *voraz-heur*, *lawnmower*, *expanding_sq*, *ACO*, *ABC* y *BHA*. Además, se calculan los valores de todas las funciones objetivo para los algoritmos bioinspirados.

Dado el elevado número de pruebas posibles, la diversidad de algoritmos disponibles, la variedad de funciones objetivo y las múltiples combinaciones de escenarios de evaluación, no ha sido factible realizar la totalidad de las pruebas potenciales. Por este motivo, se ha optado por ejecutar un número reducido de experimentos, limitado a 200 iteraciones por cada combinación considerada, y restringiendo el análisis solo usando el mapa de tamaño medio.

A pesar de la reducción de pruebas respecto a la planificación inicial, estos experimentos permiten identificar tendencias generales en el rendimiento de los algoritmos, incluyendo número de pasos, distancia recorrida, probabilidad de localizar el objetivo y nivel de optimización alcanzado por los algoritmos bioinspirados.

7.2.1 Análisis de resultados

En esta sección se analizan las métricas obtenidas con los datos de las pruebas.

Número de pasos

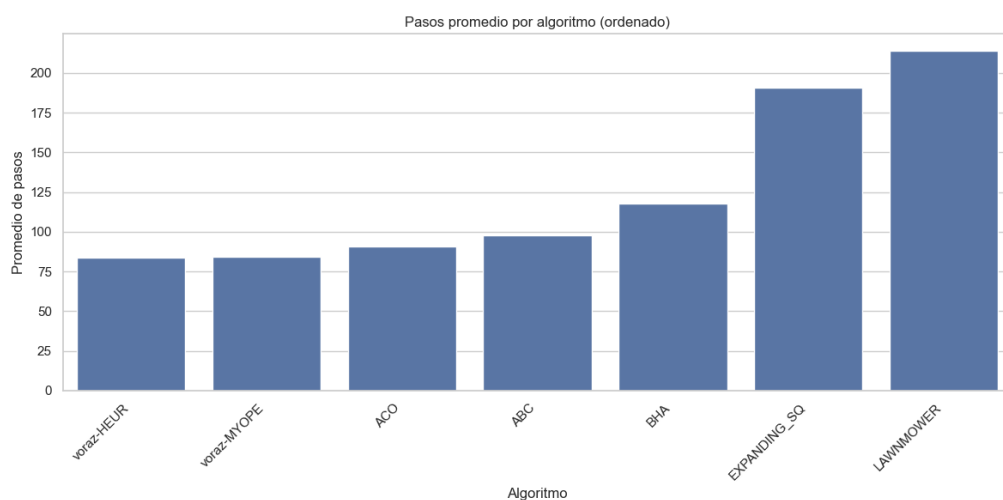


Figura 7.37: Diagrama de barras ordenado del promedio de pasos por algoritmo.

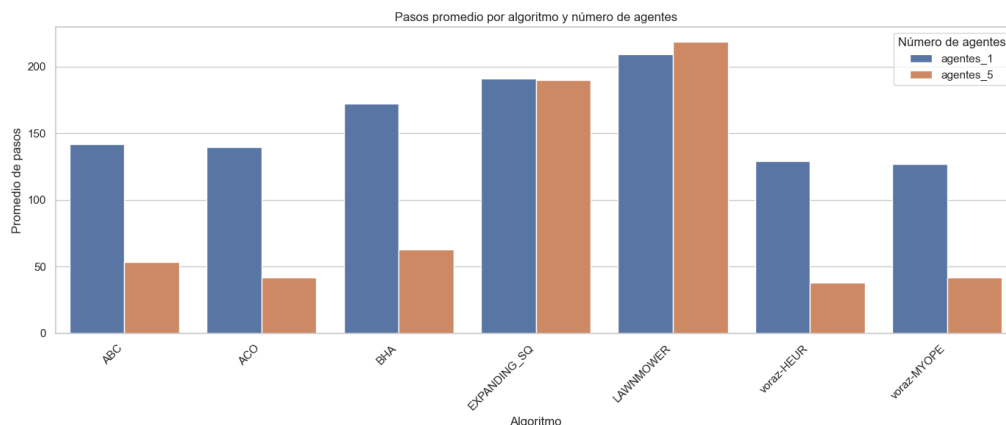


Figura 7.38: Diagrama de barras del promedio de pasos por algoritmo y número de agentes.

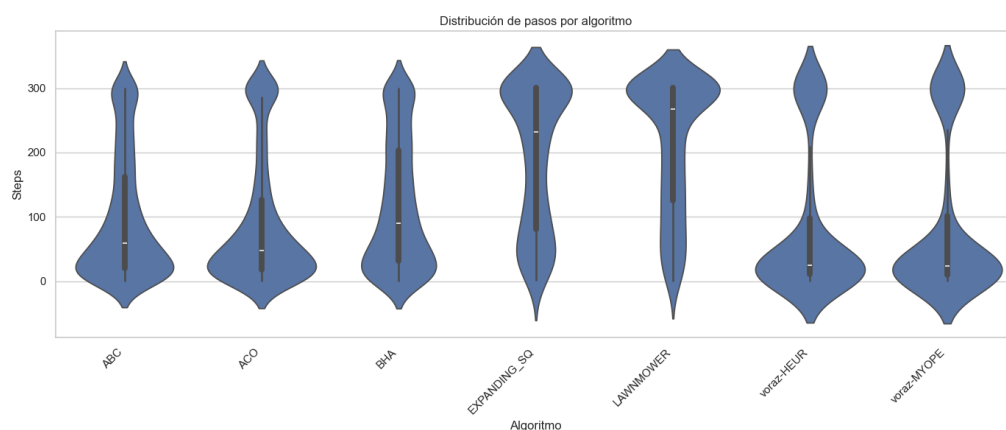


Figura 7.39: Diagrama de violín de los pasos promedio por algoritmo.

En cuanto al número de pasos por algoritmo, se observa que los algoritmos de fuerza bruta presentan el mayor número de pasos promedio, como era de esperar. A continuación se encuentran los algoritmos bioinspirados y los algoritmos voraces, entre los cuales no se aprecian diferencias significativas.

Respecto al número de pasos en función del número de agentes, se evidencia que a mayor cantidad de agentes, se requieren menos pasos para encontrar el objetivo, con la excepción de los algoritmos de fuerza bruta.

Dentro de los algoritmos bioinspirados, ACO es el que obtiene mejor desempeño tanto para un solo agente como para cinco agentes, lo cual se refleja en su promedio de pasos por número de agentes y en su número de pasos promedio general.

En el diagrama de violín se puede observar cómo las zonas más densas de los algoritmos de fuerza bruta se concentran en valores promedio más elevados, mientras que para el resto de los algoritmos estos se mantienen por debajo. En el caso del algoritmo BHA, se aprecia que la distribución del número de pasos es casi uniforme.

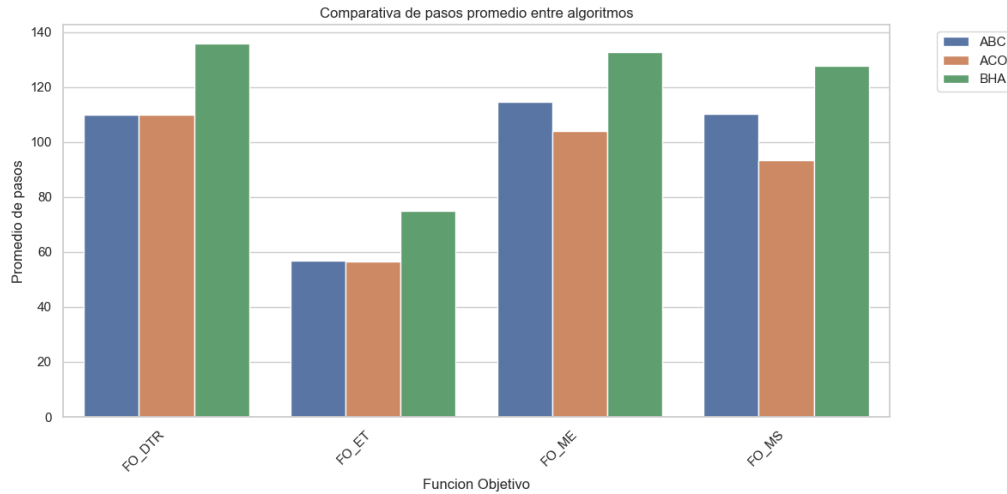


Figura 7.40: Diagrama de barras del promedio de pasos por algoritmo bioinspirado y función objetivo.

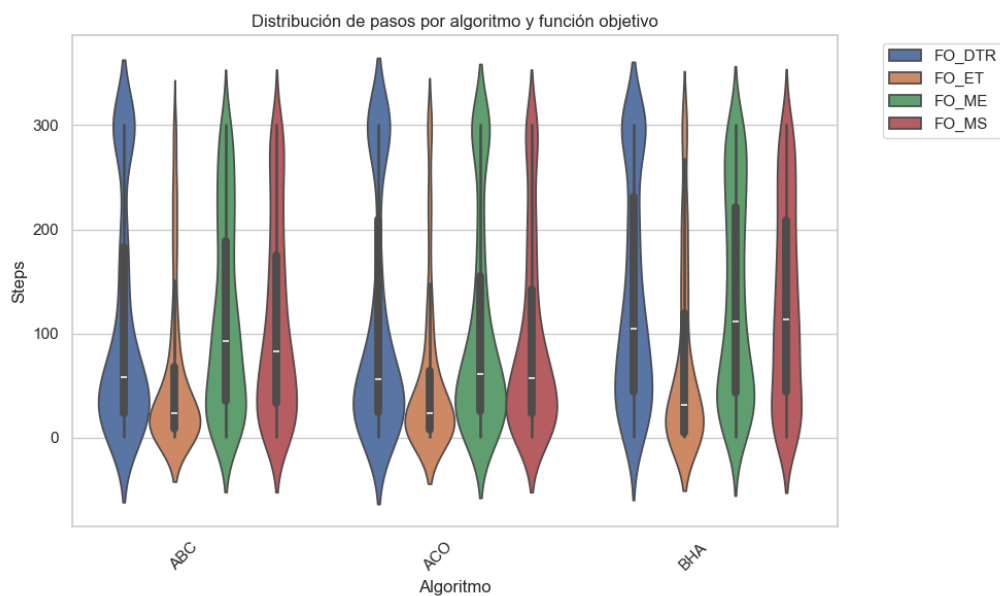


Figura 7.41: Diagrama de violín de los pasos promedio por algoritmo bioinspirado y función objetivo.

Entrando en mayor detalle en las funciones objetivo, se observa que ACO y ABC presentan un comportamiento similar para las funciones ET y DTR, mientras que BHA muestra un promedio de pasos superior en todas las funciones. Por su parte, ACO mantiene un promedio menor para las funciones ME y MS, y la función ET es la que contribuye a que los tres algoritmos bioinspirados presenten un promedio de pasos reducido.

En el diagrama de violín, las funciones MS y ME muestran una distribución bastante uniforme para los algoritmos ABC y BHA. Cabe destacar que, para la función ET, en los tres algoritmos la densidad principal se encuentra en valores bajos, lo que indica que esta función conduce a trayectorias con un menor número de pasos.

Distancia

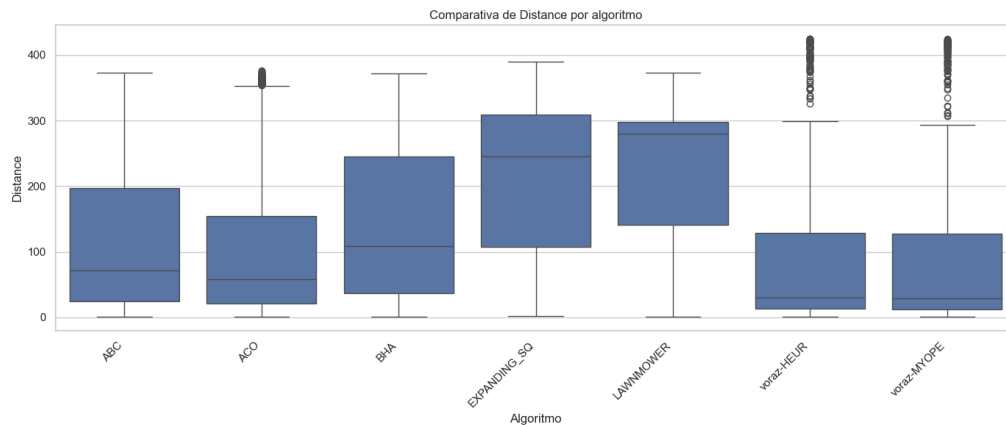


Figura 7.42: Conjunto de diagramas de cajas y bigotes de las distancias recorridas por algoritmo.

Respecto a las distancias recorridas, de manera similar a lo observado para el número de pasos, los algoritmos de fuerza bruta presentan las mayores distancias recorridas, sin una presencia significativa de valores atípicos, como se puede apreciar en los diagramas de cajas y bigotes. Los algoritmos voraces muestran las distancias más bajas en promedio, aunque con numerosos outliers que, en muchos casos, superan incluso los valores de los algoritmos de fuerza bruta. Esto podría indicar que, en ciertos escenarios, los algoritmos voraces no logran encontrar el objetivo de manera eficiente y recorren grandes distancias sin obtener información relevante. En cuanto a los algoritmos bioinspirados, ACO es el que recorre la menor distancia promedio, aunque con algunos outliers destacados, seguido por ABC y, finalmente, BHA, que es el que recorre la mayor distancia promedio.

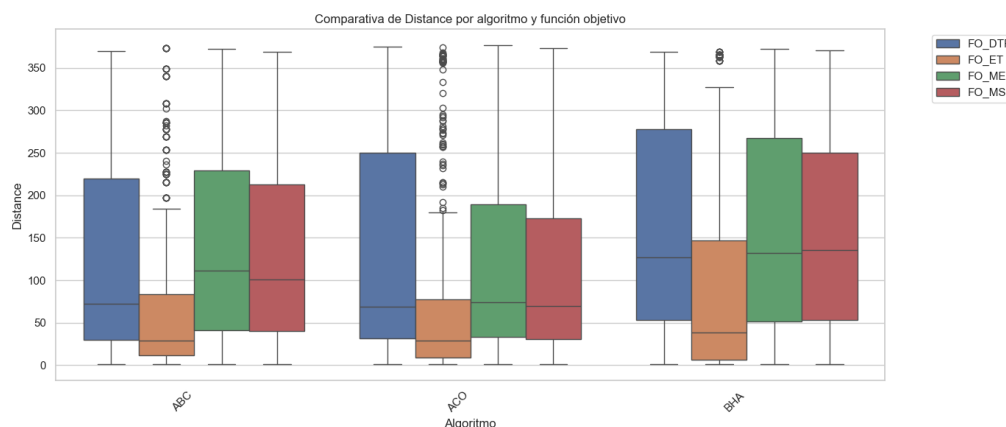


Figura 7.43: Conjunto de diagramas de cajas y bigotes de las distancias recorridas por algoritmo bioinspirado y función objetivo.

Al analizar las distancias recorridas por cada algoritmo en función de cada objetivo, se observa nuevamente un comportamiento destacado de ET, que presenta un número de pasos considerablemente menor que las demás funciones objetivo para los tres algoritmos, seguido de MS, DTR y ME.

Tasa de éxito

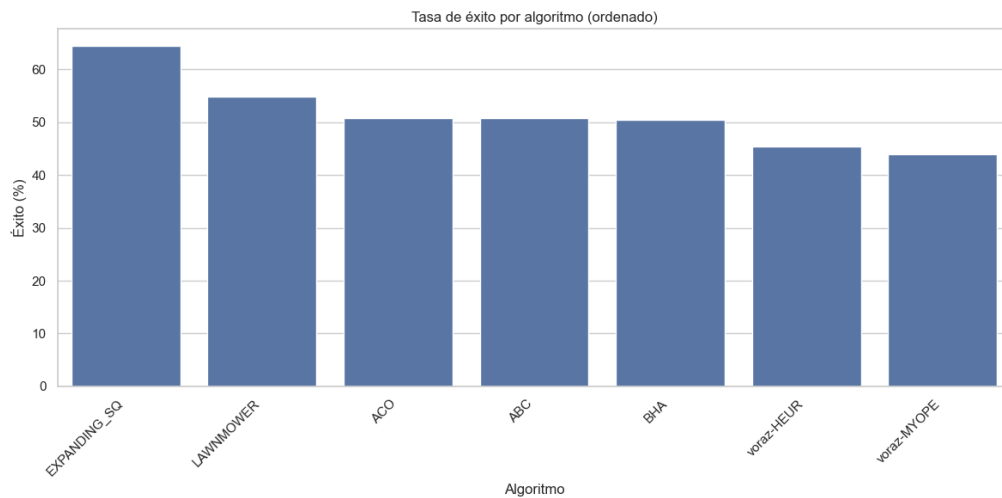


Figura 7.44: Diagrama de barras de tasa de éxito general por algoritmo.

Otro factor relevante para evaluar la viabilidad de los algoritmos bioinspirados en este tipo de tareas es la tasa de éxito, definida como la proporción de veces que los agentes logran encontrar el objetivo. Resulta interesante observar que los algoritmos bioinspirados presentan una tasa de éxito superior a la de los algoritmos voraces. Los algoritmos de fuerza bruta muestran la mayor tasa de éxito, dado que, para un mapa con estas características y con los indicios y agentes colocados de forma aleatoria, es muy probable que encuentren el objetivo. No obstante, su principal desventaja es que requieren un mayor número de pasos que todos los demás algoritmos, como se ha mostrado anteriormente. A partir de estos resultados, se puede concluir que los algoritmos bioinspirados son viables para tareas de búsqueda, alcanzando una tasa de éxito aproximada del 50 %, lo que sugiere perspectivas prometedoras.

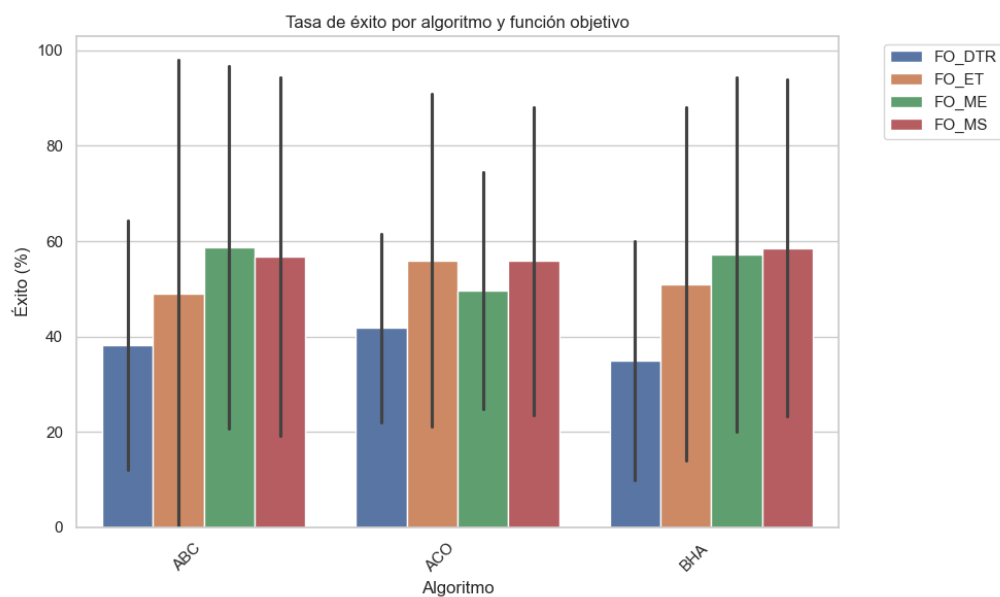


Figura 7.45: Diagrama de barras de tasa de éxito general por algoritmo bioinspirado y función objetivo.

Para cada función objetivo, resulta interesante observar que cada algoritmo alcanza su mayor tasa de éxito con funciones distintas: en el caso de ACO, la función que mejor rendimiento muestra es ET; para

ABC, ME; y para BHA, MS. Esto se debe a que, para estas funciones objetivo, cada algoritmo obtiene la mayor tasa de éxito. Por el contrario, la función DTR se asocia con una tasa de éxito menor para los tres algoritmos.

En general, los algoritmos bioinspirados han demostrado ser competitivos frente a los algoritmos tradicionales, obteniendo en algunos casos resultados superiores. Además, se evidencia que, de manera general, para los atributos medidos en los datos, la función objetivo ET resulta más conveniente para este tipo de problemas de búsqueda.

Capítulo 8

Conclusiones y líneas futuras

8.1 Conclusiones

En este capítulo se ha presentado el marco teórico que sustenta el problema de búsqueda probabilística multi-UAV. Se ha descrito la evolución histórica de la búsqueda probabilística, desde sus orígenes en aplicaciones militares hasta los desarrollos recientes en sistemas autónomos cooperativos. Se ha justificado la complejidad *NP-Hard* del problema y la necesidad de recurrir a métodos aproximados en lugar de algoritmos exactos.

Las técnicas clásicas analizadas proporcionan una base importante para comprender las limitaciones de los enfoques tradicionales. En particular, el problema de la miopía identificado en las búsquedas voraces motiva el desarrollo de algoritmos más sofisticados que puedan equilibrar exploración y explotación de forma más efectiva.

Este análisis del estado del arte establece el contexto necesario para comprender la propuesta metodológica del presente trabajo, que se centra en el uso de algoritmos bioinspirados (ACO, ABC, BHA) para superar las limitaciones de las técnicas clásicas y generar trayectorias de búsqueda más eficientes en entornos probabilísticos complejos.

En este trabajo de fin de grado se ha abordado de manera integral el problema de la planificación de trayectorias de búsqueda en entornos con incertidumbre, un desafío de gran relevancia en aplicaciones como la exploración autónoma y las misiones de búsqueda y rescate con vehículos aéreos no tripulados (UAVs). La complejidad inherente de este problema proviene, por un lado, de la incertidumbre asociada a la localización del objetivo y, por otro, del carácter combinatorio de la planificación de rutas, lo que lo sitúa dentro de la clase de los problemas *NP-Hard*. Esta naturaleza hace impracticable el uso de métodos exactos en escenarios realistas, reforzando la necesidad de recurrir a técnicas aproximadas capaces de ofrecer soluciones de calidad en tiempos computacionales razonables.

A lo largo del trabajo se ha puesto de manifiesto cómo las estrategias de búsqueda tradicionales, basadas en criterios locales o voraces, pueden verse seriamente afectadas por el fenómeno de la miopía, dando lugar a trayectorias que, si bien resultan óptimas a corto plazo, conducen a un comportamiento global ineficiente. Este efecto limita de forma significativa su aplicabilidad en entornos complejos y dinámicos. Frente a esta problemática, los algoritmos bioinspirados emergen como una alternativa especialmente adecuada, al incorporar mecanismos de cooperación, adaptación y aleatoriedad controlada que permiten equilibrar de forma natural la exploración y la explotación del espacio de búsqueda.

Desde un punto de vista metodológico, se ha diseñado y desarrollado un entorno de simulación flexible que permite modelar mapas de probabilidad, generar trayectorias de búsqueda y evaluar su rendimiento

bajo distintas condiciones. En este marco se han implementado diversas funciones objetivo —*Expected Time* (ET), *Discounted Time Reward* (DTR), *Maximum Slope* (MS) y *Minimum Entropy* (ME)—, cada una de ellas orientada a capturar distintos criterios de eficiencia en el proceso de búsqueda, tales como la rapidez en la detección del objetivo, la acumulación de recompensa descontada o la reducción progresiva de la incertidumbre. Este enfoque ha permitido analizar el problema desde múltiples perspectivas y evidenciar que no existe una métrica universalmente óptima, sino que la idoneidad de cada función depende del objetivo concreto de la misión.

Asimismo, se han implementado y analizado distintos algoritmos bioinspirados, como *Ant Colony Optimization* (ACO), *Artificial Bee Colony* (ABC) y *Black Hole Algorithm* (BHA). El estudio detallado de sus parámetros de control ha puesto de relieve la influencia decisiva que estos ejercen sobre el comportamiento del algoritmo, especialmente en términos de capacidad de exploración, velocidad de convergencia y calidad de las trayectorias generadas. En este sentido, el entorno de pruebas desarrollado ha resultado fundamental para comparar de manera sistemática las fortalezas y limitaciones de cada enfoque, así como para analizar su sensibilidad a la configuración de parámetros.

Los resultados experimentales obtenidos muestran que los algoritmos bioinspirados constituyen una herramienta viable, robusta y eficaz para la búsqueda de objetivos en mapas con incertidumbre. En comparación con técnicas tradicionales, estos algoritmos alcanzan tasas de éxito superiores y generan trayectorias más equilibradas, manteniendo al mismo tiempo un número de pasos y un coste computacional aceptables. No obstante, los resultados también ponen de manifiesto que la calidad de las soluciones obtenidas está estrechamente ligada tanto a la función objetivo seleccionada como al ajuste de los parámetros del algoritmo, lo que subraya la importancia de un diseño cuidadoso del proceso de optimización.

No obstante, es importante reconocer las limitaciones encontradas durante el desarrollo de este trabajo. En primer lugar, los recursos computacionales disponibles han condicionado la exhaustividad de los experimentos, especialmente en lo relativo al ajuste fino de los parámetros de cada algoritmo bioinspirado. Estos parámetros —como el número de hormigas en ACO, el tamaño de la población en ABC o el radio de influencia en BHA— requieren un proceso de calibración meticuloso que puede influir de manera drástica en los resultados obtenidos. Dado el amplio espacio de configuraciones posibles y el coste temporal asociado a cada simulación, no ha sido posible explorar todas las combinaciones óptimas para cada escenario, lo que podría haber afectado al rendimiento observado de algunos algoritmos.

Adicionalmente, debido a la extensión del trabajo y las limitaciones temporales, no se ha podido evaluar en profundidad el comportamiento de estos algoritmos en espacios de búsqueda de gran tamaño. La literatura científica sugiere que los algoritmos bioinspirados muestran su mayor potencial precisamente en escenarios de elevada dimensionalidad, donde las técnicas tradicionales se vuelven impracticables. Por tanto, aunque los resultados obtenidos en entornos de tamaño medio son prometedores, quedan pendientes de validación las ventajas esperadas en espacios más extensos y complejos.

8.2 Líneas futuras

El trabajo desarrollado abre diversas líneas de investigación y mejora que permitirían incrementar la calidad de las trayectorias generadas y la robustez del sistema de planificación. Entre las posibles extensiones destacan las siguientes:

- **Optimización automática de parámetros.** Implementar técnicas de ajuste automático de hiperparámetros, como búsqueda en rejilla (*grid search*), optimización bayesiana o algoritmos

evolutivos, permitiría identificar configuraciones óptimas de forma sistemática y reducir la dependencia del ajuste manual.

- **Actualización dinámica de parámetros para distintos entornos.** Ajustar de manera adaptativa parámetros como α , β o la tasa de evaporación permitiría que el algoritmo reaccionara en tiempo real a escenarios cambiantes, mejorando el equilibrio entre exploración y explotación.
- **Combinación o hibridación de algoritmos.** La mezcla de varios métodos bioinspirados o incluso la integración con técnicas clásicas de búsqueda o planificación podría mejorar la calidad de las soluciones y acelerar la convergencia.
- **Funciones objetivo más sofisticadas.** Incluir funciones que integren múltiples criterios —como restricciones energéticas del UAV, riesgo operativo, cobertura del terreno, persistencia del objetivo o modelos de movimiento del objetivo— permitiría resolver escenarios más realistas y complejos.
- **Estudio de más algoritmos bioinspirados para este problema.** Explorar otras técnicas inspiradas en la naturaleza o algoritmos de optimización más avanzados podría ofrecer mejoras en eficiencia, robustez y capacidad de adaptación frente a entornos de búsqueda complejos.
- **Evaluación en espacios de búsqueda de gran tamaño.** Ampliar la experimentación a mapas de mayor dimensión permitiría validar las ventajas teóricas de los algoritmos bioinspirados en escenarios donde su potencial se maximiza, tal como sugiere la literatura.
- **Validación en escenarios reales o simulaciones físicas más avanzadas.** Probar el sistema en simuladores de vuelo de mayor fidelidad o en condiciones reales permitiría evaluar su rendimiento frente a ruido sensorial, restricciones físicas y condiciones externas.

Estas líneas futuras permiten ampliar el alcance de los algoritmos desarrollados y avanzar hacia soluciones más eficientes, robustas y aplicables a misiones reales de búsqueda y localización con UAVs.

Bibliografía

- [1] B. O. Koopman, *Search and Screening: Statistical Methods in Search and Rescue*. Operations Evaluation Group, U.S. Navy, 1946, Originally classified; declassified and republished in 1980.
- [2] B. O. Koopman, *Search and Screening: General Principles with Historical Applications*. New York: Pergamon Press, 1980.
- [3] S. B. McGrayne, *The Theory That Would Not Die: How Bayes' Rule Cracked the Enigma Code, Hunted Down Russian Submarines, and Emerged Triumphant from Two Centuries of Controversy*. New Haven: Yale University Press, 2014.
- [4] R. R. Richardson y L. D. Stone, "Operations Analysis During the Search for the Submarine Scorpion", *Naval Research Logistics Quarterly*, vol. 18, n.º 4, págs. 353-365, 1971. DOI: [10.1002/nav.3800180404](https://doi.org/10.1002/nav.3800180404).
- [5] L. D. Stone, *Theory of Optimal Search*. New York: Academic Press, 1975, Winner of the Lanchester Prize.
- [6] S. M. Pollock, "Search Detection and Surveillance", *Operations Research*, vol. 18, n.º 2, págs. 246-272, 1970.
- [7] J. M. Dobbie, "Search Theory: A Game Theoretic Perspective", *Naval Research Logistics Quarterly*, vol. 21, n.º 1, págs. 1-14, 1974.
- [8] K. E. Trummel y J. R. Weisinger, "The Complexity of the Optimal Searcher Path Problem", *Operations Research*, vol. 34, n.º 2, págs. 324-327, 1986. DOI: [10.1287/opre.34.2.324](https://doi.org/10.1287/opre.34.2.324).
- [9] F. Bourgault, T. Furukawa y H. F. Durrant-Whyte, "Optimal Search for a Lost Target in a Bayesian World", *Advanced Robotics*, vol. 20, n.º 3, págs. 361-383, 2006. DOI: [10.1163/156855306776985582](https://doi.org/10.1163/156855306776985582).
- [10] P. Lanillos, A. C. Murillo y J. J. Guerrero, "Optimization of Search Strategies for a Moving Target Using Cross-Entropy Methods", *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, págs. 2565-2570, 2012. DOI: [10.1109/IROS.2012.6385609](https://doi.org/10.1109/IROS.2012.6385609).
- [11] P. Lanillos, A. C. Murillo y J. J. Guerrero, "Bayesian Optimization Algorithm Applied to the Minimum Time Search Problem", *Robotics and Autonomous Systems*, vol. 61, n.º 10, págs. 1079-1091, 2013. DOI: [10.1016/j.robot.2013.06.003](https://doi.org/10.1016/j.robot.2013.06.003).
- [12] S. P. Carabaza, "Ant Colony Optimization for Minimum Time Search Problems", Ph.D. Thesis, Tesis doct., University of the Balearic Islands, 2021.
- [13] P. Lanillos, "Minimum time search of moving targets in uncertain environments", Tesis doct., Universidad Complutense de Madrid, jul. de 2013.

- [14] S. Pérez Carabaza, E. Besada Portas, J. A. López Orozco y G. Blanco Rodríguez, “Modelado y optimización de misiones de búsqueda de objetivos mediante UAVs”, spa, en *XL Jornadas de Automática: libro de actas. Ferrol, 4-6 de septiembre de 2019, 2019, ISBN 978-84-9749-716-9, págs. 574-581*, Section: XL Jornadas de Automática: libro de actas. Ferrol, 4-6 de septiembre de 2019, Servizo de Publicacións, 2019, págs. 574-581, ISBN: 978-84-9749-716-9. dirección: <https://dialnet.unirioja.es/servlet/articulo?codigo=7031368> (visitado 02-05-2024).
- [15] A. Papoulis y S. Pillai, *Probability, Random Variables, and Stochastic Processes* (McGraw-Hill electrical and electronic engineering series). McGraw-Hill, 2002, ISBN: 9780073660110. dirección: <https://books.google.es/books?id=YYwQAQAIAAJ>.
- [16] T. Furukawa, F. Bourgault, B. Lavis y H. Durrant-Whyte, “Recursive Bayesian search-and-tracking using coordinated uavs for lost targets”, en *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006.*, ISSN: 1050-4729, mayo de 2006, págs. 2521-2526. DOI: 10.1109/ROBOT.2006.1642081. dirección: <https://ieeexplore.ieee.org/document/1642081/?arnumber=1642081> (visitado 02-09-2024).
- [17] Wikipedia contributors, *Test functions for optimization*, https://en.wikipedia.org/wiki/Test_functions_for_optimization, Accessed: 19 November 2025, 2025.
- [18] M. Dorigo, M. Birattari y T. Stützle, “Ant colony optimization: a survey of the theoretical results”, *Theoretical Computer Science*, vol. 344, n.º 2-3, págs. 243-278, 2005.
- [19] A. A. Sari, “Recent research in finding the optimal path by ant colony optimization”, *Bulletin of Electrical Engineering and Informatics*, vol. 12, n.º 3, págs. 1505-1513, 2023.
- [20] D. Karaboga y B. Basturk, “A comprehensive survey: artificial bee colony (ABC) algorithm and applications”, *Applied Soft Computing*, vol. 12, n.º 1, págs. 684-710, 2012.
- [21] E. Kaya y A. Yildiz, “A review on the studies employing Artificial Bee Colony algorithm to solve combinatorial optimization problems”, *Computers & Operations Research*, vol. 142, pág. 105 704, 2022.
- [22] S. Q. Salih, A. N. Ahmed, Z. M. Yaseen et al., “Multi-population Black Hole Algorithm for optimization problems”, *PLOS ONE*, vol. 18, n.º 7, e0288044, 2023.
- [23] A. Raj y D. Singh, “Black Hole Optimization Algorithm – a novel metaheuristic approach”, *SN Applied Sciences*, vol. 2, n.º 2, pág. 282, 2020.



Universidad
de Alcalá

2026